

# WARPWARE

## WARPTWIN USER'S GUIDE

VERSION 26.09

SEPTEMBER 2026

# Contents

---

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                  | <b>7</b>  |
| 1.1      | What WarpTwin Is                     | 7         |
| 1.2      | What WarpTwin Is Not                 | 7         |
| 1.3      | What You Received                    | 8         |
| 1.4      | Two Ways to Build a Simulation       | 8         |
| 1.5      | How This Guide Is Organized          | 9         |
| 1.6      | Conventions Used in This Guide       | 9         |
| 1.6.1    | Typography                           | 9         |
| 1.6.2    | Units                                | 10        |
| 1.6.3    | Naming of dynamic quantities         | 10        |
| 1.6.4    | Accessing values                     | 10        |
| 1.7      | A Note on Authority                  | 11        |
| <b>2</b> | <b>Support Resources</b>             | <b>12</b> |
| 2.1      | This Guide and the Doxygen Reference | 12        |
| 2.2      | WarpAssist                           | 12        |
| 2.3      | Direct Support                       | 12        |
| 2.4      | Release Cadence                      | 12        |
| <b>3</b> | <b>Installing WarpTwin</b>           | <b>14</b> |
| 3.1      | What You Received                    | 14        |
| 3.2      | Choosing an Installation Path        | 15        |
| 3.3      | Verifying a Delivery                 | 15        |
| 3.4      | Installing with the Container Image  | 15        |
| 3.4.1    | First run                            | 16        |
| 3.4.2    | The launcher subcommands             | 16        |
| 3.4.3    | Your files                           | 17        |
| 3.4.4    | The 3D viewer                        | 17        |
| 3.4.5    | The GUI                              | 17        |
| 3.4.6    | Loading the image by hand            | 17        |
| 3.5      | Installing Natively on Ubuntu        | 17        |
| 3.5.1    | Requirements                         | 17        |
| 3.5.2    | Install                              | 18        |
| 3.5.3    | What gets installed                  | 18        |
| 3.5.4    | Air-gapped machines                  | 18        |
| 3.5.5    | Optional extras                      | 19        |
| 3.6      | Installing on macOS                  | 19        |
| 3.7      | Your License                         | 19        |
| 3.8      | Checking the Installation            | 20        |
| 3.9      | Your First Run                       | 20        |
| 3.10     | Updating WarpTwin                    | 20        |
| <b>4</b> | <b>A First Simulation</b>            | <b>22</b> |
| 4.1      | The Scenario                         | 22        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 4.2      | Building the Script                              | 22        |
| 4.2.1    | Imports                                          | 22        |
| 4.2.2    | The executive                                    | 22        |
| 4.2.3    | The Earth                                        | 23        |
| 4.2.4    | The spacecraft                                   | 23        |
| 4.2.5    | Logging                                          | 23        |
| 4.2.6    | Startup, then initial state                      | 24        |
| 4.2.7    | Run                                              | 24        |
| 4.3      | The Whole Script                                 | 24        |
| 4.4      | Running It                                       | 25        |
| 4.5      | What WarpTwin Wrote                              | 26        |
| 4.6      | Looking at the Data                              | 26        |
| 4.7      | Seeing It: Visuals                               | 27        |
| 4.8      | Changing the Scenario                            | 27        |
| 4.9      | Where to Go Next                                 | 27        |
| <b>5</b> | <b>The WarpTwin GUI</b>                          | <b>28</b> |
| 5.1      | Launching                                        | 28        |
| 5.2      | The Workspace                                    | 28        |
| 5.3      | The Simulation Executive Block                   | 28        |
| 5.4      | Adding and Connecting Blocks                     | 29        |
| 5.5      | Setting Values                                   | 29        |
| 5.6      | Build Validity                                   | 29        |
| 5.7      | Saving and Loading                               | 29        |
| 5.8      | Exporting a Script                               | 30        |
| 5.9      | Custom Models in the GUI                         | 31        |
| 5.10     | After a Run                                      | 31        |
| <b>6</b> | <b>Building Simulations in Python</b>            | <b>32</b> |
| 6.1      | The Skeleton                                     | 32        |
| 6.2      | The Executive                                    | 33        |
| 6.2.1    | Rate and integrator                              | 33        |
| 6.2.2    | Command-line arguments                           | 33        |
| 6.2.3    | The simulation epoch                             | 34        |
| 6.3      | Building the World                               | 34        |
| 6.3.1    | SpicePlanet                                      | 34        |
| 6.4      | The Spacecraft Model                             | 34        |
| 6.4.1    | Key parameters                                   | 35        |
| 6.4.2    | Key outputs                                      | 35        |
| 6.4.3    | Setting the initial state                        | 35        |
| 6.4.4    | Impulsive maneuvers                              | 36        |
| 6.4.5    | Reaching the wrapped models                      | 36        |
| 6.5      | The Launch Vehicle Model                         | 36        |
| 6.6      | Adding Models                                    | 37        |
| 6.7      | Wiring Models Together                           | 37        |
| 6.7.1    | Pointer configuration — set once, before startup | 38        |
| 6.7.2    | Signal connection — tracked every step           | 38        |
| 6.8      | Finding Signal Addresses                         | 38        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 6.9      | Logging and Visuals                            | 39        |
| 6.10     | Driving the Run                                | 39        |
| 6.11     | Guidance, Navigation, and Control              | 40        |
| 6.11.1   | Where the loop closes                          | 40        |
| 6.11.2   | The ADCS assembly                              | 40        |
| 6.11.3   | The building blocks                            | 41        |
| 6.12     | Structuring a Larger Script                    | 41        |
| 6.13     | Checklist                                      | 42        |
| <b>7</b> | <b>Simulation Architecture</b>                 | <b>43</b> |
| 7.1      | The Simulation Executive                       | 43        |
| 7.1.1    | What startup() does                            | 43        |
| 7.2      | Simulation Time                                | 44        |
| 7.3      | The Simulation Step                            | 44        |
| 7.4      | The Model Contract                             | 45        |
| 7.5      | Models and Apps                                | 45        |
| 7.5.1    | Scheduling                                     | 46        |
| 7.6      | Signals and Addresses                          | 46        |
| 7.6.1    | Connecting signals                             | 47        |
| 7.6.2    | Addresses                                      | 47        |
| 7.7      | The Frame Tree                                 | 47        |
| 7.8      | The Managers                                   | 47        |
| <b>8</b> | <b>The Dynamics Engine</b>                     | <b>49</b> |
| 8.1      | Kinematics and Dynamics                        | 49        |
| 8.2      | Frames                                         | 49        |
| 8.3      | Joints                                         | 50        |
| 8.4      | Bodies                                         | 50        |
| 8.5      | Nodes                                          | 51        |
| 8.6      | How a Model Joins the Dynamics                 | 51        |
| 8.7      | The Integration Loop                           | 52        |
| 8.8      | N-Body Gravity                                 | 52        |
| 8.8.1    | What is modelled                               | 52        |
| 8.8.2    | The formulation                                | 52        |
| 8.8.3    | Why the origin choice does not affect accuracy | 53        |
| 8.8.4    | Notes and limitations                          | 53        |
| <b>9</b> | <b>Vector, Matrix, and Attitude Math</b>       | <b>54</b> |
| 9.1      | Matrix                                         | 54        |
| 9.1.1    | Construction and element access                | 54        |
| 9.1.2    | Operations                                     | 54        |
| 9.1.3    | Arithmetic                                     | 55        |
| 9.2      | CartesianVector                                | 55        |
| 9.3      | Attitude Representations                       | 56        |
| 9.3.1    | Common operations                              | 56        |
| 9.3.2    | Composing and rotating                         | 56        |
| 9.4      | Constants                                      | 57        |

|           |                                     |           |
|-----------|-------------------------------------|-----------|
| <b>10</b> | <b>Built-in Model Reference</b>     | <b>58</b> |
| 10.1      | How to Use This Chapter             | 58        |
| 10.2      | Assemblies                          | 58        |
| 10.3      | Gravity and Orbital Dynamics        | 60        |
| 10.4      | Atmosphere, Drag, and Aerodynamics  | 61        |
| 10.5      | Other Environment                   | 62        |
| 10.6      | Actuators and Propulsion            | 63        |
| 10.7      | Sensors                             | 64        |
| 10.8      | Navigation, Guidance, and Control   | 66        |
| 10.9      | Power                               | 67        |
| 10.10     | Communications and Access           | 68        |
| 10.11     | State Initialization                | 69        |
| <b>11</b> | <b>Logging and Data Analysis</b>    | <b>70</b> |
| 11.1      | Loggers                             | 70        |
| 11.2      | Choosing What to Log                | 70        |
| 11.3      | Rates and the Log Manager           | 71        |
| 11.4      | Where the Output Goes               | 71        |
| 11.5      | CSV and HDF5 in Practice            | 71        |
| 11.5.1    | CSV                                 | 71        |
| 11.5.2    | HDF5                                | 71        |
| 11.6      | The JSON Companions                 | 71        |
| 11.7      | Loading Data in Python              | 72        |
| 11.7.1    | One run                             | 72        |
| 11.7.2    | A Monte Carlo set                   | 72        |
| 11.8      | Loading Data in MATLAB              | 72        |
| 11.9      | Analysis Helpers                    | 73        |
| 11.10     | Automatic Reports: AutoDocPy        | 73        |
| 11.11     | The Interactive Results Browser     | 74        |
| 11.12     | Stepping Through a Run: FrameViewer | 74        |
| <b>12</b> | <b>Visualization</b>                | <b>75</b> |
| 12.1      | MultiBodyViz: The 3D Viewer         | 75        |
| 12.1.1    | Turning it on                       | 75        |
| 12.1.2    | Browser requirements                | 75        |
| 12.1.3    | What the simulation writes          | 75        |
| 12.1.4    | Reopening the viewer                | 76        |
| 12.1.5    | Watching a run live                 | 76        |
| 12.1.6    | Appearance                          | 76        |
| 12.1.7    | Custom viewing frames               | 76        |
| 12.1.8    | Links, ground stations, and cameras | 77        |
| 12.1.9    | Sampling and trails                 | 77        |
| 12.1.10   | Navigating the viewer               | 77        |
| 12.1.11   | The world frame                     | 77        |
| <b>13</b> | <b>Monte Carlo and Dispersions</b>  | <b>78</b> |
| 13.1      | Declaring a Dispersion              | 78        |
| 13.2      | Using a Dispersion's Value          | 78        |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| 13.3      | Run Numbers and Seeds                                           | 79        |
| 13.4      | Running a Set                                                   | 80        |
| 13.5      | Recording the Dispersion Set                                    | 80        |
| 13.6      | Analyzing the Ensemble                                          | 80        |
| <b>14</b> | <b>Worked Examples</b>                                          | <b>82</b> |
| 14.1      | Start Here                                                      | 82        |
| 14.2      | Orbits and Maneuvers                                            | 82        |
| 14.3      | Payloads and Subsystems                                         | 83        |
| 14.4      | Advanced Dynamics                                               | 83        |
| 14.5      | Launch                                                          | 83        |
| 14.6      | Monte Carlo                                                     | 83        |
| 14.7      | Flight Software and Operations                                  | 84        |
| 14.8      | Custom Analysis                                                 | 84        |
| 14.9      | The Full List                                                   | 84        |
| <b>15</b> | <b>Building Custom Models</b>                                   | <b>85</b> |
| 15.1      | The Model Contract                                              | 85        |
| 15.2      | A Worked Example: Slope-Intercept                               | 85        |
| 15.2.1    | The header — <code>src/models/SlopeIntercept.h</code>           | 85        |
| 15.2.2    | The implementation — <code>src/models/SlopeIntercept.cpp</code> | 86        |
| 15.3      | The Workspace                                                   | 87        |
| 15.4      | Testing a Custom Model                                          | 88        |
| 15.5      | Using a Custom Model in Python                                  | 89        |
| 15.6      | Using a Custom Model in the GUI                                 | 89        |
| <b>16</b> | <b>Flight Software in the Loop</b>                              | <b>90</b> |
| 16.1      | WarpOS Apps as Models                                           | 90        |
| 16.1.1    | An app must be scheduled                                        | 91        |
| 16.2      | Deterministic Scheduling with the Table Scheduler               | 91        |
| 16.3      | Simulated Time                                                  | 92        |
| 16.4      | Telemetry and Commands                                          | 92        |
| 16.4.1    | Routing to a ground system                                      | 93        |
| 16.4.2    | Running in real time                                            | 93        |
| 16.5      | Driving Your Own Flight Software                                | 93        |
| 16.5.1    | Hardware in the loop                                            | 94        |
| 16.6      | Worked References                                               | 94        |
| <b>17</b> | <b>WarpLink Interfacing</b>                                     | <b>96</b> |
| 17.1      | The Simulation Control Channel                                  | 96        |
| 17.2      | Addressing Simulation Values                                    | 97        |
| 17.3      | The WarpTwin Side                                               | 97        |
| 17.3.1    | The listener is already there                                   | 97        |
| 17.3.2    | Binding a specific signal to a specific socket                  | 98        |
| 17.4      | Flight Software Telemetry and Commands                          | 98        |
| 17.5      | Configuring the Connection                                      | 98        |
| 17.5.1    | Sim Control                                                     | 99        |
| 17.6      | Under the Hood: the SIM Target Definition                       | 99        |

|          |                                                         |            |
|----------|---------------------------------------------------------|------------|
|          | 17.7 Troubleshooting the Channel . . . . .              | 100        |
| <b>A</b> | <b>Command-Line Arguments and Environment Variables</b> | <b>101</b> |
|          | A.1 Built-in Script Arguments . . . . .                 | 101        |
|          | A.2 Environment Variables . . . . .                     | 102        |
|          | A.3 Command-Line Tools . . . . .                        | 102        |
| <b>B</b> | <b>Troubleshooting</b>                                  | <b>103</b> |
|          | B.1 Installation . . . . .                              | 103        |
|          | B.2 Running Simulations . . . . .                       | 104        |
|          | B.3 Visuals . . . . .                                   | 104        |

# 1 Introduction

---

WarpTwin is WarpWare’s simulation engine for space mission design, guidance, navigation and control (GN&C) development, and flight-software test and verification. It combines a high-fidelity C++ dynamics and environment core with a Python scripting layer, and it can run *real* flight software against that simulated environment — WarpOS, WarpWare’s flight-software framework, has a ready-made integration, and your own in-house flight code can be driven the same way — so the software you fly can be exercised against a simulated vehicle, environment, and sensor suite before it ever reaches hardware.

This guide is the tutorial and reference for using WarpTwin. It assumes you have received a WarpTwin delivery from WarpWare and want to build, run, and analyze simulations with it. Every concept in this guide is taught by demonstration, using the models, the Python API, and the runnable examples that ship with your delivery.

## 1.1 What WarpTwin Is

At its core, WarpTwin propagates the six-degree-of-freedom (6-DOF) state of one or more rigid bodies — typically spacecraft and launch vehicles — through a modeled space environment, and lets you instrument, perturb, and analyze that propagation. Around that core it provides:

- **A library of built-in models** for gravity (point-mass, J2/J3 aspherical, spherical harmonics, and full n-body third-body perturbations), atmospheric drag (NRLMSISE-00), solar radiation pressure, actuators, sensors, power, communications, ground assets, and launch vehicles. Chapter 10 is the catalog.
- **High-precision ephemerides and time** through NAIF SPICE: planet, moon, and Sun states; UTC, TDB, GPS, and Julian date; and a configurable simulation epoch.
- **A graphical builder**, `wt-gui`, in which a simulation is assembled as a diagram of connected model blocks and exported to a Python script (Chapter 5).
- **A Python API** in which the same simulation is written directly as a script — the form that scales to Monte Carlo studies, custom GN&C, and integration with your own tools (Chapter 6).
- **Logging and analysis** to CSV and HDF5, with Python helpers for loading, plotting, and report generation (Chapter 11).
- **A 3D visualizer**, MultiBodyViz, served in a web browser and optionally streamed live during a run (Chapter 12).
- **Flight-software-in-the-loop** execution, running flight software under simulated time against the simulated environment — WarpOS apps through a ready-made integration, or your own flight code driven the same way (Chapter 16) — and **WarpLink**, which lets a ground operator drive a running simulation the way they would drive a real vehicle (Chapter 17).

## 1.2 What WarpTwin Is Not

WarpTwin is not a real-time operating system, a ground system, or a flight computer. It is an offline (or faster-than-real-time, or on request real-time-paced) simulation environment. It is not an orbit determination product: it propagates trajectories you define, it does not estimate them from tracking data. And it is not a black box — while you do not get the source, every model’s interface is documented, every value in the simulation has an address you can log or inspect, and the behavior

of the engine is described in this guide and in the Doxygen reference.

**Note.** The source code for an individual model or flight-software app can be provided on request through WarpWare support (Chapter 2) — for instance, when you are building a custom model and want a delivered one as a reference. It is not part of a standard delivery.

## 1.3 What You Received

A WarpTwin delivery is built for a specific platform (a particular Ubuntu release, or macOS for a particular Python version, or a portable container image — see Chapter 3). Whichever form you received, it installs the same components:

| Component                        | What it is                                                                                                                                                                                                        |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| wt-gui                           | The graphical simulation builder.                                                                                                                                                                                 |
| The warptwin Python package      | The simulation API: one importable module per model, plus warptwin.WarpTwinPy for the executive, loggers, math types, and constants.                                                                              |
| The warptwinutils Python package | Analysis and plotting helpers: HDF5 loading, Google Earth export, automatic report generation, the frame viewer.                                                                                                  |
| C++ headers                      | The interface to every built-in model and to the engine, under /usr/include/warptwin/ on a native install. These are the authoritative model documentation and the starting point for custom models (Chapter 15). |
| SPICE kernels and model data     | Planetary ephemerides, leap seconds, gravity fields, atmosphere tables, and the prebuilt 3D viewer.                                                                                                               |
| Example simulations              | Runnable scripts under /usr/share/doc/warptwin/examples/ (Chapter 14).                                                                                                                                            |
| Command-line tools               | warptwin-doctor (installation self-check), warptwin-license (license status and installation), warptwin-visuals (viewer launcher).                                                                                |

**Note.** NASA's Earth-GRAM atmosphere model is export controlled and WarpWare cannot redistribute it, so it is *not* part of any delivery. Atmospheric density, temperature, and speed of sound come from the built-in NRLMSISE-00 (MSIS) model instead.

WarpTwin requires a license file to run. Your delivery may include an evaluation license; a longer-term license is installed separately and is covered in Section 3.7.

## 1.4 Two Ways to Build a Simulation

There is no difference in the physics between a simulation built in the GUI and one written as a script — both assemble the same C++ models and run them through the same engine. The GUI is the fastest way to stand up a scenario and to learn what the models are; it writes out a Python script as you work. Scripting is where you go for anything the GUI cannot express: loops, parameter sweeps,

Monte Carlo, custom control logic, or calling WarpTwin from a larger workflow. A common path is to lay out the skeleton of a scenario in the GUI, export it, and then extend the script.

This guide teaches scripting as the primary interface (Chapters 4 and 6) and treats the GUI as a companion to it (Chapter 5).

## 1.5 How This Guide Is Organized

**Getting started** Chapter 3 installs WarpTwin. Chapter 4 walks a complete first simulation end to end – write it, run it, read the output, plot it, visualize it.

**The two interfaces** Chapter 5 covers the GUI. Chapter 6 covers the anatomy of a Python simulation script in depth.

**How the engine works** Chapter 7 (the executive, time, scheduling, the model contract, signals), Chapter 8 (frames, bodies, nodes, integration, n-body gravity), and Chapter 9 (the vector, matrix, and attitude math library).

**Reference** Chapter 10 catalogs every built-in model. Chapter 11 covers logging and analysis, Chapter 12 visualization, Chapter 13 Monte Carlo and dispersions.

**Going further** Chapter 14 is a guided tour of the shipped examples. Chapter 15 builds a custom model. Chapter 16 runs flight software in the loop. Chapter 17 connects a running simulation to the WarpLink ground platform.

**Appendices** Appendix A lists every command-line argument and environment variable. Appendix B is a troubleshooting reference.

If you are entirely new to WarpTwin, read Chapters 3 through 6 in order, then dip into the reference chapters as needed.

## 1.6 Conventions Used in This Guide

### 1.6.1 Typography

**Editable code** appears in a framed, syntax-highlighted block. Python:

```
exc = SimulationExecutive()
```

C++ and shell commands are shown the same way, labeled by language where it is not obvious from context.

**Terminal output** – what a command or script prints – appears in a plain framed block with no highlighting:

```
$ python3 script.py
Simulation complete. End time 10000.0 s.
```

**Worked examples** are collected in numbered boxes:

---

#### Example 1.1 – Reading an example box

A worked example states a scenario, builds the script for it, and shows what to expect when it runs. Examples are numbered per chapter (this is Example 1.1) and are referenced elsewhere in the guide by that number.

**Admonitions** call out three kinds of aside:

**Note.** A **note** adds context or a cross-reference that is useful but not essential on a first read.

**Tip.** A **tip** is a shortcut or a better way to do something you could do the long way.

**Warning.** A **warning** marks something that will cost you time or give you wrong results if you get it wrong — an ordering requirement, a silent failure mode, a units trap.

## 1.6.2 Units

WarpTwin is unit-agnostic internally: it does arithmetic on the numbers you give it. Every built-in model, every example, and this entire guide use **SI base units** consistently:

| Quantity          | Unit          | Notes                                                                                                 |
|-------------------|---------------|-------------------------------------------------------------------------------------------------------|
| Length / position | meter (m)     | Orbit semimajor axes are meters, not km.                                                              |
| Velocity          | m/s           |                                                                                                       |
| Time              | second (s)    | Simulation time; also a Time object type.                                                             |
| Angle             | radian (rad)  | Use the DEGREES_TO_RADIANS constant to convert. Latitude, longitude, inclination, RAAN — all radians. |
| Mass              | kilogram (kg) |                                                                                                       |
| Angular rate      | rad/s         |                                                                                                       |
| Force / torque    | N / N m       |                                                                                                       |
| Power             | watt (W)      |                                                                                                       |

**Warning.** The single most common first-time mistake is passing kilometers or degrees where WarpTwin expects meters or radians. A semimajor axis of 7000 is seven kilometers — deep inside the Earth — not a 622 km orbit.

## 1.6.3 Naming of dynamic quantities

Signals that carry a physical vector or attitude follow a consistent naming pattern borrowed from the WarpWare C++ style guide:

`descriptor_item_referencepoint__representation`

Read `pos_sc_pci` as “position of the spacecraft (sc) relative to the planet-centered inertial point, expressed in the PCI frame”. Where the reference frame and the representation frame are the same, the trailing `__representation` is dropped, as it is here. A name like `vel_tgt_ref__ref` is “velocity of the target relative to the reference frame, expressed in the reference frame”. You will see this pattern throughout the model outputs in Chapter 10.

## 1.6.4 Accessing values

Every configurable or observable quantity in a simulation is a *signal*. A signal is read by calling it with empty parentheses and written by calling it with a value:

```
mass = sc.params.mass()      # read
sc.params.mass(12.5)         # write (kg)
```

Signals are grouped into `params` (configuration, set once before the run), `inputs` (values that change during the run, usually driven by another model), and `outputs` (results the model produces). This is the *model contract*, and it is the same for every model in the library. It is described in full in Chapter 7.

## 1.7 A Note on Authority

This guide is the place to learn WarpTwin and to look things up quickly. When you need the last word — the exact set of a model's signals, an argument's precise meaning, a method's return type — the authority is the code you received: the C++ header for each model, and the Doxygen reference generated from those headers (Chapter 2). Where this guide and your delivered headers ever disagree, the headers are correct; please let WarpWare know so the guide can be fixed.

## 2 Support Resources

---

WarpWare's work does not end when a WarpTwin delivery goes out. Several resources are available to help your team succeed, from quick lookups to direct engineering support.

### 2.1 This Guide and the Doxygen Reference

This User's Guide is the tutorial and conceptual reference. Alongside it, the **Doxygen reference** is generated directly from the C++ headers you received and is the authoritative, exhaustive description of every class, method, and model signal. When you need the exact interface to a model, or a method signature this guide summarizes rather than quotes in full, the Doxygen pages are the place to look. They are published by WarpWare and are also included offline in your delivery under `/usr/share/doc/warptwin/`.

### 2.2 WarpAssist

**WarpAssist** is an AI assistant trained on the WarpTwin code base, this guide, and the accumulated support history. It is a good first stop for “how do I...” and “why is...” questions, and it is specific: the more precisely you describe your model configuration, the error you are seeing, or the analysis you are trying to do, the better its answer.

<https://assist.warpware.co/>

**Note.** WarpAssist is a starting point, not an authority. Confirm anything it tells you about a model's interface against the header or the Doxygen reference before you rely on it.

### 2.3 Direct Support

For anything the resources above do not resolve, email WarpWare directly:

[support@warpware.co](mailto:support@warpware.co)

When reporting a problem with an installed WarpTwin, include the output of `warptwin -doctor - -verbose` (Chapter 3); when reporting a simulation problem, include the script, the exact command you ran, and the terminal output.

This is also the channel for a request that is not a problem report — among them, the source code for a particular built-in model or flight-software app, which WarpWare can provide on request even though it is not part of a standard delivery.

The WarpWare website is <https://www.warpware.co/>.

### 2.4 Release Cadence

WarpTwin is released roughly quarterly. Releases are versioned `YY.MM-revision` — `26.09-0` is the first `26.09` release, cut in September 2026 — and WarpWare may issue point releases in

between. Whether to move to a new release is your team's decision; Chapter 3 covers updating an installation.

## 3 Installing WarpTwin

---

WarpTwin runs natively on Linux and on Apple macOS, and in a container anywhere Docker runs. It is delivered in three forms, and you install whichever one matches your situation:

- a **container image** that runs anywhere Docker runs — Windows, macOS, or any Linux distribution — and needs nothing else installed;
- **native Ubuntu packages**, built for a specific Ubuntu release, for running WarpTwin directly on a Linux workstation or server;
- a **macOS installer package**, built for a specific Python version, for running WarpTwin natively on Apple Silicon.

All three install the same WarpTwin — the same engine, the same `wt-gui`, the same Python API. The container image is in fact built *from* the native Ubuntu packages, so it cannot behave differently from a native install. This chapter covers verifying a delivery, all three installation paths, licensing, and confirming the result.

### 3.1 What You Received

A delivery arrives as a directory (usually inside a signed `.tar.gz`) named for the version and target, for example `warptwin-26.09-1-ubuntu22.04-amd64/`. It contains:

| File                                                                   | Purpose                                                                                                                                           |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>install.sh</code>                                                | The installer. Verifies the delivery, then installs it.                                                                                           |
| <code>README.md</code>                                                 | A short version of this chapter, specific to the build.                                                                                           |
| <code>attx-release.gpg</code>                                          | WarpWare's public release-signing key.                                                                                                            |
| <code>SHA256SUMS</code> , <code>SHA256SUMS.asc</code>                  | The checksum manifest and its OpenPGP signature.                                                                                                  |
| <code>warptwin_*.deb</code>                                            | The engine package (Ubuntu deliveries): the executive, <code>wt-gui</code> , the Python API, C++ headers, CLI tools. About 120 MB.                |
| <code>warptwin-data_*.deb</code>                                       | The data package (Ubuntu deliveries): SPICE kernels, model data, the prebuilt 3D viewer. About 270 MB, and identical across every Ubuntu release. |
| <code>warptwin-*.pkg</code>                                            | The installer package (macOS deliveries), one per Python version.                                                                                 |
| <code>warptwin-*-image.tar.gz</code> , <code>warptwin-docker.sh</code> | The container image archive and its launcher (included with the Ubuntu 22.04 delivery).                                                           |
| <code>MANIFEST.json</code>                                             | A record of exactly what was built and verified.                                                                                                  |

## 3.2 Choosing an Installation Path

| Your situation                                                        | Use                                                               |
|-----------------------------------------------------------------------|-------------------------------------------------------------------|
| Windows or macOS, and you would rather not set up a Linux environment | The <b>container image</b> (Section 3.4).                         |
| A Linux distribution other than the one WarpTwin was built for        | The <b>container image</b> .                                      |
| An Ubuntu workstation or server on the release WarpTwin was built for | The <b>native Ubuntu packages</b> (Section 3.5).                  |
| An Apple Silicon Mac, running WarpTwin natively (no container)        | The <b>macOS package</b> (Section 3.6).                           |
| A shared build server, CI, or a cluster node                          | Native packages if the OS matches; otherwise the container image. |

**Note.** The native Ubuntu packages are built *per Ubuntu release* because WarpTwin's Python extension modules and its HDF5 linkage are tied to that release's system libraries. `install.sh` checks this and refuses to install on the wrong release rather than fail confusingly later. If your Ubuntu version is not one WarpWare packages for, use the container image or ask WarpWare for a build targeting your release.

## 3.3 Verifying a Delivery

Every delivery is signed. `install.sh` verifies it automatically before installing anything, and aborts if verification fails. You can also verify by hand — do this if you received a `.deb` or `.pkg` on its own, or simply want to check before running the installer:

```
# 1. Import WarpWare's public release key into a scratch keyring.
gpg --import attx-release.gpg

# 2. Confirm the fingerprint it prints OUT OF BAND -- against
#    https://www.warpware.co/ or your signed contract. A fingerprint that
#    only appears inside the thing it is authenticating proves nothing.
gpg --fingerprint

# 3. Verify the signed checksum manifest, then the files it covers.
gpg --verify SHA256SUMS.asc SHA256SUMS      # must say: Good signature
sha256sum -c SHA256SUMS                     # must say: OK for every line
```

**Warning.** If any of these steps fails — a bad signature, a checksum mismatch, a fingerprint that does not match the one WarpWare publishes — **do not install**. Contact [support@warpware.co](mailto:support@warpware.co). A failure here means the delivery was corrupted in transit or tampered with.

## 3.4 Installing with the Container Image

This is the path that needs the least from your machine: only **Docker**. It works on Windows, macOS, and any Linux distribution. The image carries the same packages as the native `.deb` files, already

installed.

### 3.4.1 First run

From the delivery directory:

```
./warptwin-docker.sh          # opens a shell in the container
```

The first invocation loads the image from the archive next to the script (this takes a minute; it happens only once). You are then dropped into a shell *inside* the container, with your current directory on the host mounted at /work:

```
Loading WarpTwin 26.09 image (first run only, this takes a minute)
```

```
WarpTwin 26.09 -- /home/you/project is mounted at /work
```

```
Try: warptwin-doctor . python3 your_script.py . exit
```

### 3.4.2 The launcher subcommands

You do not have to work interactively. `warptwin-docker.sh` takes a subcommand:

| Command                                                | Effect                                                                           |
|--------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>./warptwin-docker.sh</code>                      | Interactive shell, current directory mounted at /work.                           |
| <code>./warptwin-docker.sh doctor</code>               | Run the installation self-check (Section 3.8).                                   |
| <code>./warptwin-docker.sh run script.py [args]</code> | Run one of your simulation scripts. The script must be in the mounted directory. |
| <code>./warptwin-docker.sh visuals</code>              | Serve the 3D viewer at <code>http://localhost:8002</code> .                      |
| <code>./warptwin-docker.sh gui</code>                  | Launch <code>wt-gui</code> (needs an X server — see below).                      |
| <code>./warptwin-docker.sh load</code>                 | Load the image and do nothing else.                                              |

Two options apply to any subcommand:

|                             |                                                                    |
|-----------------------------|--------------------------------------------------------------------|
| <code>--license FILE</code> | Use your own license file instead of the one baked into the image. |
| <code>--dir DIR</code>      | Mount DIR at /work instead of the current directory.               |

#### Example 3.1 — Running a simulation in the container

Copy an example out of the delivery, run it in the container, and look at the output — all from the host, with no WarpTwin installed on the host at all:

```
mkdir ~/first-run && cd ~/first-run
./path/to/delivery/warptwin-docker.sh run script.py
```

The script runs inside the container; `script.py` and everything it writes live in `~/first-run` on your machine, owned by you. If the script enables visuals, open `http://localhost:8002` in a WebGPU-capable browser while it runs.

### 3.4.3 Your files

The directory you launch from (or the one you pass to `--dir`) is mounted at `/work` inside the container, and that is where the shell starts and where `run` looks for your script. Edit scripts on your own machine with your own editor; run them in the container; the output — log files, JSON, plots — lands back in the same directory. On Linux the launcher runs the container as your user, so those files belong to you rather than to `root`.

### 3.4.4 The 3D viewer

The container publishes ports 8002 (the viewer) and 9002 (its live-state channel) to the host. When a simulation enables visuals, or when you run `./warptwin-docker.sh visuals`, open `http://localhost:8002` in a WebGPU-capable browser *on the host*. Nothing needs to be installed in the browser, and Node.js is not required anywhere.

### 3.4.5 The GUI

`wt-gui` draws through X11, which containers do not provide on their own:

**Linux** It works as-is, from a graphical session.

**Windows** Install [VcXsrv](#), start it allowing connections, and set `DISPLAY=host.docker.internal:0` before running `./warptwin-docker.sh gui`.

**macOS** Install [XQuartz](#) and enable *Allow connections from network clients* in its security settings.

The Python API, running simulations, and the 3D viewer need none of this — only `wt-gui` does.

### 3.4.6 Loading the image by hand

If you would rather not use the launcher:

```
gunzip -c warptwin-26.09-1-image.tar.gz | docker load
docker run --rm -it -v "$PWD:/work" -w /work -p 8002:8002 -p 9002:9002 \
    warpware/warptwin:26.09-1
```

## 3.5 Installing Natively on Ubuntu

### 3.5.1 Requirements

- The exact Ubuntu release the delivery was built for, on amd64.
- Network access during installation, so `apt` can fetch the stock Ubuntu dependencies (NumPy, h5py, HDF5, GLFW, Matplotlib, ...).
- A graphical session for `wt-gui`. The Python API and the simulation engine run headless.
- About 2 GB of free disk space.

### 3.5.2 Install

```
tar xzf warptwin-26.09-*.tar.gz
cd warptwin-26.09-*/
./install.sh
```

`install.sh` performs, in order:

1. **Platform check** — confirms this machine is the Ubuntu release and architecture the packages were built for, and stops with a clear message if not.
2. **Signature and checksum verification** — Section 3.3, done automatically against a scratch keyring so it never touches your real one.
3. **Installation** — `apt-get install` of both packages in one transaction, so dependencies resolve.
4. **Self-check** — runs `warptwin-doctor` and reports what works.

To install a license at the same time:

```
./install.sh --license /path/to/your/licensefile.enc
```

Other options:

|                            |                                                                              |
|----------------------------|------------------------------------------------------------------------------|
| <code>--verify-only</code> | Verify the delivery and stop. Install nothing.                               |
| <code>--no-verify</code>   | Skip signature checks (not recommended).                                     |
| <code>--uninstall</code>   | Remove WarpTwin. Your license file and simulation outputs are left in place. |
| <code>-y, --yes</code>     | Do not prompt.                                                               |

### 3.5.3 What gets installed

| Path                                                       | Contents                                  |
|------------------------------------------------------------|-------------------------------------------|
| <code>/usr/bin/wt-gui</code>                               | The graphical simulation builder.         |
| <code>/usr/bin/warptwin-doctor</code>                      | Installation self-check.                  |
| <code>/usr/bin/warptwin-license</code>                     | License installation and status.          |
| <code>/usr/bin/warptwin-visuals</code>                     | 3D viewer launcher.                       |
| <code>/usr/lib/python3/dist-packages/warptwin/</code>      | The Python simulation API.                |
| <code>/usr/lib/python3/dist-packages/warptwinutils/</code> | Analysis and plotting helpers.            |
| <code>/usr/include/warptwin/</code>                        | C++ headers.                              |
| <code>/usr/share/warptwin/</code>                          | SPICE kernels, model data, the 3D viewer. |
| <code>/usr/share/doc/warptwin/examples/</code>             | Runnable example simulations.             |

Because `warptwin` installs into the system Python path, it is importable from any script, any directory, and any interpreter session — there is no environment variable to set and no activation step.

### 3.5.4 Air-gapped machines

`install.sh` needs `apt` to reach the Ubuntu archive for dependencies. On a machine with no network access, pre-stage the dependency list — it is printed in the delivery's `README.md` under *Requirements* — then run `install.sh`.

### 3.5.5 Optional extras

Neither is packaged by Ubuntu, so neither is installed automatically:

- **The interactive results browser** that `wt -gui` launches after a run:

```
pip install -r /usr/share/warptwin/python/plot-app/requirements.txt
```

- **Static (PNG) plot export** from `warptwinutils.AutoDocPy`: `pip install kaleido`. Interactive HTML export needs nothing extra.

## 3.6 Installing on macOS

The macOS delivery is an installer package (`.pkg`), one per Python minor version, for Apple Silicon (arm64) only. You do *not* need Homebrew — the package carries its own dependencies.

```
tar xzf warptwin-26.09-*-macos.tar.gz
cd warptwin-26.09-*-macos/
./install.sh
```

`install.sh` verifies the delivery (Section 3.3), confirms the target Python version matches the interpreter on your machine, and then installs the package. As on Ubuntu, add `--license /path/to/licensefile.enc` to install a license at the same time, or `--verify-only` to check without installing.

**Note.** If the delivery was signed and notarised by WarpWare, it installs with no further prompts. If it was delivered unsigned (a supported option for direct delivery to known recipients), `install.sh` still installs it cleanly — installer performs no Gatekeeper assessment. Only *double-clicking* an unsigned `.pkg` in Finder after a browser download is blocked; in that case, right-click the package and choose *Open*. The OpenPGP signature check in `install.sh` is what establishes the package is genuinely from WarpWare, signed or not.

After installation the CLI tools (`wt-gui`, `warptwin-doctor`, `warptwin-license`, `warptwin-visuals`) are on your PATH and `import warptwin` works from the matching Python interpreter.

## 3.7 Your License

WarpTwin requires a license file to run. In a release build the check runs inside the simulation executive's startup(): with no valid license, a simulation exits immediately, before it does any work.

Your delivery may have been built with a license already embedded — an EVALUATION license (90 days) unless WarpWare built it for your organization specifically. To see what you have:

```
warptwin-license status
```

To install a license file you received separately:

```
sudo warptwin-license install /path/to/licensefile.enc
```

It is written to `/etc/warptwin/licensefile.enc`. A license you install this way **takes precedence over any embedded one and survives package upgrades and removal**, so you install your real license once and it persists across quarterly updates. In the container image, pass your license per run with `./warptwin-docker.sh --license FILE ...`.

To renew, contact [support@warpware.co](mailto:support@warpware.co).

## 3.8 Checking the Installation

`warptwin-doctor` is the self-check. Run it any time:

---

|                                        |                                                                                           |
|----------------------------------------|-------------------------------------------------------------------------------------------|
| <code>warptwin-doctor</code>           | Check that the installation is healthy. Names each failing check and what to do about it. |
| <code>warptwin-doctor --full</code>    | ... and run a real simulation end to end.                                                 |
| <code>warptwin-doctor --verbose</code> | ... with the underlying detail. This is the output to send to support.                    |

---

The two most common findings are a missing or expired license (check `warptwin-license status`) and a machine with no graphics stack (`wt-gui` needs a display; the Python API and the engine do not).

## 3.9 Your First Run

```
warptwin-doctor --full      # confirm everything works, including a sim
wt-gui                      # the graphical builder (needs a desktop
  ↪ session)

# Run a shipped example
cp -r /usr/share/doc/warptwin/examples/simple_gravity_spacecraft ~/
cd ~/simple_gravity_spacecraft
python3 script.py
```

And from your own Python, in any directory:

```
import warptwin
from warptwin.WarpTwinPy import SimulationExecutive, Hdf5Logger,
  ↪ DEGREES_TO_RADIANS
from warptwin.SpicePlanet import SpicePlanet
from warptwin.Spacecraft import Spacecraft

print(warptwin.__version__)      # the version you installed
```

Chapter 4 builds that first simulation properly, line by line.

## 3.10 Updating WarpTwin

WarpTwin is released quarterly (Chapter 2). To move to a new release:

**Native install** Download the new delivery and run its `install.sh`. `apt` upgrades the packages in place. A license you installed with `warptwin-license` carries over untouched.

**Container image** Load the new image archive (`./warptwin-docker.sh load`, or `gunzip -c ... | docker load`). Each release is a distinct image tag, so the new one does not disturb the old — point your `warptwin-docker.sh` at the new delivery directory.

Whether to update is your team's decision; a simulation script written against one release will generally run unchanged against the next, but review the release notes for model changes that affect your results.

# 4 A First Simulation

---

This chapter builds one complete simulation from an empty file: a spacecraft in low Earth orbit, propagated for a few hours, with its trajectory logged to disk and then plotted. It is deliberately small — every line is explained — and everything after it in the guide is an elaboration of the same shape.

The finished script is the shipped `simple_gravity_spacecraft` example; if you would rather read the end result first, copy that out of your delivery (Chapter 3) and run it.

## 4.1 The Scenario

A single spacecraft orbits the Earth. Its orbit is eccentric and inclined — the elements below put the semimajor axis about 900 km above the surface at  $e = 0.1$ , so a perigee near 170 km and an apogee near 1,600 km — so that the altitude, latitude, and longitude all visibly change over a run. We propagate for ten hours of simulation time, recording the spacecraft's inertial position and velocity and its altitude once per simulated second, and then load the log in Python and plot the altitude history.

There is no thrust, no attitude control, and no sensor in this first simulation. What exercises the engine is gravity: the Earth's point-mass attraction plus its J2 and J3 oblateness terms, which the `Spacecraft` model includes automatically.

## 4.2 Building the Script

### 4.2.1 Imports

Every model is its own importable module; the executive, the loggers, the math types, and the physical constants come from `warptwin.WarpTwinPy`.

```
import sys

from warptwin.WarpTwinPy import SimulationExecutive, Hdf5Logger,
    ↪ DEGREES_TO_RADIANS
from warptwin.SpicePlanet import SpicePlanet
from warptwin.Spacecraft import Spacecraft
```

### 4.2.2 The executive

The `SimulationExecutive` is the object that owns the simulation: time, the schedule, the dynamics engine, logging, and visuals all hang off it. By convention it is named `exc`.

```
exc = SimulationExecutive()
exc.parseArgs(sys.argv)      # let --end, --out-dir, etc. be set on the
    ↪ command line
exc.setRateHz(1)            # advance the simulation in 1-second steps
```

`parseArgs` hands the executive the command-line arguments so that options like `--end` and `--out-dir` (Appendix A) take effect. `setRateHz(1)` sets the step size to one second; you could

equally call `exc.setRateSec(0.5)` for half-second steps. The default integrator is fourth-order Runge–Kutta, which is what you want here.

### 4.2.3 The Earth

```
earth = SpicePlanet(exc, "earth")
```

`SpicePlanet` is a planet, moon, or the Sun whose position, velocity, and orientation come from NASA SPICE ephemerides. The second argument is the SPICE body name. Creating it is all that is required — it registers itself with the executive as a gravitating body, and it publishes an inertial frame and a body-fixed rotating frame that other models attach to. Its physical constants (radius, gravitational parameter, J2, J3) default to WGS-84 Earth values.

### 4.2.4 The spacecraft

```
sc = Spacecraft(exc, "sc")
sc.params.planet_ptr(earth)
```

`Spacecraft` is a convenience assembly: a rigid body plus a bundle of environment models — aspherical (J2/J3) gravity, gravity-gradient torque, planet-relative state (latitude / longitude / altitude), third-body perturbations from any other bodies in the scene, and, for Earth, MSIS atmospheric drag. You wire it to the body it orbits by setting the `planet_ptr` parameter.

**Warning.** The parameter is `sc.params.planet_ptr(earth)`. Some older material refers to a `sc.configFromPlanet(earth)` method — there is no such method. Set the `planet_ptr` parameter.

### 4.2.5 Logging

A logger writes chosen signals to a file at a fixed rate. There are two: `Hdf5Logger` (compact binary, exact values — the recommended default) and `CsvLogger` (plain text). The pattern is the same for both: create it, add the parameters you want, hand it to the log manager with a rate.

```
states = Hdf5Logger(exc, "sc_states.h5")
states.addParameter(exc.time().base_time, "sim_time")
states.addParameter(sc.outputs.pos_sc_pci, "pos_eci_m")
states.addParameter(sc.outputs.vel_sc_pci, "vel_eci_mps")
states.addParameter(sc.outputs.altitude_detic, "altitude_m")
exc.logManager().addLog(states, 1) # log at 1 Hz
```

Each `addParameter` call takes the signal to record and the column name to record it under. `exc.time().base_time` is the running simulation clock; `sc.outputs.pos_sc_pci` is the spacecraft position in the planet-centered inertial frame; `altitude_detic` is the geodetic altitude in meters. The final `addLog(states, 1)` registers the logger to run once per simulated second.

**Note.** You can also add a parameter by its address string, e.g. `states.addParameter(".exc.sc.outputs.pos_sc_pci", "pos_eci_m")`. The object form is easier to write and to check; the address form is useful when the signal is buried inside a

sub-model. Chapter 11 covers both.

#### 4.2.6 Startup, then initial state

```
exc.startup()

sc.initializeFromOrbitalElements(
    7278137.0,          # semimajor axis, m
    0.1,               # eccentricity
    DEGREES_TO_RADIANS * 40.0, # inclination
    DEGREES_TO_RADIANS * 10.0, # right ascension of the ascending node
    DEGREES_TO_RADIANS * 10.0, # argument of perigee
    DEGREES_TO_RADIANS * 30.0, # true anomaly
)
```

`exc.startup()` initializes every model in the simulation: it elects the central body, places the planet frames at the simulation epoch, and lets each model do its one-time setup. **The spacecraft's initial state is set after startup**, because it is defined relative to the planet frames that startup just placed. `initializeFromOrbitalElements` takes the six classical elements — semimajor axis in meters, the four angles in radians — and positions the spacecraft on that orbit about its `planet_ptr`. (If you need the resulting state vector as a signal to wire elsewhere rather than applied to a spacecraft, `OrbitalElementsStateInit` in Chapter 10 is the model form of the same computation.)

**Warning.** Order matters: `planet_ptr` and all params are set *before* `startup()`; the initial state is set *after*. A state initialized before startup is overwritten; a parameter changed after startup has undefined effect.

#### 4.2.7 Run

```
exc.run()
```

`run()` steps the simulation from the current time to the end time (default 10 000 s; overridable with `--end`), advancing time, evaluating every model each step, integrating the dynamics, and flushing the logger at its rate.

### 4.3 The Whole Script

#### Example 4.1 — A first simulation — Earth orbit, propagated and logged

```
import sys

from warptwin.WarpTwinPy import SimulationExecutive, Hdf5Logger,
    DEGREES_TO_RADIANS
from warptwin.SpicePlanet import SpicePlanet
from warptwin.Spacecraft import Spacecraft
```

```
exc = SimulationExecutive()
exc.parseArgs(sys.argv)
exc.setRateHz(1)

earth = SpicePlanet(exc, "earth")

sc = Spacecraft(exc, "sc")
sc.params.planet_ptr(earth)

states = Hdf5Logger(exc, "sc_states.h5")
states.addParameter(exc.time().base_time, "sim_time")
states.addParameter(sc.outputs.pos_sc_pci, "pos_eci_m")
states.addParameter(sc.outputs.vel_sc_pci, "vel_eci_mps")
states.addParameter(sc.outputs.altitude_detic, "altitude_m")
exc.logManager().addLog(states, 1)

exc.startup()

sc.initializeFromOrbitalElements(
    7278137.0, 0.1,
    DEGREES_TO_RADIANS * 40.0,
    DEGREES_TO_RADIANS * 10.0,
    DEGREES_TO_RADIANS * 10.0,
    DEGREES_TO_RADIANS * 30.0,
)

exc.run()
```

## 4.4 Running It

```
python3 script.py
```

On startup WarpTwin prints its banner and the arguments in effect — the ones you passed, then the defaults:

```
#####
# Copyright (c) ATTX INC 2026. All Rights Reserved.
# ...
#####
SET ARGUMENTS:
(Default Values):
    end:          10000
    run:          0
    out-dir:      results/
    rng-seed:     0
    write-data-json: 1
    save-all-outputs: 0
```

```
enable-visuals: 0
real-time: 0
```

and when the run finishes it logs a completion line:

```
Simulation run completed successfully.
Executed 10000 seconds of sim time at 480 times real time.
```

(The “times real time” figure depends on your machine.) To propagate for a different duration without editing the script, pass `--end` in seconds:

```
python3 script.py --end=36000          # ten hours
```

## 4.5 What WarpTwin Wrote

Everything lands in the output directory — `results/` by default, changeable with `--out-dir`:

|                                      |                                                                                                                                                               |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>results/sc_states.h5</code>    | The log: one dataset per parameter you added, sampled at 1 Hz.                                                                                                |
| <code>results/sim_data.json</code>   | Every model's parameters and configuration, as run. A record of exactly what you simulated.                                                                   |
| <code>results/graph_tree.json</code> | Every addressable value in the simulation, with its dotted address. Useful for finding the address of a signal to log or to drive from WarpLink (Chapter 17). |

The two JSON files are written because the `--write-data-json` argument defaults on; set it to `false` to suppress them.

## 4.6 Looking at the Data

`warptwinutils` loads an HDF5 log straight into a pandas DataFrame:

```
import matplotlib.pyplot as plt
from warptwinutils.analysisutils import readH5Dataframe

df = readH5Dataframe("results/sc_states.h5")

plt.plot(df["sim_time"] / 60.0, df["altitude_m"] / 1000.0)
plt.xlabel("time [min]")
plt.ylabel("altitude [km]")
plt.title("First simulation -- altitude history")
plt.savefig("altitude.png", dpi=150)
```

The altitude traces out the eccentric orbit — rising to apogee, falling back to perigee — with a slow secular drift from the J2 term superimposed on it. The shipped example's `analysis.py` goes further, exporting the ground track to a Google Earth KML with `plotGoogleEarth`. Loading and analyzing logs is the subject of Chapter 11.

## 4.7 Seeing It: Visuals

You do not need to touch the script. Pass `--enable-visuals=true` on the command line and re-run:

```
python3 script.py --enable-visuals=true
```

When the run finishes, the 3D viewer opens in your browser at `http://localhost:8002`, showing the Earth and the spacecraft's trajectory. Under the container image, the port is already published; run the script with `./warptwin-docker.sh run script.py --enable-visuals=true` and open the same URL.

Keeping visuals on the command line rather than in the script is the recommended pattern: the same script then runs headless in a batch or Monte Carlo job and interactively with visuals, with no edit in between. If you do want a script that always opens the viewer, add `exc.enableVisuals()` before `startup()` instead. Visualization is covered in Chapter 12.

## 4.8 Changing the Scenario

A few small edits show how the pieces move:

**A different orbit** Change the arguments to `initializeFromOrbitalElements`. Remember: meters and radians.

**A longer run** `python3 script.py --end=86400` for a full day, no edit needed.

**Add the Moon and Sun** Create `SpicePlanet(exc, "moon")` and `SpicePlanet(exc, "sun")`. Nothing else changes — the spacecraft automatically picks up their third-body perturbations (Chapter 8). Over a long run you will see the difference in the altitude history.

**Log more** Add `sc.outputs.latitude_detic` and `sc.outputs.longitude` (both radians) to the logger and plot the ground track.

**Give it mass properties** `sc.params.mass(250.0)` and an inertia tensor, before `startup()`, to make the gravity-gradient torque meaningful.

## 4.9 Where to Go Next

- Chapter 6 takes the script apart properly: connecting models to each other, adding actuators and sensors, driving the simulation step by step, and structuring a larger script.
- Chapter 7 explains what `startup()` and `run()` actually do — the schedule, the sim steps, the model contract.
- Chapter 14 walks the shipped examples, from this one up to launch-vehicle ascent and multi-spacecraft rendezvous.

# 5 The WarpTwin GUI

`wt-gui` is the graphical simulation builder. A simulation is assembled on a canvas as a diagram of model blocks joined by typed wires, checked for validity as you build, and written out as a Python script. It is the fastest way to stand up a scenario and to learn what the model library contains; anything the GUI cannot express is done by editing the exported script (Chapter 6), which is the same script the GUI runs.

## 5.1 Launching

Run `wt-gui` from a terminal in the directory you want the simulation's files to live in — the exported script, the save file, and the run output all land in the working directory. It needs a graphical session (a `DISPLAY`, or a `WAYLAND_DISPLAY` with Xwayland available); on a headless machine it does nothing (Appendix B).

---

|                                                     |                                             |
|-----------------------------------------------------|---------------------------------------------|
| <code>wt-gui</code>                                 | Start with an empty canvas or an example.   |
| <code>wt-gui --custom-model-file=models.json</code> | Also load your custom models (Section 5.9). |
| <code>wt-gui --startup-gui-file=sim.json</code>     | Open a saved simulation immediately.        |

---

Under the container image the GUI is reached through `./warptwin-docker.sh gui`, which needs an X server on the host (Chapter 3). On launch, a **Start** dialog lists the installed examples and the saved simulations in the current folder, or you can begin from a blank canvas.

## 5.2 The Workspace

The window has three regions.

**The sidebar** holds **Save sim** and **Load sim**, the build-validity band (Section 5.6), and the **Models** palette: a search box over a set of collapsible category drawers. The categories, in order, are Dynamics, GN&C, Assemblies, Environment, Power, Sensors, Actuators, Initial States, Logging, and Custom; a category with no models in your delivery is hidden. Typing in the search box replaces the drawers with a flat list matched on display name, class name, or category. Visualization models are not in the palette — the 3D viewer runs automatically from a checkbox on the executive block, not as a block you place (Chapter 12).

**The canvas** is the node graph: scroll to pan, wheel to zoom. Right-clicking empty canvas offers **Leave a Comment**, a free-standing note that is saved with the diagram but never affects the simulation.

**The inspector** appears when a block is selected. It shows the block's name (editable), its model class, its description, and every parameter and input with a value field you can edit. Outputs are listed read-only. An input that is fed by a wire shows (driven by link) instead of a field.

## 5.3 The Simulation Executive Block

Every diagram has exactly one **Simulation Executive** block. It is placed for you and cannot be deleted. Its parameters are the run's start time, end time, and step size; its outputs are the simulation root

frame, simulation time, and TDB time. It also carries the **Run Simulation** and **Stop Simulation** buttons and a status line that reads `Running simulation...` and then `Simulation complete`, `Simulation stopped`, or `Simulation ERROR`.

## 5.4 Adding and Connecting Blocks

Click a palette entry to drop that model on the canvas. Right-clicking a block (other than the executive) offers **Delete Node**, **Collapse / Expand Node**, and **Configure From Defaults** — which loads a named preset shipped with the model, if it has one.

Parameters and inputs are drawn on the left of a block, outputs on the right. Only the signals a model's author marked for wiring are drawn as connectable pins; every other value is set in the inspector. Drag from an output pin to an input pin to connect them. Pins are colored by data type, and the editor enforces the connection rules:

---

|                      |                                                                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Output to input only | Two inputs, or two outputs, cannot be joined.                                                                                                                                                        |
| Different blocks     | A block cannot wire to itself.                                                                                                                                                                       |
| One wire per input   | An input takes a single upstream source; an output may fan out to many.                                                                                                                              |
| Matching type        | The two ends must carry the same data type (Numeric, String, Pointer, Vector3, Quaternion, Frame Pointer, Matrix, Array, Time, <code>std::vector</code> ), unless one end's type is still Undefined. |

---

A drag the editor will not accept shows `Cannot connect` and the reason — `Type mismatch: Vector3 -> Numeric`, `Input already connected`, and so on.

## 5.5 Setting Values

Scalar and small-array values — a mass, a name, an initial velocity, an inertia matrix, a time — are typed into the inspector, with an editor appropriate to the type. A value you enter is marked as user-set and is written into the exported script. Pointer-valued parameters — the planet a spacecraft orbits, a body, a reference frame — are not typed; they are set by wiring the producing block's output into them.

## 5.6 Build Validity

A short moment after you stop editing, the GUI writes the script and runs it through a zero-length step. The sidebar band then reads **Build OK** in green — the simulation will run — or **Build failed** in red, with the reason shown beneath it (a required value left unset, an unconnected planet pointer, a value that will not parse). The reason is truncated in the band and shown in full on hover, and is also written to the log. The check is skipped while a simulation is running.

## 5.7 Saving and Loading

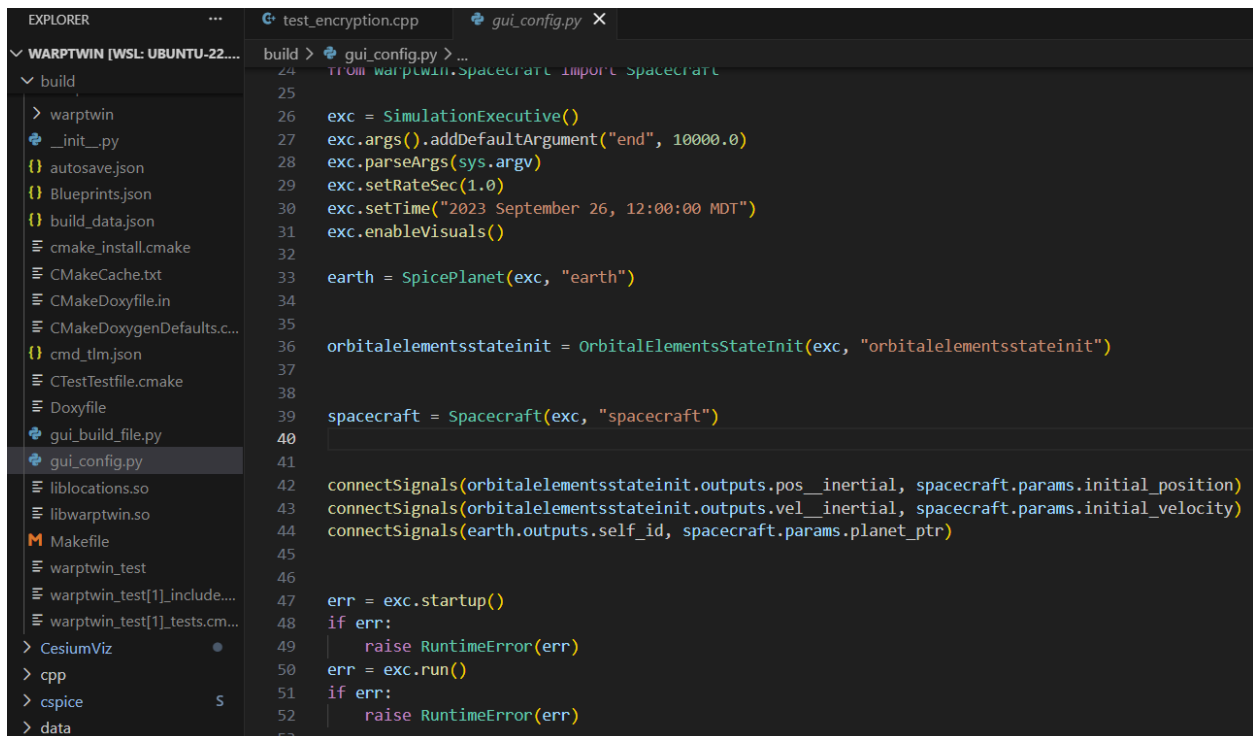
**Save sim** writes a `.json` file into the working directory, along with a Title and Description that are stored inside the file so it stays self-describing wherever it is copied. **Load sim** lists the saved simulations in the current folder and the installed examples.

The save file is a JSON array of block objects — each block's class, its canvas position, and every configured value — plus optional metadata and comment entries. A malformed file is rejected whole rather than loaded half-way, so a bad file cannot leave the editor in a broken state.

**Warning.** `autosave.json` is written automatically a moment after each edit. It is a crash-recovery net, not your save file: it is kept out of the Load list, and the next `wt -gui` session overwrites it. Use **Save sim** to keep a simulation.

## 5.8 Exporting a Script

The GUI keeps a `gui_config.py` in the working directory, rewritten on every build check and every run. It is an ordinary WarpTwin script: each block becomes a model constructor, each wire becomes a `connectSignals` call (pointer parameters included), each value you set becomes a `params` or `inputs` assignment, and the loggers become the logging setup. Running from the GUI simply executes this file.



```

EXPLORER
WARPTWIN [WSL: UBUNTU-22...
  build
    > warptwin
    > _init_.py
    > autosave.json
    > Blueprints.json
    > build_data.json
    > cmake_install.cmake
    > CMakeCache.txt
    > CMakeDoxyfile.in
    > CMakeDoxygenDefaults.c...
    > cmd_tlm.json
    > CTestTestfile.cmake
    > Doxyfile
    > gui_build_file.py
    > gui_config.py
    > liblocations.so
    > libwarptwin.so
    > Makefile
    > warptwin_test
    > warptwin_test[1].include...
    > warptwin_test[1].tests.cm...
  > CesiumViz
  > cpp
  > cspice
  > data

build > gui_config.py > ...
24 from warptwin.Spacecraft import Spacecraft
25
26 exc = SimulationExecutive()
27 exc.args().addDefaultArgument("end", 10000.0)
28 exc.parseArgs(sys.argv)
29 exc.setRateSec(1.0)
30 exc.setTime("2023 September 26, 12:00:00 MDT")
31 exc.enableVisuals()
32
33 earth = SpicePlanet(exc, "earth")
34
35 orbitalelementsstateinit = OrbitalElementsStateInit(exc, "orbitalelementsstateinit")
36
37 spacecraft = Spacecraft(exc, "spacecraft")
38
39
40
41
42 connectSignals(orbitalelementsstateinit.outputs.pos_inertial, spacecraft.params.initial_position)
43 connectSignals(orbitalelementsstateinit.outputs.vel_inertial, spacecraft.params.initial_velocity)
44 connectSignals(earth.outputs.self_id, spacecraft.params.planet_ptr)
45
46
47 err = exc.startup()
48 if err:
49     raise RuntimeError(err)
50 err = exc.run()
51 if err:
52     raise RuntimeError(err)
53
  
```

Figure 5.1: `gui_config.py`, generated from the diagram: model constructors, then a `connectSignals` call for every wire, then `startup()` and `run()`. This is the bridge to Chapter 6 — open it and extend it by hand for anything the GUI cannot express.

If the working directory is not writable, the GUI falls back to a temporary location for the generated script rather than failing.

## 5.9 Custom Models in the GUI

Launch with `wt-gui --custom-model-file=custom_models.json`, pointing at the meta-data file your custom-model workspace produces (Chapter 15). Your models then appear under the **Custom** category, drawn with the display name, category, and signal aliases from the `imdata` block in each model's header. They are added, wired, configured, and exported exactly like built-in models.



Figure 5.2: A custom model in the palette and on the canvas. The block's display name and its Offset/Multiplier/Input/Output pin labels come from the `imdata` and `aliases` comment block in the model's header.

## 5.10 After a Run

A successful run opens the interactive results browser (Chapter 11) on the output directory. **Stop Simulation** ends the run and closes the browser. The browser's additional Python dependencies are covered with the rest of the analysis tooling in Chapter 11.

# 6 Building Simulations in Python

Chapter 4 built one simulation. This chapter is the reference for building any simulation in Python: the fixed skeleton every script follows, the executive's controls, how models are created and wired to each other, and how a run is driven. Chapter 7 explains *why* the skeleton has the shape it does; this chapter is about writing it.

## 6.1 The Skeleton

Almost every WarpTwin script has the same seven parts in the same order. The order is not stylistic — several steps depend on earlier ones having happened.

1. **Imports** — `warptwin.WarpTwinPy` for the executive, loggers, math types and constants; one module per model.
2. **Create the executive** and set its rate, integrator, and (optionally) default arguments; then `parseArgs`.
3. **Create the world** — planets, the Sun, moons.
4. **Create the vehicles and models** — spacecraft, sensors, actuators, GN&C — and set their params.
5. **Wire models together** — pointer configuration and signal connections.
6. **Set up logging and visuals**.
7. **`startup()`**, then initial states, then **`run()`**.

### Example 6.1 — The skeleton, annotated

```
import sys
from warptwin.WarpTwinPy import SimulationExecutive, Hdf5Logger,
    ↪ DEGREES_TO_RADIANS
from warptwin.SpicePlanet import SpicePlanet
from warptwin.Spacecraft import Spacecraft

# 2. Executive
exc = SimulationExecutive()
exc.setRateHz(10)
exc.args().addDefaultArgument("end", 5400)      # BEFORE parseArgs
exc.parseArgs(sys.argv)

# 3. World
earth = SpicePlanet(exc, "earth")

# 4. Vehicles and models
sc = Spacecraft(exc, "sc")
sc.params.planet_ptr(earth)
sc.params.mass(200.0)

# 5. Wiring -- (nothing to wire in this minimal case)

# 6. Logging and visuals
log = Hdf5Logger(exc, "states.h5")
```

```
log.addParameter(exc.time().base_time, "t")
log.addParameter(sc.outputs.pos_sc_pci, "pos")
exc.logManager().addLog(log, 1)

# 7. Startup -> initial state -> run
exc.startup()
sc.initializeFromOrbitalElements(6778137.0, 0.001,
                                DEGREES_TO_RADIANS * 51.6, 0.0, 0.0, 0.0)
exc.run()
```

## 6.2 The Executive

### 6.2.1 Rate and integrator

| Operation                    | C++                                   | Python                                   |
|------------------------------|---------------------------------------|------------------------------------------|
| Create the executive         | <code>SimulationExecutive exc;</code> | <code>exc = SimulationExecutive()</code> |
| Set the step size in hertz   | <code>exc.setRateHz(10);</code>       | <code>exc.setRateHz(10)</code>           |
| Set the step size in seconds | <code>exc.setRateSec(0.1);</code>     | <code>exc.setRateSec(0.1)</code>         |
| Choose the integrator        | <code>exc.integrator(4);</code>       | <code>exc.integrator(4)</code>           |
| Set the end time (s)         | <code>exc.end(5400.0);</code>         | <code>exc.end(5400.0)</code>             |

The integrator is 1 for forward Euler (one derivative evaluation per step) or 4 for fourth-order Runge–Kutta (four per step, the default). RK4 is the right choice for orbital dynamics; forward Euler is occasionally useful for debugging or for a model that is being driven at a very high rate.

The step size (`setRateHz` / `setRateSec`) is how far simulated time advances per step. Smaller steps cost more but resolve fast dynamics — attitude control, thruster pulses, launch ascent — more accurately. A pure orbit propagation is happy at 1 Hz or slower.

### 6.2.2 Command-line arguments

`exc.parseArgs(sys.argv)` lets a run be reconfigured from the command line without editing the script. The built-in arguments (`--end`, `--run`, `--out-dir`, `--enable-visuals`, and the rest) are listed in Appendix A. To change a *default* — so the script has a sensible end time but the command line can still override it — call `addDefaultArgument` **before** `parseArgs`:

```
exc = SimulationExecutive()
exc.args().addDefaultArgument("end", 5400)    # new default end time,
    ↪ seconds
exc.parseArgs(sys.argv)                       # must come after
```

**Warning.** `addDefaultArgument` after `parseArgs` has no effect on that run — the arguments have already been read. Set defaults first.

### 6.2.3 The simulation epoch

Simulation time (`base_time`) starts at zero. The *calendar* epoch that SPICE uses to place the planets defaults to 2023-09-26 12:00:00 MDT. Set it explicitly for a scenario tied to a real date:

```
exc.setTime("2027 MAR 21 00:00:00 UTC")      # any SPICE STR2ET-parseable
      ↪ string
```

## 6.3 Building the World

### 6.3.1 SpicePlanet

```
earth = SpicePlanet(exc, "earth")
moon  = SpicePlanet(exc, "moon")
sun   = SpicePlanet(exc, "sun")
```

The name is the SPICE body. Each `SpicePlanet` registers itself as a gravitating body, so **order of creation does not matter** and you do not wire planets to anything — every spacecraft in the scene automatically feels every body's gravity (Chapter 8). Each planet publishes an `inertial_frame` and a `rotating_frame` that other models attach to, a `self_id` handle that pointer params such as a spacecraft's `planet_ptr` can be wired from, and physical constants (`eq_radius`, `flattening`, `mu`, `J2`, `J3`, `mean_ang_vel`) drawn from a built-in table.

For a body not in the SPICE kernels, or for a hypothetical one, use `CustomPlanet` instead and set its constants and motion by hand (Chapter 10).

## 6.4 The Spacecraft Model

`Spacecraft` is the workhorse assembly. It is a rigid `Body` wrapped around a set of environment models that are configured automatically from its `planet_ptr`:

- aspherical (`J2/J3`) gravity for the central body, or spherical harmonics where enabled;
- gravity-gradient torque;
- planet-relative state — geodetic latitude, longitude, altitude;
- N-body third-body perturbations from every other body in the scene;
- MSIS atmospheric drag *for Earth only*;
- an LVLH frame and two frame-state sensors.

### 6.4.1 Key parameters

#### Spacecraft params (set before `startup()`)

| Signal                          | Type       | Default  | Meaning                                                                                                                                        |
|---------------------------------|------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>planet_ptr</code>         | planet     | -        | The body this spacecraft orbits. Required unless you accept the auto-elected central body. Set with <code>sc.params.planet_ptr(earth)</code> . |
| <code>mass</code>               | double     | 1.0      | Spacecraft mass, kg.                                                                                                                           |
| <code>inertia</code>            | Matrix3    | identity | Body inertia tensor, kg m <sup>2</sup> .                                                                                                       |
| <code>initial_position</code>   | Vector3    | zeros    | Initial position in the init frame, m (see the initializers below — usually you use those instead).                                            |
| <code>initial_velocity</code>   | Vector3    | zeros    | Initial velocity, m/s.                                                                                                                         |
| <code>initial_attitude</code>   | Quaternion | identity | Initial attitude wrt the init frame.                                                                                                           |
| <code>initial_ang_vel</code>    | Vector3    | zeros    | Initial body angular velocity, rad/s.                                                                                                          |
| <code>pos_vel_init_frame</code> | frame      | -        | Frame the position/velocity initializer works in. Unset → the <code>planet_ptr</code> inertial frame.                                          |
| <code>att_init_frame</code>     | frame      | -        | Frame the attitude initializer works in. Unset → the <code>planet_ptr</code> inertial frame.                                                   |
| <code>visuals_model</code>      | string     | ""       | Name of a .glb model for the viewer.                                                                                                           |
| <code>visuals_color</code>      | string     | ""       | Trace/marker color, e.g. "#ff8800".                                                                                                            |

### 6.4.2 Key outputs

#### Spacecraft outputs

| Signal                            | Type       | Default | Meaning                                                              |
|-----------------------------------|------------|---------|----------------------------------------------------------------------|
| <code>pos_sc_pci</code>           | Vector3    | -       | Position wrt the planet-centered inertial frame, m.                  |
| <code>vel_sc_pci</code>           | Vector3    | -       | Velocity wrt the planet-centered inertial frame, m/s.                |
| <code>quat_sc_pci</code>          | Quaternion | -       | Attitude wrt the planet inertial frame.                              |
| <code>ang_vel_sc_pci__body</code> | Vector3    | -       | Angular velocity wrt the planet, expressed in the body frame, rad/s. |
| <code>latitude_detic</code>       | double     | -       | Geodetic latitude, rad.                                              |
| <code>longitude</code>            | double     | -       | Longitude, rad.                                                      |
| <code>altitude_detic</code>       | double     | -       | Geodetic altitude, m.                                                |
| <code>body</code>                 | frame      | -       | The spacecraft body frame (attach sensors and actuators here).       |
| <code>lvlh</code>                 | frame      | -       | The local-vertical/local-horizontal frame.                           |
| <code>total_power_in</code>       | double     | -       | Sum of registered power sources, W.                                  |
| <code>total_power_out</code>      | double     | -       | Sum of registered power loads, W.                                    |

### 6.4.3 Setting the initial state

All three initializers are called **after** `startup()`:

| Operation                | C++                                             | Python                                                          |
|--------------------------|-------------------------------------------------|-----------------------------------------------------------------|
| From orbital elements    | <code>initializeFromOrbitalElements(...)</code> | <code>initializeFromOrbitalElements(a, e, i, raan, w, f)</code> |
| From position & velocity | <code>initializePositionVelocity(...)</code>    | <code>initializePositionVelocity(r, v, frame)</code>            |
| Attitude & body rate     | <code>initializeAttitude(...)</code>            | <code>initializeAttitude(q, w_body, frame)</code>               |

(each called on the spacecraft object, e.g. `sc.initializeFromOrbitalElements(...)`)

`initializeFromOrbitalElements` takes the semimajor axis in meters and inclination, RAAN, argument of perigee, and true anomaly in radians, and places the spacecraft on that orbit about its `planet_ptr`. `initializePositionVelocity` takes a position and velocity (a `CartesianVector3` each) and, optionally, the frame they are expressed in — a `Frame` object or a frame name string. If you omit it (or set the `pos_vel_init_frame` param to `None`),

the state is taken in the `planet_ptr`'s inertial frame, which is almost always what you want. `initializeAttitude` takes a Quaternion and a body-frame angular-velocity vector, and defaults its frame the same way (`att_init_frame`).

#### 6.4.4 Impulsive maneuvers

```
burn = sc.programManeuver(
    CartesianVector3([0.0, 120.0, 0.0]),  # delta-v vector, m/s, in the
    ↪ given frame
    3600.0,                               # sim time (s) OR a SPICE time
    ↪ string
    sc.outputs.lvlh(),                    # frame the delta-v is
    ↪ expressed in
)
```

This schedules an instantaneous velocity change at the given time. `programManeuver` returns the underlying burn model so you can inspect or reconfigure it. For finite burns, add a thruster model (Chapter 10).

#### 6.4.5 Reaching the wrapped models

The environment models inside `Spacecraft` are reachable through accessors, so you can log their outputs or adjust their configuration:

```
sc.planetRelativeModel()  # latitude / longitude / altitude, and more
sc.asphericalGravityModel() # the J2/J3 field (None if harmonics are
    ↪ active)
sc.msisModel()            # MSIS atmosphere (None if not Earth)
sc.atmosDrag()            # the flat-plate drag model
sc.gravityGradientModel()
sc.nBodyPerturbationModel()
```

### 6.5 The Launch Vehicle Model

`LaunchVehicle` is the other assembly, a sibling to `Spacecraft` for powered atmospheric flight — boosters, sounding rockets, ascent to orbit. It is a rigid body with a gimbaled, throttleable main engine (with an optional actuation latency), mass depletion as propellant burns, a simple rocket-aerodynamics model referenced to the nose tip, and drogue and main recovery parachutes. Its state outputs match `Spacecraft`'s (`pos_sc_pci`, `vel_sc_pci`, `quat_sc_pci`, `latitude_detic`, `longitude`, `altitude_detic`), so downstream logging, visuals and analysis treat the two the same.

## LaunchVehicle – selected params

| Signal                    | Type    | Default  | Meaning                                                                                  |
|---------------------------|---------|----------|------------------------------------------------------------------------------------------|
| planet_ptr                | planet  | –        | The body it flies from / to.                                                             |
| mass                      | double  | 1.0      | Dry structural mass, kg (propellant is tracked separately).                              |
| main_engine_thrust        | double  | 0.0      | Nominal main-engine thrust, N.                                                           |
| main_engine_mdot          | double  | 0.0      | Propellant mass flow at nominal thrust, kg/s.                                            |
| main_engine_location_body | Vector3 | zeros    | Engine position in the body frame, m.                                                    |
| main_engine_max_gimbal    | double  | $\pi/2$  | Gimbal angle limit, rad.                                                                 |
| main_engine_latency       | int     | 0        | Actuation delay between a throttle command and the engine acting on it, in milliseconds. |
| propellant_mass_init      | double  | 0.0      | Propellant loaded at start, kg.                                                          |
| inertia_full_body         | Matrix3 | identity | Inertia tensor, fully fuelled.                                                           |
| inertia_dry_body          | Matrix3 | identity | Inertia tensor, empty.                                                                   |
| aero_reference_area       | double  | 0.0      | Aerodynamic reference area, m <sup>2</sup> (0 disables the whole aero force).            |
| aero_cn_alpha             | double  | 2.0      | Normal-force coefficient slope, per rad.                                                 |
| aero_nose_tip_body        | Vector3 | zeros    | Nose-tip location in the body frame, m – the reference point for aero_pos_cp_nose.       |
| main_chute_area           | double  | 0.0      | Main parachute canopy area, m <sup>2</sup> (0 disables it).                              |

## LaunchVehicle – inputs (drive these from your guidance)

| Signal              | Type    | Default | Meaning                                                                                                              |
|---------------------|---------|---------|----------------------------------------------------------------------------------------------------------------------|
| thrust_command_body | Vector3 | zeros   | Commanded thrust vector in the body frame, N.                                                                        |
| deploy_drogue       | bool    | false   | Rising edge deploys the drogue chute.                                                                                |
| deploy_main_chute   | bool    | false   | Rising edge deploys the main chute.                                                                                  |
| aero_ca             | double  | 0.0     | Axial-force coefficient (feed from a table if you have one).                                                         |
| aero_pos_cp_nose    | Vector3 | zeros   | Center-of-pressure station measured aft of the nose tip, m; X is distance aft. An input so it can migrate with Mach. |

Initialize its state after `startup()` with the same `initialize...` calls as `Spacecraft` – a launch typically starts from a position and velocity on the ground. The shipped `sounding_rocket` and `orbital_insertion` examples build complete ascents around this model.

## 6.6 Adding Models

Every other model follows the same three steps: construct it as a child of the executive, set its params, and wire its inputs.

```
from warptwin.EarthObservationModel import EarthObservationModel

eom = EarthObservationModel(exc, "eom")
eom.params.planet_rotating_frame(earth.outputs.rotating_frame())
eom.params.spacecraft_frame(sc.outputs.body())
eom.params.target_lat_rad(DEGREES_TO_RADIANS * 40.0)
eom.params.target_lon_rad(DEGREES_TO_RADIANS * -105.0)
```

The model name (second constructor argument) is what appears in signal addresses and in the schedule, so give models distinct, meaningful names – "star\_tracker\_a", not "m1".

## 6.7 Wiring Models Together

There are two distinct ways one model is connected to another, and they are not interchangeable.

### 6.7.1 Pointer configuration — set once, before startup

When a parameter is a *pointer* — a frame, a planet, another model — you set it once by passing the upstream value:

```
eom.params.spacecraft_frame(sc.outputs.body())           # note the () --
    ↪ pass the value
sc.params.planet_ptr(earth)
```

This is configuration. It happens before `startup()` and does not change during the run.

`planet_ptr` also accepts the planet's `self_id` output — `sc.params.planet_ptr(earth.outputs.self_id)` — which is the form `wt-gui` writes when you drag the connection. The two are equivalent; passing the object directly is shorter in a script.

### 6.7.2 Signal connection — tracked every step

When an *input value* should follow an *output value* as both change over the run, connect them with `connectSignals`:

```
from warptwin.WarpTwinPy import connectSignals

connectSignals(planet_relative.outputs.altitude, # upstream output
               atmosphere.inputs.altitude)      # downstream input
```

Note there are **no parentheses** here: you pass the signal objects themselves, not their current values. After this call the atmosphere model's `altitude` input reflects the planet-relative model's `altitude` output at every step, automatically. This is exactly what dragging a connection in `wt-gui` does.

**Note.** Rule of thumb: a `params` pointer (frame, planet, model handle) is set by passing its value once; an `inputs` signal that varies over time is joined to an upstream `outputs` signal with `connectSignals`. A model's outputs may feed many downstream inputs; a given input should be driven by only one upstream output.

## 6.8 Finding Signal Addresses

Every value in a simulation has a dotted address, e.g. `.exc.sc.outputs.pos_sc_pci`. Two ways to discover them:

- `exc.search("altitude")` returns a list of every address matching the string — signals and frames both. Run it in an interactive session while building a script.
- `results/graph_tree.json`, written after every run, is the complete address map with each signal's type and current value.

| Operation                   | C++                                         | Python                                     |
|-----------------------------|---------------------------------------------|--------------------------------------------|
| Search the whole simulation | <code>exc.search("drag");</code>            | <code>exc.search("drag")</code>            |
| Search only the frame tree  | <code>exc.searchFrameTree("lvlh");</code>   | <code>exc.searchFrameTree("lvlh")</code>   |
| Search only the model tree  | <code>exc.searchSimTree("thruster");</code> | <code>exc.searchSimTree("thruster")</code> |

## 6.9 Logging and Visuals

Both are covered in full later — logging in Chapter 11, visuals in Chapter 12 — but the minimum is:

```
log = Hdf5Logger(exc, "states.h5")           # or CsvLogger(exc,
    ↪ "states.csv")
log.addParameter(exc.time().base_time, "t")
log.addParameter(sc.outputs.pos_sc_pci, "pos")
exc.logManager().addLog(log, 10)             # 10 Hz

exc.enableVisuals()                          # 3D viewer on run completion
```

You may create several loggers at different rates — a fast one for attitude, a slow one for orbit — each with its own filename.

## 6.10 Driving the Run

| Operation                  | C++                               | Python                           |
|----------------------------|-----------------------------------|----------------------------------|
| Initialize all models      | <code>exc.startup();</code>       | <code>exc.startup()</code>       |
| Advance one step           | <code>exc.step();</code>          | <code>exc.step()</code>          |
| Advance by a specific time | <code>exc.step(Time(0.5));</code> | <code>exc.step(Time(0.5))</code> |
| Run to the end time        | <code>exc.run();</code>           | <code>exc.run()</code>           |
| Has the run ended?         | <code>exc.isTerminated();</code>  | <code>exc.isTerminated()</code>  |
| Stop the run early         | <code>exc.terminate();</code>     | <code>exc.terminate()</code>     |

`run()` is `step()` in a loop until the end time. Step the simulation yourself when you need to interleave Python logic with the run — a custom controller, a stopping condition, logging something Python computes:

### Example 6.2 — Stepping the simulation from Python

```
exc.startup()
sc.initializeFromOrbitalElements(6778137.0, 0.001,
                                DEGREES_TO_RADIANS * 51.6, 0.0, 0.0, 0.0)

apogee_seen = 0.0
```

```
while not exc.isTerminated():
    exc.step()
    alt = sc.outputs.altitude_detic()
    apogee_seen = max(apogee_seen, alt)
    if exc.time().base_time().asFloatingPoint() > 6000.0:
        exc.terminate()

print(f"max altitude in first 6000 s: {apogee_seen/1000:.1f} km")
```

**Warning.** Initial states go *after* `startup()` whether you call `run()` or step by hand. `params` go *before*.

## 6.11 Guidance, Navigation, and Control

Developing GN&C is one of the main reasons WarpTwin exists. “Doing GN&C” here means closing a loop through the simulation: **sensor models** produce measurements, your **navigation** turns those into a state estimate, your **guidance** produces a reference, your **control** law produces actuator commands, **actuator models** turn those commands into forces and torques on the body, the dynamics integrate, and the loop repeats on the next step.

WarpTwin does not prescribe how you implement the navigation, guidance, and control boxes. There are three ways, and they are a progression:

**In Python, by stepping the run** Read sensor outputs, compute, write actuator inputs, `exc.step()`, repeat — the pattern of Example 6.2. This is the fastest way to prototype a control law and see it work.

**As a custom C++ model** Once the algorithm settles, move it into a model with sensor inputs and actuator-command outputs, scheduled into the loop like any other model (Chapter 15). It runs at full speed, is reusable across scenarios, and shows up in `wt-gui`.

**As real flight software** Run the actual GN&C flight code — a WarpOS app or your own — in the loop against the simulated sensors and actuators (Chapter 16). This is the verification step: the code that flies, exercised against the simulated vehicle.

### 6.11.1 Where the loop closes

The sim step (Chapter 7) has three phases. Sensing that only reports state fits `START_STEP`. A control law whose output feeds a force or torque that must be integrated *this* step belongs in `DERIVATIVE`, so its effect is picked up by the integrator. Actuator models apply their force at a node on the body; the dynamics engine sums those each derivative evaluation (Chapter 8).

### 6.11.2 The ADCS assembly

For attitude determination and control specifically, the ADCS model is a ready-made bundle: a bank of reaction wheels and a bank of magnetic torque rods, with realistic torque limits, momentum caps, saturation handling, power draw, and attitude-knowledge noise. You give it the body and planet, describe the hardware, then command a pointing mode and target frames:

```
from warptwin.ADCS import ADCS
```

```
adcs = ADCS(exc, "adcs")
adcs.params.sc_body(sc.outputs.body())
adcs.params.planet_ptr(earth)
adcs.params.num_reaction_wheels(4)
adcs.params.rw_peak_torque(0.02)           # N m
adcs.params.rw_momentum_cap(0.3)          # N m s
adcs.params.num_torque_rods(3)
adcs.params.tr_peak_torque(1.0e-3)        # N m
adcs.params.att_knowledge_noise_std(1.0e-4) # rad, 1-sigma

# ... after startup, command pointing (see the ADCS model reference for the
# mode values and the primary/secondary constraint convention):
adcs.inputs.mode(1)
adcs.inputs.desired_primary(sc.outputs.lvlh())
```

ADCS publishes `est_quat_body__ref` (its estimated attitude, i.e. the “navigation” output), `power_draw`, and `total_angular_momentum`. Chapter 10 documents its modes and full signal set.

### 6.11.3 The building blocks

Sensor models (IMU, gyro, star tracker, Sun sensor, GPS, `FrameStateSensorModel`), navigation models such as `StochasticNavigation` (truth state in, a biased and noisy estimate out), actuator models (reaction wheels, magnetic torque rods, thrusters, and `programManeuver` for an impulsive  $\Delta v$ ), and the reference frames a controller points against (the spacecraft `lvlh` frame, a planet’s inertial and rotating frames, a ground target) are all in the model reference, Chapter 10. The shipped `orbital_insertion` example wires a full ascent GN&C loop — `StochasticNavigation` feeding a pointing controller feeding the launch vehicle’s engine gimbal — and is worth reading once you have the pieces.

## 6.12 Structuring a Larger Script

As a scenario grows, a few conventions keep it readable. The shipped `constellation` example is a good model:

- A **parameters block** of named constants at the top — orbit elements, counts, rates — rather than magic numbers buried in calls.
- **Loops** to build repeated structure. Keep the created objects in a list so you can initialize them in a second loop after `startup()`:

```
sats = []
for i in range(N):
    s = Spacecraft(exc, f"sc_{i}")
    s.params.planet_ptr(earth)
    sats.append(s)

exc.startup()

for i, s in enumerate(sats):
    s.initializeFromOrbitalElements(A, E, INC, RAAN, 0.0,
                                   DEGREES_TO_RADIANS * i * (360.0 /
```

```
↪ N))
```

- **Functions** for anything you do more than once — attaching a standard sensor suite to a spacecraft, say.
- A separate **analysis.py** that reads the logs and produces plots, kept apart from the script that runs the simulation. Most shipped examples are organized this way.

## 6.13 Checklist

- Executive created; rate and integrator set; `addDefaultArgument` calls (if any) *before* `parseArgs`.
- All planets/moons/Sun created (order irrelevant).
- Every model constructed as a child of `exc`, with a distinct name.
- All params set — including `planet_ptr` — and all pointer wiring done, *before* `startup()`.
- Time-varying inputs joined to upstream outputs with `connectSignals`.
- Loggers created, parameters added, handed to `exc.logManager().addLog(...)`.
- `startup()` → initial states → `run()`.

# 7 Simulation Architecture

---

Chapter 6 showed how to write a simulation. This chapter explains what the engine does with it: the executive and the managers it owns, how simulation time is kept, how the work of a step is divided into phases, the contract every model obeys, the difference between a model and a flight-software app, and how signals carry values between them. Chapter 8 takes the dynamics engine further; this chapter is the frame around it.

## 7.1 The Simulation Executive

The `SimulationExecutive` is the one object that owns a simulation. Everything else — the schedule, the dynamics engine, time, logging, visuals, SPICE, dispersions, and the root of the frame tree — hangs off it, and every model is constructed with a reference to it. It is a specialization of the WarpOS `FlightExecutive`, which is what lets real flight software share the same executive as the simulation (Chapter 16).

You reach the parts of the executive through accessors:

---

|                                 |                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------|
| <code>exc.schedule()</code>     | The simulation scheduler: the per-phase run lists and the 6-DOF dynamics model. |
| <code>exc.time()</code>         | The time manager (below).                                                       |
| <code>exc.logManager()</code>   | Registers loggers and runs them at their rates (Chapter 11).                    |
| <code>exc.visualsModel()</code> | The visualization feed (Chapter 12).                                            |
| <code>exc.spiceManager()</code> | SPICE kernels, body states, and the central-body choice.                        |
| <code>exc.dispersions()</code>  | The Monte Carlo dispersion engine (Chapter 13).                                 |
| <code>exc.args()</code>         | The parsed command-line arguments (Appendix A).                                 |
| <code>exc.rootFrame()</code>    | The root of the frame tree.                                                     |

---

### 7.1.1 What `startup()` does

`exc.startup()` is the boundary between building a simulation and running it. In order, it:

1. registers the visuals model and checks the license;
2. loads the SPICE kernels;
3. starts every registered planet, which is what **elects the central body** (the most massive body, unless you chose one) and places every planet's frames at the simulation epoch;
4. runs `start()` on every other scheduled unit, in registration order;
5. re-synchronizes the planet frames to the final central body, so inter-body geometry is consistent no matter what order the planets were created in.

After `startup()` the frame tree is placed and every model has done its one-time setup, which is why **initial vehicle states are set after `startup()`** — they are defined relative to frames that only exist once startup has run. `exc.run()` is then simply `exc.step()` in a loop until the end time or a termination condition.

## 7.2 Simulation Time

Simulation time starts at zero and advances by exactly the step size each step. The calendar epoch that anchors it to real dates – and therefore to SPICE body positions – is separate, and defaults to 2023-09-26 12:00:00 MDT. Set it with `exc.setTime("<SPICE time string>")`.

The time manager publishes several clocks, each a `Time` object, addressable and loggable like any signal:

|                                             |                                                                                                                         |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>base_time</code>                      | Simulation time. Zero at start; += step size each step.                                                                 |
| <code>tdb_time</code>                       | Barycentric Dynamical Time (ephemeris time), seconds past the J2000 epoch. What SPICE uses.                             |
| <code>utc_time, gps_time</code>             | UTC and GPS, derived from <code>tdb_time</code> .                                                                       |
| <code>step_start_time, step_end_time</code> | The bracket of the step currently being computed.                                                                       |
| <code>system_time, navigation_time</code>   | The simulated <code>os.systemTime()</code> and <code>os.navigationTime()</code> that flight software sees (Chapter 16). |

String and numeric forms are available as methods on `exc.time()`: `utcTime()` ("YYYY MM DD HH:MM:SS"), `jdTime()`, `gpsTime()`, and `decimalYear()`. `configureSystemTime()` and `configureNavigationTime()` tune the offset, resolution, and source of the two flight-software clocks.

## 7.3 The Simulation Step

A run is a sequence of steps. Each step advances `base_time` by the step size and is divided into three phases, at which scheduled units run:

|                         |                                                                                                                                                                                                                                            |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>START_STEP</code> | Once per step, at the step's start time, before any state has been updated. Sensing and setup.                                                                                                                                             |
| <code>DERIVATIVE</code> | Where forces, moments, and accelerations are formed. Runs <b>once per step for forward Euler, four times for RK4</b> ; the dynamics engine integrates at the end of each. Anything whose output must be integrated this step belongs here. |
| <code>END_STEP</code>   | Once per step, after the state has been advanced to the step's end time. Logging happens here by default.                                                                                                                                  |

Every scheduled unit is assigned to exactly one *slot*:

|                                                 |                                                                                                                                                  |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ALL</code>                                | Runs in <code>START_STEP</code> , <code>DERIVATIVE</code> , and <code>END_STEP</code> .                                                          |
| <code>START_STEP / DERIVATIVE / END_STEP</code> | Runs in that one phase only.                                                                                                                     |
| <code>STARTUP_ONLY</code>                       | <code>start()</code> runs at startup; <code>execute()</code> never runs.                                                                         |
| <code>NOT_SCHEDULED</code>                      | The unit is known to the simulation (addressable, loggable) but neither <code>start()</code> nor <code>execute()</code> is called automatically. |

There is no per-unit rate in the sim scheduler: everything in a slot runs every step. Rate limiting applies to *loggers* (Chapter 11), not to models. To make a model effectively run slower, gate its own `execute()` on `base_time`, or drive it yourself from a stepped loop (Chapter 6).

**Note.** Two units are scheduled by the engine itself and always run last in their phase: the 6-DOF dynamics model (end of every DERIVATIVE) and the external interface (start of START\_STEP, end of END\_STEP). You do not register these.

## 7.4 The Model Contract

Every model — built-in or your own — exposes three groups of signals and overrides a small set of lifecycle methods. This uniformity is what lets models connect to each other like parts of a kit.

**params** Configuration. Set once, *before startup* (). Changing a param after startup has undefined effect.

**inputs** Values expected to change during the run, usually driven from another model's outputs (below).

**outputs** Results the model computes, read by downstream models and by loggers.

**start()** Runs once, at startup (). One-time setup from params.

**execute()** Runs every scheduled step, in the model's phase. Maps params and inputs to outputs.

**activate() / deactivate()** Turn per-step execution on and off during a run without changing the model's slot.

In a model's C++ header, signals are declared with the `SIGNAL(name, type, default)` macro inside `START_PARAMS/START_INPUTS/START_OUTPUTS` blocks, and the class itself with the `MODEL(Name)` macro (Chapter 15). Every lifecycle method returns an `int16` error code: `NO_ERROR` (zero) is success; any non-zero value is a fault and stops the run.

Three utilities are on every model:

|                                            |                                                                                                                                              |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>model.dump()</code>                  | Print the model's full instantaneous state — params, inputs, outputs — to the terminal.                                                      |
| <code>model.asJson()</code>                | The same as a JSON object.                                                                                                                   |
| <code>model.configureFromJson(path)</code> | Set params from a JSON file whose keys are param names; <code>configureFromDefault(name)</code> loads a named preset shipped with the model. |

## 7.5 Models and Apps

Everything that runs in a simulation step is, underneath, an *app* — the same unit WarpOS flight software is built from. WarpTwin's `Model` is an app with simulation conveniences layered on. In practice you deal with two kinds:

|                        | Model                                                                                                     | App                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| What it is             | A WarpTwin simulation unit: physics, environment, a sensor, an actuator, your GN&C.                       | A WarpOS (or your own) flight-software unit, run in the loop for verification (Chapter 16). |
| Gets into the schedule | <b>Automatically</b> , in its constructor, at the slot the model was written to use.                      | <b>Manually</b> –<br><code>exc.registerApp(app, slot).</code>                               |
| Interface              | <code>params / inputs / outputs, start(), execute().</code>                                               | The same signal groups, plus the WarpOS app handlers.                                       |
| Typical slot           | Chosen by the model (often ALL or START_STEP; force one with the <code>schedule_slot</code> constructor). | Whichever slot you register it to.                                                          |

### 7.5.1 Scheduling

A model is placed in the schedule when it is constructed. To override the slot it was written to use, pass one explicitly:

```
from warptwin.SlopeIntercept import SlopeIntercept
from warptwin.WarpTwinPy import DERIVATIVE

si = SlopeIntercept(exc, DERIVATIVE, "si")  # force this model into
      ↪ DERIVATIVE
```

An app is not scheduled until you say so:

```
from warptwin.WarpTwinPy import END_STEP

exc.registerApp(my_flight_app, END_STEP)
```

Registering the same unit twice is ignored with a warning. A unit registered at NOT\_SCHEDULED is still constructed, addressable, and loggable – it just will not run until something activates it. During a run, `activate()` and `deactivate()` turn a unit's per-step execution on and off without changing its slot.

## 7.6 Signals and Addresses

A signal is a typed value that lives at a fixed place in the simulation. You read it by calling it with empty parentheses and write it by calling it with a value:

```
m = si.params.m()      # read
si.params.m(2.0)       # write
y = si.outputs.y()     # read a result
```

### 7.6.1 Connecting signals

An inputs signal is made to track an upstream outputs signal with `connectSignals` – passing the signal objects, *not* their values:

```
from warptwin.WarpTwinPy import connectSignals

connectSignals(sensor.outputs.altitude,    # upstream output
               atmosphere.inputs.altitude) # downstream input
```

After this call the downstream input reflects the upstream output at every step. One output may fan out to many inputs; a given input should be driven by only one output. Pointer-valued params (a frame, a planet, a model handle) are not connected this way – they are set once, by value, before `startup()` (Chapter 6).

### 7.6.2 Addresses

Every signal and frame has a dotted address rooted at the executive, for example `.exc.si.outputs.y` or `.exc.schedule.time.base_time`. Addresses are how you log a value that is buried in a sub-model, and how WarpLink injects values into a running simulation (Chapter 17). Two ways to find one:

|                                      |                                                                                                                                                 |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>exc.search("text")</code>      | Every address containing the text – signals and frames. <code>searchSimTree</code> restricts to models, <code>searchFrameTree</code> to frames. |
| <code>results/graph_tree.json</code> | Written after every run: the complete address map, each end signal tagged with its type and current value.                                      |

`results/sim_data.json`, written alongside it, records every model's params as they were run – a record of exactly what you simulated. Both files are controlled by `--write-data=json`.

## 7.7 The Frame Tree

The dynamics of a simulation live in a tree of frames rooted at `exc.rootFrame()`. When SPICE is active the root is the base of the J2000 frame. Planets attach their inertial and rotating frames to the root; each vehicle's body frame attaches to the inertial frame of the body it orbits (its `planet_ptr`), so its integrated state stays small and well conditioned regardless of which body sits at the origin. Sensors, actuators, and force application points attach to a vehicle's body frame.

Every frame exposes its state both relative to its parent (`pos_f__p` and friends) and relative to the root (`rootRelPosition()`, `rootRelQuaternion()`, and the rest). Chapter 8 covers frames, bodies, nodes, and how forces on them become integrated motion.

## 7.8 The Managers

**LogManager** (`exc.logManager()`) Holds the registered loggers and the rate monitors that fire them. `addLog(logger, rate)` registers one; `outDir()` is the output directory, set from `--out-dir`. Chapter 11.

**VisualsModel** (`exc.visualsModel()`) The visualization feed: it writes the viewer's configuration and per-object state files, or streams them live. Inactive until `exc.enableVisuals()` or `--enable-visuals`. Chapter 12.

**SpiceManager** (`exc.spiceManager()`) Loads SPICE kernels and answers body-state queries. It also owns the **central-body choice**: by default the most massive body in the scene, overridable with `setCentralBody("earth")` — a manual choice is not re-elected. `resyncPlanetFrames()` re-replaces the planet frames once the choice is final (the engine calls this during `startup()`).

**DispersionEngine** (`exc.dispersions()`) Declares the uncertain quantities for a Monte Carlo study and draws them per run number. Chapter 13.

**SimScheduler** (`exc.schedule()`) Owns the per-phase run lists, the 6-DOF dynamics model, and the external interface used for flight-software and WarpLink connections.

# 8 The Dynamics Engine

The dynamics engine is what turns the forces and moments that models compute into integrated motion. You rarely call it directly — the assemblies (Chapter 6) wire themselves into it — but understanding it is what lets you attach your own forces, read states in the frame you want, and know why a result looks the way it does.

## 8.1 Kinematics and Dynamics

WarpTwin keeps two things separate:

**Kinematics** — *where things are*. A tree of **frames**, each carrying its position, velocity, attitude, and angular velocity relative to its parent, chained back to a single root.

**Dynamics** — *why things move*. Forces and moments, applied at **nodes**, resolved onto **bodies** (frames with mass and inertia), turned into accelerations, and integrated forward in time.

The integrated state of a frame is thirteen numbers: position (3), velocity (3), attitude quaternion (4), and angular velocity (3). The engine integrates that 13-vector for every free frame descended from the root, every derivative evaluation.

## 8.2 Frames

A Frame is a coordinate frame with a state relative to its parent. It is created with a name, an optional parent, and a flag for whether it is *free* (its motion is integrated) or locked to its parent:

| Operation                 | C++                                         | Python                                          |
|---------------------------|---------------------------------------------|-------------------------------------------------|
| A locked child frame      | <code>Frame f("imu", &amp;bus);</code>      | <code>Frame("imu", parent=bus)</code>           |
| A free (integrated) frame | <code>Frame f("sc", &amp;pci, true);</code> | <code>Frame("sc", parent=pci, free=True)</code> |

Every frame publishes its parent-relative state as four signals:

|                             |                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------|
| <code>pos_f_p__p</code>     | Position of the frame origin relative to the parent, in parent coordinates.        |
| <code>vel_f_p__p</code>     | Velocity of the frame relative to the parent, in parent coordinates.               |
| <code>quat_f_p</code>       | Attitude of the frame relative to the parent.                                      |
| <code>ang_vel_f_p__f</code> | Angular velocity of the frame relative to the parent, in <i>frame</i> coordinates. |

The same state relative to the *root* is available through accessor methods rather than signals, because it is computed by walking the chain. Each comes as a `rootRel...` getter:

---

|                                                                          |                                               |
|--------------------------------------------------------------------------|-----------------------------------------------|
| <code>rootRelPosition(), rootRelVelocity(), rootRelAcceleration()</code> | translational state<br>in root<br>coordinates |
| <code>rootRelQuaternion(), rootRelDCM()</code>                           | attitude wrt the<br>root                      |
| <code>rootRelAngularVelocity(), rootRelAngularAcceleration()</code>      | rotational state,<br>in frame<br>coordinates  |
| <code>getRootRelFullState()</code>                                       | all of the above in<br>one traversal          |

---

The matching `setRootRel...`() setters are what the vehicle initializers call under the hood.

**Note.** An assembly's body frame is reached with `sc.outputs.body()` (the signal) or `sc.body()` (the accessor). From there, `sc.body().ang_vel_f_p_f` is the body-frame angular rate — which is what an autopilot wants, and is *not* the same as `sc.outputs.ang_vel_sc_pci_body` unless the body's parent is the planet-inertial frame.

## 8.3 Joints

A frame connects to its parent through two **joints** — one translational, one rotational — each of which says which axes are free to move. Today a joint is effectively all-free or all-locked (`FULLY_FREE` / `FULLY_LOCKED`); per-axis freedom is a planned extension.

- **Locked** is the default. A locked frame cannot move relative to its parent — an IMU bolted to a spacecraft bus, a thrust node on a launch vehicle.
- **Free** means the joint's degrees of freedom are integrated — a spacecraft moving relative to a planet's inertial frame.

Mechanically, a joint holds a  $3 \times 3$  *freedom matrix*  $F$  (identity for a free axis set, zero for locked) and its complement  $D = I - F$ . When the engine resolves an acceleration onto a frame it multiplies by  $F$ , so a locked axis simply receives no acceleration. This is how constraints are enforced without special cases.

## 8.4 Bodies

A **Body** is a **Frame** with **mass** and **inertia**. It is the unit that forces act on and that gets accelerated. **mass** is a scalar signal; **inertia** is a  $3 \times 3$  tensor signal about the body frame origin.

When bodies are nested, the engine computes `composite_mass` and `composite_inertia` for the parent — the combined properties of the body and all its child bodies. These are outputs; do not set them.

`Spacecraft` and `LaunchVehicle` each contain one body; reach it with `.body()`. Two structural rules hold today: a body may only be the child of a frame, and a node may only be the child of a body.

## 8.5 Nodes

A Node is a Frame rigidly fixed to a body, at a chosen offset, that exists to **apply a force or moment** there — or to **sense acceleration** there, which is how an accelerometer is modelled. A node carries four signals:

|                      |                                                                                                                                                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| force                | The force vector applied at the node.                                                                                                                                                                                                              |
| force_frame          | The frame the force is expressed in. Leave it unset and the force is in node coordinates. Point it at an inertial frame and the force stays inertial even as the node rotates with the body — exactly what gravity on a spinning spacecraft needs. |
| moment, moment_frame | The same, for an applied moment.                                                                                                                                                                                                                   |

The engine resolves each node's force and moment into a force/moment couple at the parent body's origin, so an off-center thrust correctly produces both a force and a torque.

## 8.6 How a Model Joins the Dynamics

A force-producing model does not touch the dynamics engine directly. It publishes a force (and/or moment) on its outputs; you connect that to a node's force input; the node hangs off the body. Figure 8.1 shows the shape: several models, each feeding its own node, all children of one body, with sensor frames as further children of that body.

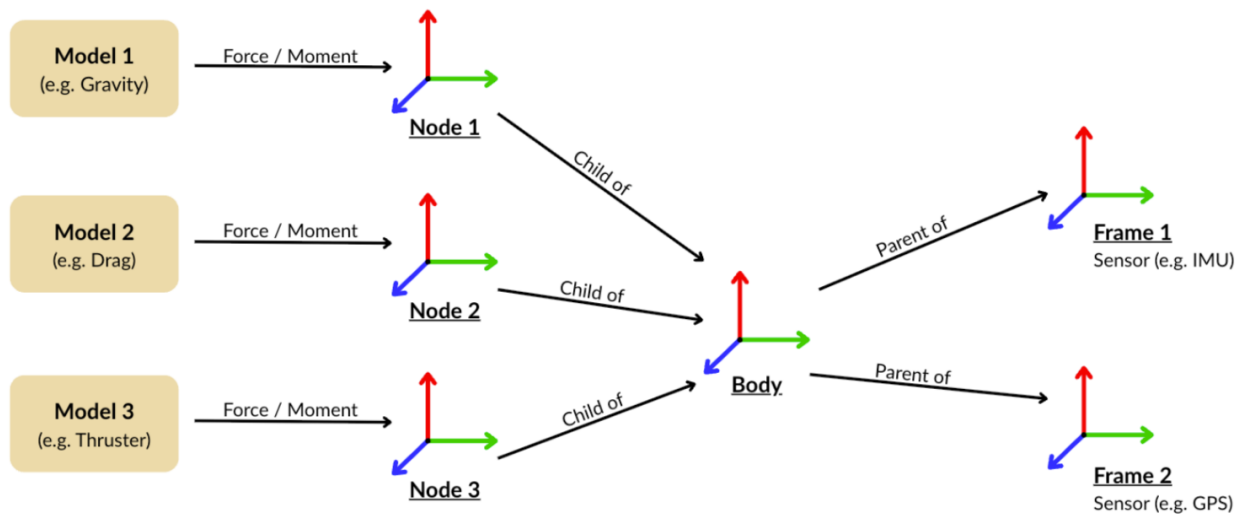


Figure 8.1: Force-producing models feed nodes; nodes are children of a body; the body accumulates the total force and moment. Sensor frames are further children of the body.

The `Spacecraft` assembly builds this wiring for you — a gravity node, a drag node, and a third-body-perturbation node, each connected to the matching built-in model. To add a force of your own, create a Node on the body, connect your model's force output to it, and (if the force is inertial) set its `force_frame`.

## 8.7 The Integration Loop

One model, `SixDOFDynamicsModel`, is the integrator. The scheduler creates exactly one per simulation and runs it **last in every DERIVATIVE phase**, after every force and moment for that evaluation has been computed. Each time it runs it:

1. refreshes every frame's state so the kinematics are current;
2. articulates the forces and moments applied at nodes through the kinematics chain to the body that actually has the freedom to respond (`calcFrameTreeExtForcesMoments`);
3. resolves those into linear and angular acceleration using each body's mass and inertia (`calcFrameTreeExtAcceleration`);
4. integrates the 13-state of every free frame, recursing from `exc.rootFrame()` down.

The integrator is Forward Euler (one derivative evaluation per step) or RK4 (four); set it with `exc.integrator(1)` or `exc.integrator(4)` (the default). Only frames descended from the root are integrated — a frame with no path to the root is inert.

## 8.8 N-Body Gravity

WarpTwin applies the gravity of **every body in the scene** to every spacecraft. You do not wire this up: create a `SpicePlanet` for the Sun, the Moon, or another planet and it participates automatically. Creation order does not matter — planets register with the executive as they are constructed.

### 8.8.1 What is modelled

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| The central body    | Full field: J2/J3 via <code>AsphericalGravityModel</code> , or spherical harmonics where enabled. |
| Every other body    | Point mass, with both the direct and the indirect (reflex) term.                                  |
| Earth, additionally | MSIS atmospheric drag.                                                                            |

The **central body** is chosen by one deterministic rule — the most massive body in the scene — so an Earth/Sun/Moon scene picks the Sun, an Earth/Moon scene picks Earth. Override it with `exc.spiceManager().setCentralBody("earth")`; a manual choice is not re-elected.

### 8.8.2 The formulation

For each perturbing body  $j$ , with all vectors relative to the central body and  $\mathbf{r}_{\text{sat},j} = \mathbf{r}_j - \mathbf{r}_{\text{sat}}$ , the classic Vallado/Escobal third-body acceleration is

$$\mathbf{a}_{\text{pert},j} = \mu_j \left( \underbrace{\frac{\mathbf{r}_{\text{sat},j}}{|\mathbf{r}_{\text{sat},j}|^3}}_{\text{direct pull}} - \underbrace{\frac{\mathbf{r}_j}{|\mathbf{r}_j|^3}}_{\text{indirect / reflex}} \right). \quad (8.1)$$

The indirect term is the central body's own acceleration toward body  $j$ : omitting it would treat the central-body frame as inertial when it is not, and get lunar and solar perturbations badly wrong. Because the two terms nearly cancel for a distant perturber, the perturbation is evaluated with **Battin's numerically stable form** rather than the raw difference:

$$q = \frac{\mathbf{r}_{\text{sat}} \cdot (\mathbf{r}_{\text{sat}} - 2 \mathbf{r}_j)}{|\mathbf{r}_j|^2}, \quad F(q) = q \frac{3 + 3q + q^2}{1 + (1 + q)^{3/2}}, \quad (8.2)$$

$$\mathbf{a}_{\text{pert},j} = -\frac{\mu_j}{|\mathbf{r}_{\text{sat},j}|^3} (\mathbf{r}_{\text{sat}} + F(q) \mathbf{r}_j). \quad (8.3)$$

This is algebraically identical but keeps full double precision, which matters over long propagations.

### 8.8.3 Why the origin choice does not affect accuracy

The integrator treats the root frame as inertial and holds the central body static there. A body pinned at the origin is not truly inertial – it accelerates toward every other body – so its own reflex,

$$\mathbf{a}_{\text{root}} = \sum_{k \neq \text{root}} \mu_k \frac{\mathbf{r}_k}{|\mathbf{r}_k|^3}, \quad (8.4)$$

is applied as a single correction to every spacecraft. Every *other* body's reflex is already carried, because non-root bodies have their motion prescribed from SPICE and the frame machinery accounts for it. When a spacecraft's central body is the root – the usual case, an Earth satellite in an Earth-only scene –  $\mathbf{a}_{\text{root}}$  equals the sum of the indirect terms and the expression collapses body-by-body to the Battin form above.

Numerically, each spacecraft integrates its state relative to **its own** central body (`planet_ptr`), not the origin. A LEO satellite integrates a  $\sim 7 \times 10^6$  m state even when the Sun is the origin  $1.5 \times 10^{11}$  m away; without this a LEO orbit would have to be resolved as a small wiggle on a huge heliocentric vector, costing roughly four orders of magnitude of precision. Together these mean the origin choice affects neither the force model nor the conditioning – it is a framing choice, not a fidelity one. Two consequences worth knowing: the visuals world origin follows the central body (adding a Sun to an Earth scene gives a heliocentric view), and `initial_position` set without `pos_vel_init_frame` is interpreted relative to `planet_ptr`, which stays correct when the origin changes.

### 8.8.4 Notes and limitations

- The force model contains exactly the bodies in the script – no more, no less – which keeps results reproducible and easy to validate against published cases.
- Third bodies are point masses; their own oblateness (the Moon's J2 on an Earth satellite, say) is negligible and not modelled. The body a spacecraft orbits always keeps its full field.
- Adding bodies is cheap: the perturbation is a short sum evaluated per spacecraft per derivative call.

# 9 Vector, Matrix, and Attitude Math

WarpTwin's core math is a templated, allocation-free C++ library built for real-time use, and every type in it is wrapped to Python so a script works with the same objects a model does. This chapter is the reference for the vector, matrix, and attitude types you meet in model signals, initializers, and your own GN&C code.

Two conventions carry through the whole library:

- The element type is a fixed `floating_point` (double in the simulation), so the matrix and vector types are templated only on *size*, not on element type.
- In method names and documentation, capital letters are matrices, lowercase are scalars, and a pass-by-reference output is named `result`. Where a C++ method returns through a `result` argument, the Python binding generally returns it as an ordinary value.

## 9.1 Matrix

`Matrix<R, C>` is an  $R$ -row,  $C$ -column matrix. Fixed sizes have short names, identical in C++ and Python:

|                                                     |                                                             |
|-----------------------------------------------------|-------------------------------------------------------------|
| <code>Matrix2, Matrix3, Matrix4, Matrix6</code>     | Square $2 \times 2$ through $6 \times 6$ .                  |
| <code>Matrix21, Matrix31, Matrix41, Matrix61</code> | Column matrices (a $3 \times 1$ is <code>Matrix31</code> ). |
| <code>Matrix63, Matrix1x3, ...</code>               | Other shapes follow <code>MatrixRxC</code> .                |

### 9.1.1 Construction and element access

```
from warptwin.WarpTwinPy import Matrix3

a = Matrix3()                # all zeros
b = Matrix3(5.0)             # every element = 5.0
c = Matrix3([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # from a nested list
d = Matrix3(c)                # copy

x = c[1][2]                  # element (row 1, col 2); bounds-checked in Python
c.set(0, 0, 10.0)            # checked setter
```

C++ construction is the same shape (`Matrix3 c({{1,2,3}, {4,5,6},{7,8,9}});`). There, `m[r][c]` is *not* bounds-checked – use `set(r,c,v)` / `get(r,c)` for the checked form – whereas the Python `[]` is checked and takes negative indices and slices.

### 9.1.2 Operations

|                            |                                                       |
|----------------------------|-------------------------------------------------------|
| <code>m.transpose()</code> | Transpose.                                            |
| <code>m.inverse()</code>   | Inverse (square).                                     |
| <code>m.trace()</code>     | Trace.                                                |
| <code>m.froNorm()</code>   | Frobenius norm.                                       |
| <code>m.identity()</code>  | Set to identity in place (alias <code>eye()</code> ). |
| <code>m.dump()</code>      | Print; <code>print(m)</code> also works in Python.    |

The determinant (`det`), Moore–Penrose pseudo-inverse (`pseudoinverse`), Cholesky decomposition (`chol`, `cholSolve`), singular value decomposition (`svd`), and eigen-decomposition (`eig`) are also provided; see the Doxygen reference for their signatures.

### 9.1.3 Arithmetic

The arithmetic operators call the same free functions C++ callers use, so Python and C++ behave identically:

---

|                           |                                               |
|---------------------------|-----------------------------------------------|
| <code>A * B</code>        | Matrix product (inner dimensions must agree). |
| <code>A * v</code>        | Matrix times a <code>CartesianVector</code> . |
| <code>a * A, A * a</code> | Scalar scaling.                               |
| <code>A + B, A - B</code> | Element-wise, same shape.                     |

---

#### Example 9.1 – Matrix arithmetic in Python

```
from warptwin.WarpTwinPy import Matrix3, CartesianVector3

A = Matrix3([[1, 2, 3], [4, 5, 6], [7, 8, 10]])
v = CartesianVector3([1, 0, 0])

Av = A * v                # CartesianVector3
At = A.transpose()
Ainv = A.inverse()
I = A * Ainv              # ~ identity
print(I)
```

## 9.2 CartesianVector

`CartesianVector<L>` is a length- $L$  column vector – a `Matrix<L, 1>` with vector operations added. The named sizes are `CartesianVector2`, `CartesianVector3`, `CartesianVector4`, and `CartesianVector6`.

---

|                                        |                                                                          |
|----------------------------------------|--------------------------------------------------------------------------|
| <code>CartesianVector3([1,2,3])</code> | Construct from a list (C++: <code>CartesianVector3 v({1,2,3});</code> ). |
| <code>v[i]</code>                      | Element $i$ .                                                            |
| <code>v.norm(), v.normSquared()</code> | Magnitude and its square.                                                |
| <code>v.unit()</code>                  | The unit vector (a copy).                                                |
| <code>v.unitize()</code>               | Normalize in place (alias <code>normalize()</code> ).                    |

---

Three free functions cover the products you need for dynamics:

---

|                          |                                                                                                                    |
|--------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>dot(a, b)</code>   | Scalar dot product, any matching length.                                                                           |
| <code>cross(a, b)</code> | Vector cross product, length 3 only.                                                                               |
| <code>tilde(v)</code>    | The $3 \times 3$ skew-symmetric (“tilde”) matrix of a 3-vector, so that <code>tilde(a) * b == cross(a, b)</code> . |

---

```
from warptwin.WarpTwinPy import CartesianVector3, dot, cross

r = CartesianVector3([7.0e6, 0.0, 0.0])
v = CartesianVector3([0.0, 7.5e3, 0.0])
h = cross(r, v) # specific angular momentum
speed = v.norm()
radial_speed = dot(v, r.unit())
```

### 9.3 Attitude Representations

Attitude is orientation – which way a frame points – and is entirely separate from position. WarpTwin implements five representations, following the conventions in Schaub & Junkins, *Analytical Mechanics of Space Systems*. All are **passive** rotations, and any one converts to any other except at that representation's singularities.

| Type           | Identity        | Notes                                                                                                |
|----------------|-----------------|------------------------------------------------------------------------------------------------------|
| DCM            | identity matrix | $3 \times 3$ ; no singularities; nine numbers.                                                       |
| Quaternion     | (1, 0, 0, 0)    | Scalar-first; four numbers; no singularities; the usual choice for propagation.                      |
| Euler321       | (0, 0, 0)       | 3–2–1 yaw–pitch–roll, radians. Singular at pitch = $\pm 90^\circ$ .                                  |
| MRP            | (0, 0, 0)       | Three numbers; singular only at a full $360^\circ$ rotation, handled by switching to the shadow set. |
| RotationVector | (0, 0, 0)       | Axis $\times$ angle.                                                                                 |

DCM, Quaternion, Euler321, and MRP are wrapped for Python; RotationVector is available in C++.

#### 9.3.1 Common operations

Every representation provides conversions to the others (toDCM(), toQuaternion(), toMRP(), toEuler321(), toRotVec(), as applicable) and a kinematic rate() that gives the time derivative of the representation from a body angular velocity:

|            |                                                                                                                                                                        |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DCM        | inverse() (the transpose), rate(omega, Cdot), toQuaternion/toMRP/toRotVec(), toEuler321(out).                                                                          |
| Quaternion | rate(omega, qdot), toDCM/toMRP/toRotVec(), rotationAngle(), inverse() (the conjugate). Jacobians of the rate and of a rotated vector are provided for estimation work. |
| Euler321   | rate(omega, edot) – returns an error code near the singularity – and toDCM().                                                                                          |
| MRP        | shadow() (switch to the shadow set, in place), rate(omega, sdot), toDCM/toQuaternion/toRotVec().                                                                       |

#### 9.3.2 Composing and rotating

Multiplication is overloaded for every representation:

- $A * B$  where A and B are the *same* representation **composes** the two rotations.
- $R * v$  where v is a CartesianVector3 **rotates the vector** into the target frame. With Schaub & Junkins naming,  $quat\_A\_B * v\_B$  gives  $v\_A$ .

There is no implicit multiplication *between* representations — convert first:

```
from warptwin.WarpTwinPy import Quaternion, DCM, CartesianVector3

q_A_B = Quaternion([0.9659, 0.2588, 0.0, 0.0]) # 30 deg about x
v_B   = CartesianVector3([0.0, 1.0, 0.0])

v_A   = q_A_B * v_B           # rotate the vector
q2    = q_A_B * q_A_B         # compose (60 deg about x)
C_A_B = q_A_B.toDCM()         # same rotation as a DCM
angle = q_A_B.rotationAngle()
```

## 9.4 Constants

Common physical and unit-conversion constants are exported from `warptwin.WarpTwinPy`. A selection:

|                                                                               |                                                      |
|-------------------------------------------------------------------------------|------------------------------------------------------|
| M_PI, TWO_PI, M_PI_2                                                          | $\pi, 2\pi, \pi/2$ .                                 |
| DEGREES_TO_RADIANS,<br>RADIANS_TO_DEGREES                                     | Angle. Multiply degrees by the first to get radians. |
| KM_TO_METERS, AU_TO_METERS, and their<br>inverses                             | Length.                                              |
| MINUTES_TO_SECONDS,<br>HOURS_TO_SECONDS, DAYS_TO_SECONDS,<br>YEARS_TO_SECONDS | Time.                                                |
| SECONDS_TO_NANOSECONDS and the milli /<br>micro / nano set                    | Sub-second time.                                     |
| SPEED_OF_LIGHT                                                                | $2.998 \times 10^8$ m/s.                             |
| LBS_TO_KG, MPS_TO_KNOTS,<br>METERS_TO_FEET, PSI_TO_PASCAL, and<br>more        | Other unit conversions, each with its inverse.       |

Planet constants — gravitational parameter, equatorial radius, J2, J3, mean rotation rate — for the planets, the Moon, and the Sun are also available (they are what `SpicePlanet` uses by default; see Chapter 10).

# 10 Built-in Model Reference

---

This chapter catalogs the models that ship with WarpTwin. Every model obeys the same contract (Chapter 7): `params` set before `startup()`, `inputs` driven from other models' outputs, outputs read by downstream models and loggers, and the lifecycle methods `start()` / `execute()`.

## 10.1 How to Use This Chapter

For each model, the **delivered C++ header** (`/usr/include/warptwin/models/...`) is the authoritative and complete reference: it lists every signal with its type, default, and a Doxygen comment, and it is what the online reference is generated from. This chapter gives, for each model, its purpose and the handful of signals you actually set or read, and a short example for the ones you will use most.

Three things to know while reading:

- **Some models are wired for you.** The gravity, drag, gravity-gradient, and third-body models listed here are created and connected *inside* the Spacecraft assembly (Chapter 6). You reach them through accessors (`sc.asphericalGravityModel()` and friends) to tune or log them; you rarely construct them yourself.
- **Sensor models share a pattern.** Almost every sensor has a `mount_frame` param, a `rate_hz`, a `seed_value` (for repeatable noise), an optional `latency` (in milliseconds), and an `..._power_draw`. Each publishes both a `perfect_...` (truth) output and a `meas_...` (noisy) output, an `output_time`, and an `is_valid` flag. Feed the noise standard deviations in through the `inputs`.
- **Category = GUI palette group.** The sections below follow the groups `wt-gui` shows in its sidebar.

## 10.2 Assemblies

Assemblies bundle a rigid body with the models that make it behave like a real vehicle or place.

---

|                    |                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| Spacecraft         | Orbiting rigid body plus gravity, drag, gravity gradient, third-body, and planet-relative state. Chapter 6.                  |
| LaunchVehicle      | Powered atmospheric vehicle: gimballed throttleable engine, mass depletion, rocket aerodynamics, recovery chutes. Chapter 6. |
| SpicePlanet        | A planet, moon, or the Sun, with states from SPICE.                                                                          |
| CustomPlanet       | A body with hand-set constants and rotation, for bodies not in the kernels.                                                  |
| ADCS               | A reaction-wheel + torque-rod attitude determination and control bundle.                                                     |
| LaunchPadModel     | A launch pad / rail: holds a launch vehicle at a site and releases it up the rail.                                           |
| GroundStationModel | A site on a rotating planet, with range, azimuth, elevation, and a visibility mask to a spacecraft.                          |
| RadioModel         | A spacecraft radio with a full link budget to a peer radio.                                                                  |

---

## SpicePlanet

Assemblies • `from warptwin.SpicePlanet import SpicePlanet`

A planet, moon, or the Sun whose position, velocity, and orientation come from NASA SPICE. The constructor's second argument is the SPICE body name; the model registers itself as a gravitating body, so creation order does not matter.

### Params

| Signal          | Type   | Default | Meaning                                                                                    |
|-----------------|--------|---------|--------------------------------------------------------------------------------------------|
| enable_attitude | int    | true    | Publish the body's attitude / rotating frame.                                              |
| visuals_texture | string | ""      | Texture image name for the viewer.                                                         |
| ephemeris_body  | string | ""      | SPICE body for the position lookup when it differs from the name (e.g. "MARS BARYCENTER"). |

### Outputs

| Signal                            | Type    | Default | Meaning                                                                                       |
|-----------------------------------|---------|---------|-----------------------------------------------------------------------------------------------|
| self_id                           | planet  | -       | Handle for wiring pointer params (a spacecraft's planet_ptr, a gravity model's field source). |
| inertial_frame                    | frame   | -       | The body's inertial frame (attach spacecraft here).                                           |
| rotating_frame                    | frame   | -       | The body-fixed rotating frame.                                                                |
| eq_radius, flattening, mu, J2, J3 | double  | WGS-84  | Physical constants, from a built-in table.                                                    |
| mean_ang_vel                      | Vector3 | WGS-84  | Mean rotation rate, rad/s.                                                                    |

```
earth = SpicePlanet(exc, "earth")
moon = SpicePlanet(exc, "moon")
sc.params.planet_ptr(earth)
```

## CustomPlanet

Assemblies • `from warptwin.CustomPlanet import CustomPlanet`

A body you define by hand — constants and a constant rotation rate — for anything not in the SPICE kernels. Same frame and constant outputs as SpicePlanet, so a spacecraft configures from it identically.

### Params

| Signal           | Type       | Default    | Meaning                                                   |
|------------------|------------|------------|-----------------------------------------------------------|
| eq_radius        | double     | WGS-84     | Equatorial radius, m.                                     |
| flattening       | double     | WGS-84     | Ellipsoid flattening.                                     |
| mu               | double     | WGS-84     | Gravitational parameter, m <sup>3</sup> /s <sup>2</sup> . |
| J2, J3           | double     | WGS-84     | Zonal harmonics.                                          |
| omega            | Vector3    | Earth rate | Rotation rate vector, rad/s.                              |
| initial_attitude | Quaternion | identity   | Body-fixed frame attitude at epoch.                       |

## ADCS

Assemblies • `from warptwin.ADCS import ADCS`

A ready-made attitude determination and control assembly: a bank of reaction wheels and a bank of magnetic torque rods, with torque limits, momentum caps, saturation handling, power draw, and attitude-knowledge noise (Chapter 6, GN&C section).

### Params (selected)

| Signal                          | Type   | Default | Meaning                                   |
|---------------------------------|--------|---------|-------------------------------------------|
| sc_body                         | body   | -       | The controlled body (sc.body()).          |
| planet_ptr                      | planet | -       | The orbited body.                         |
| num_reaction_wheels             | uint   | 0       | Wheels in the bank.                       |
| rw_peak_torque, rw_momentum_cap | double | 0       | Per-wheel limits, N m and N m s.          |
| num_torque_rods                 | uint   | 0       | Rods in the bank.                         |
| att_knowledge_noise_std         | double | 0       | 1- $\sigma$ attitude estimate error, rad. |

### Inputs / Outputs

| Signal                                | Type       | Default | Meaning                                |
|---------------------------------------|------------|---------|----------------------------------------|
| mode                                  | int        | 0       | Pointing mode (see the header).        |
| desired_primary,<br>desired_secondary | frame      | -       | Target frames.                         |
| est_quat_body_ref                     | Quaternion | -       | Estimated attitude (the "nav" output). |
| total_angular_momentum                | Vector3    | -       | Stored wheel momentum, N m s.          |

## GroundStationModel

Assemblies • from warptwin.GroundStationModel import

GroundStationModel

A fixed site on a rotating planet. Give it the spacecraft body frame and the planet rotating frame, plus a geodetic latitude/longitude and an elevation mask; it publishes the look geometry and a visibility flag.

### Params / Outputs

| Signal                                   | Type   | Default | Meaning                              |
|------------------------------------------|--------|---------|--------------------------------------|
| spacecraft_frame                         | frame  | -       | The tracked spacecraft body frame.   |
| planet_rotating_frame                    | frame  | -       | From earth.outputs.rotating_frame(). |
| latitude_detic_rad,<br>longitude_rad     | double | 0       | Site location, rad.                  |
| elevation_mask_rad                       | double | 0       | Horizon mask.                        |
| range, range_rate, azimuth,<br>elevation | double | -       | Look geometry.                       |
| visible                                  | bool   | -       | Above the mask and in view.          |

## 10.3 Gravity and Orbital Dynamics

These produce the accelerations a spacecraft integrates. Inside Spacecraft the right ones are already wired; construct them yourself only for a bespoke setup.

|                                |                                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------|
| AsphericalGravityModel         | Point mass plus J2 and J3. The default field for a Spacecraft's central body.       |
| SphericalHarmonicsGravityModel | Full field to degree/order $n, m$ from a coefficient file (WGS-84 shipped).         |
| PointMassGravityModel          | Point mass only.                                                                    |
| NBodyPerturbationModel         | Sum of third-body point-mass perturbations (Chapter 8). Auto-wired; not in the GUI. |
| GravityGradientModel           | Gravity-gradient torque from the inertia tensor.                                    |
| CR3BPDynamicsModel             | Circular restricted three-body dynamics, for libration-point and NRHO work.         |

## AsphericalGravityModel

Gravity • via `sc.asphericalGravityModel()`

### Params / key signals

| Signal                     | Type          | Default | Meaning                                                               |
|----------------------------|---------------|---------|-----------------------------------------------------------------------|
| <code>mu</code>            | double        | WGS-84  | Gravitational parameter, $\text{m}^3/\text{s}^2$ .                    |
| <code>J2, J3</code>        | double        | WGS-84  | Zonal harmonics.                                                      |
| <code>r_planet</code>      | double        | WGS-84  | Reference (equatorial) radius, m.                                     |
| <code>body_mass</code>     | double        | 1.0     | Spacecraft mass, kg — only used to scale <code>grav_force__f</code> . |
| <code>pos_body__f</code>   | Vector3 (in)  | –       | Spacecraft position in the field frame.                               |
| <code>grav_accel__f</code> | Vector3 (out) | –       | Resulting acceleration, $\text{m}/\text{s}^2$ .                       |
| <code>grav_force__f</code> | Vector3 (out) | –       | Acceleration $\times$ <code>body_mass</code> , N.                     |

## SphericalHarmonicsGravityModel

Gravity • via `sc.sphericalHarmGravityModel()`

Set `n` and `m` to the degree and order you want (both default 0, i.e. point mass); `filename` points at a coefficient file (the shipped `WGS84.txt` by default). Its position input is in the planet-centered planet-fixed frame.

## GravityGradientModel

Gravity • via `sc.gravityGradientModel()`

Takes the spacecraft position, attitude, and body inertia tensor; outputs `ggTorque_body_body`, the gravity-gradient torque in the body frame. Only meaningful once you give the spacecraft a real `inertia`.

## 10.4 Atmosphere, Drag, and Aerodynamics

|                                                                                                                    |                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MSISAtmosphereModel</code>                                                                                   | NRLMSISE-00 density, temperature, and speed of sound. The atmosphere source for Earth. Auto-wired inside <code>Spacecraft</code> ; not in the GUI. |
| <code>FlatPlateDragModel</code>                                                                                    | Drag force from $C_d$ , area, density, and wind-relative velocity. The <code>Spacecraft</code> drag model.                                         |
| <code>FlatPlateDrag3D</code>                                                                                       | Drag with a direction-dependent area.                                                                                                              |
| <code>LaunchVehicleAerodynamicForceModel</code>                                                                    | Axial/normal force and dynamic pressure for a slender body, referenced to the nose.                                                                |
| <code>AerodynamicsStateModel</code>                                                                                | Mach, dynamic pressure, angle of attack, and related aero state.                                                                                   |
| <code>Parachute</code>                                                                                             | Canopy drag with a modeled deployment (untangle then fill).                                                                                        |
| <code>SimpleMeanWindModel</code> ,<br><code>SimpleDisturbanceWindModel</code> ,<br><code>CompositeWindModel</code> | Wind fields to feed the drag models.                                                                                                               |

## FlatPlateDragModel

Drag • via `sc.atmosDrag()`

### Signals

| Signal        | Type           | Default | Meaning                                                           |
|---------------|----------------|---------|-------------------------------------------------------------------|
| cd            | double (param) | 1.0     | Drag coefficient.                                                 |
| area          | double (in)    | 1.0     | Reference area, m <sup>2</sup> .                                  |
| density       | double (in)    | 0.0     | Local air density, kg/m <sup>3</sup> (from the atmosphere model). |
| wind_velo__f  | Vector3 (in)   | 0       | Wind-relative velocity in the force frame, m/s.                   |
| drag_force__f | Vector3 (out)  | -       | Drag force, N.                                                    |

## Parachute

Actuators • from `warptwin.Parachute` import `Parachute`

### Signals

| Signal                       | Type          | Default | Meaning                                      |
|------------------------------|---------------|---------|----------------------------------------------|
| cd                           | double        | 1.0     | Canopy drag coefficient.                     |
| full_area                    | double        | 1.0     | Fully inflated canopy area, m <sup>2</sup> . |
| untangle_time_s, fill_time_s | double        | 0       | Deployment phases.                           |
| deploy_command               | bool (in)     | false   | Rising edge begins deployment.               |
| percent_open                 | double (out)  | 0       | Inflation fraction.                          |
| drag_force__f                | Vector3 (out) | -       | Current drag force, N.                       |

## 10.5 Other Environment

|                             |                                                                                                   |
|-----------------------------|---------------------------------------------------------------------------------------------------|
| SolarRadiationPressureModel | SRP force from solar flux, a reflectivity coefficient, and an effective area.                     |
| OccultationModel            | Fraction of a source (e.g. the Sun) blocked by a sphere (e.g. a planet) as seen from an observer. |
| DipoleMagneticFieldModel    | Tilted-dipole Earth field.                                                                        |
| WorldMagneticFieldModel     | Spherical-harmonic (WMM) Earth field.                                                             |

## SolarRadiationPressureModel

Environment • from `warptwin.SolarRadiationPressureModel`

import `SolarRadiationPressureModel`

### Signals

| Signal              | Type           | Default | Meaning                                                   |
|---------------------|----------------|---------|-----------------------------------------------------------|
| S                   | double (param) | 1361.0  | Solar flux at 1 AU, W/m <sup>2</sup> .                    |
| C_r                 | double (in)    | 1.0     | Reflectivity coefficient (1 = absorbing, 2 = reflecting). |
| A_srp               | double (in)    | 1.0     | Effective sunlit area, m <sup>2</sup> .                   |
| position_body_sun_f | Vector3 (in)   | -       | Spacecraft position relative to the Sun.                  |
| srp_force_body_f    | Vector3 (out)  | -       | SRP force, N.                                             |

## 10.6 Actuators and Propulsion

|                         |                                                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| SimpleThrusterModel     | Throttleable thruster: thrust, mass flow, and $I_{sp}$ at a body location and direction, with bias, noise, and latency. |
| ReactionWheelModel      | One reaction wheel: torque about an axis, with peak torque, a momentum cap, saturation, and power draw.                 |
| TorqueRodModel          | One magnetic torque rod: torque from a current in a local magnetic field, with resistance and power.                    |
| TorqueCoilModel         | Air-core magnetic coil variant of the torque rod.                                                                       |
| Servo                   | A commanded rotary actuator with rate and torque limits.                                                                |
| TimedImpulsiveBurnModel | Applies a fixed $\Delta v$ at a set simulation or UTC time. <code>sc.programManeuver(...)</code> creates one.           |
| PropellantTankModel     | Propellant mass, a shifting center of gravity, and depletion, mounted at a frame.                                       |

### SimpleThrusterModel

Actuators • from `warptwin.SimpleThrusterModel` import

SimpleThrusterModel

#### Params / signals

| Signal                            | Type         | Default | Meaning                                                                       |
|-----------------------------------|--------------|---------|-------------------------------------------------------------------------------|
| control_body                      | body         | -       | The body the thrust acts on.                                                  |
| thruster_location__body           | Vector3      | 0       | Nozzle position in the body frame, m.                                         |
| thruster_pointing__body           | Vector3      | 0       | Thrust direction (unit) in the body frame.                                    |
| thrust_max, mdot_max              | double       | 0       | Full-throttle thrust (N) and mass flow (kg/s).                                |
| thrust_bias, thrust_noise_std     | double       | 0       | Error terms.                                                                  |
| latency                           | int          | 0       | Command-to-thrust delay, in milliseconds (rounded up to the next whole step). |
| throttle                          | double (in)  | 0       | Commanded throttle, 0-1.                                                      |
| applied_thrust, applied_mdot, Isp | double (out) | -       | Realized values.                                                              |

```
from warptwin.SimpleThrusterModel import SimpleThrusterModel
from warptwin.WarpTwinPy import CartesianVector3, connectSignals

thr = SimpleThrusterModel(exc, "thr")
thr.params.control_body(sc.body())
thr.params.thruster_pointing__body(CartesianVector3([0.0, 0.0, 1.0]))
thr.params.thrust_max(22.0)      # N
thr.params.mdot_max(0.0073)     # kg/s

connectSignals(autopilot.outputs.throttle_cmd, thr.inputs.throttle)
```

### ReactionWheelModel

Actuators • from `warptwin.ReactionWheelModel` import

ReactionWheelModel

### Params / signals

| Signal                    | Type        | Default | Meaning                                           |
|---------------------------|-------------|---------|---------------------------------------------------|
| sc_body                   | body        | -       | The controlled body.                              |
| wheel_normal_axis__body   | Vector3     | [0,0,1] | Spin axis in the body frame.                      |
| peak_torque, momentum_cap | double      | 0       | Limits, N m and N m s.                            |
| mom_inertia               | double      | 0       | Wheel inertia about its axis, kg m <sup>2</sup> . |
| active_power_draw,        | double      | 0       | Power, W.                                         |
| idle_power_draw           |             |         |                                                   |
| torque_com                | double (in) | 0       | Commanded torque, N m.                            |
| momentum, wheel_speed,    | (out)       | -       | Wheel state.                                      |
| saturated, power_draw,    |             |         |                                                   |
| applied_torque            |             |         |                                                   |

## TimedImpulsiveBurnModel

GN&C • from warptwin.TimedImpulsiveBurnModel import

TimedImpulsiveBurnModel

### Params

| Signal           | Type    | Default | Meaning                                              |
|------------------|---------|---------|------------------------------------------------------|
| body_frame       | body    | -       | The body to apply the impulse to.                    |
| burn_frame       | frame   | -       | The frame <code>deltav__burn</code> is expressed in. |
| time_of_ignition | Time    | never   | When to fire.                                        |
| deltav__burn     | Vector3 | 0       | The $\Delta v$ vector, m/s.                          |

Usually created for you by `sc.programManeuver(dv, time, frame)`, which returns the model so you can inspect or re-arm it.

## PropellantTankModel

Propulsion • from warptwin.PropellantTankModel import

PropellantTankModel

### Params / signals

| Signal                       | Type        | Default | Meaning                               |
|------------------------------|-------------|---------|---------------------------------------|
| mount_frame                  | frame       | -       | Where the tank is mounted.            |
| propellant_mass_init,        | double      | 0       | Loaded and capacity, kg.              |
| propellant_mass_max          |             |         |                                       |
| dry_mass                     | double      | 0       | Empty tank mass, kg.                  |
| pos_cg_prop_full__tank,      | Vector3     | 0       | CG at full / empty, for the shift.    |
| pos_cg_prop_empty__tank      |             |         |                                       |
| mdot                         | double (in) | 0       | Outflow rate, kg/s (from a thruster). |
| propellant_mass, total_mass, | (out)       | -       | Tank state.                           |
| pos_cg__tank, depleted       |             |         |                                       |

## 10.7 Sensors

Every sensor follows the pattern in Section 10 above: a `mount_frame`, a `rate_hz`, a `seed_value`, optional latency (milliseconds), a truth output and a measured output, an `output_time`, and an `is_valid` flag. Noise standard deviations come in through the inputs.

|                            |                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------|
| IMU                        | Combined accelerometer + gyroscope, with bias, random walk, scale factor, resolution, and range limits.    |
| Gyroscope, Accelerometer   | The two halves on their own.                                                                               |
| StarTracker                | Attitude quaternion relative to a reference frame, with cross-axis and twist accuracy and a field of view. |
| SunSensor                  | Unit look vector to the Sun, with a field of view and an eclipse check.                                    |
| Magnetometer               | Local field vector, fed from a magnetic field model.                                                       |
| GPS                        | Position and velocity in the planet-rotating frame, with altitude and speed limits (COCOM).                |
| SimpleCamera               | A pointed imager: target in frame, range, look angle.                                                      |
| FrameStateSensorModel      | The generic sensor — relative position, velocity, attitude, and rates between any two frames.              |
| OrbitalElementsSensorModel | Instantaneous classical elements from position and velocity.                                               |
| GroundStationSensor        | Range and range-rate to a ground station, with navigation noise.                                           |

## IMU

Sensors • from warptwin.IMU import IMU

### Params (selected)

| Signal                      | Type    | Default | Meaning                        |
|-----------------------------|---------|---------|--------------------------------|
| mount_frame                 | frame   | -       | The body the IMU is bolted to. |
| mount_position_mf,          | -       | -       | Lever arm and misalignment.    |
| mount_alignment_mf          |         |         |                                |
| rate_hz                     | int     | 0       | Output rate (0 = every step).  |
| accelerometer_seed_value,   | int     | 0       | Noise seeds.                   |
| gyro_seed_value             |         |         |                                |
| accelerometer_bias_initial, | Vector3 | 0       | Turn-on bias.                  |
| gyroscope_bias_initial      |         |         |                                |
| latency                     | int     | 0       | Output delay, in milliseconds. |

### Inputs / Outputs (selected)

| Signal                         | Type          | Default | Meaning                           |
|--------------------------------|---------------|---------|-----------------------------------|
| gyroscope_gaussian_noise,      | Vector3 (in)  | 0       | Noise 1- $\sigma$ / random walk.  |
| gyroscope_walking_bias_std     |               |         |                                   |
| meas_ang_vel_sf, meas_accel_sf | Vector3 (out) | -       | Measured rate and specific force. |
| perfect_ang_vel_sf,            | Vector3 (out) | -       | Truth, for comparison.            |
| perfect_accel_sf               |               |         |                                   |
| gyro_bias, accel_bias          | Vector3 (out) | -       | Current bias estimate.            |

```

from warptwin.IMU import IMU
from warptwin.WarpTwinPy import CartesianVector3

imu = IMU(exc, "imu")
imu.params.mount_frame(sc.body())
imu.params.rate_hz(100)
imu.params.gyro_seed_value(1)
imu.inputs.gyroscope_gaussian_noise(CartesianVector3([1e-4, 1e-4, 1e-4]))
# ... after startup, read imu.outputs.meas_ang_vel_sf each step

```

## StarTracker

Sensors • from warptwin.StarTracker import StarTracker

### Params / Outputs

| Signal                                 | Type                | Default | Meaning                                                 |
|----------------------------------------|---------------------|---------|---------------------------------------------------------|
| reference_frame                        | frame               | –       | The frame the measured quaternion is relative to.       |
| cross_axis_accuracy,<br>twist_accuracy | double              | 0       | 1- $\sigma$ errors, rad.                                |
| fov                                    | double              | $\pi/6$ | Field of view, rad (for the Sun/Earth exclusion check). |
| meas_quat_sf_ref                       | Quaternion<br>(out) | –       | Measured attitude.                                      |
| perfect_quat_sf_ref                    | Quaternion<br>(out) | –       | Truth attitude.                                         |

## GPS

Sensors • from warptwin.GPS import GPS

### Params / Outputs

| Signal                                             | Type          | Default | Meaning                                                     |
|----------------------------------------------------|---------------|---------|-------------------------------------------------------------|
| earth_rotating_frame                               | frame         | –       | Output is in this frame.                                    |
| max_altitude, max_speed                            | double        | COCOM   | Denial limits; outside them <code>is_valid</code> is false. |
| meas_pos_GPS_PCR,<br>meas_vel_GPS_PCR              | Vector3 (out) | –       | Measured position / velocity.                               |
| latitude_detic,<br>longitude_detic, altitude_detic | double (out)  | –       | Geodetic fix.                                               |

## FrameStateSensorModel

Sensors • from warptwin.FrameStateSensorModel import

FrameStateSensorModel

The general-purpose sensor: give it two frames and it publishes the full relative state — position, velocity, attitude, angular velocity — between them. Spacecraft uses two internally for its planet-relative outputs.

## 10.8 Navigation, Guidance, and Control

|                                                         |                                                                                                  |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| StochasticNavigation                                    | Truth 6-DOF state in, a biased and noisy estimate out — a stand-in for a real navigation filter. |
| ADCS                                                    | Closed-loop attitude control (in <i>Assemblies</i> above).                                       |
| TimedImpulsiveBurnModel                                 | Scheduled $\Delta v$ (in <i>Actuators</i> above).                                                |
| DirectionalAdaptiveGuidance,<br>PidTranslationalControl | Reference guidance and control laws, usable from a script.                                       |

The GN&C *flight-software* apps — `SingleAxisPointingControl`, `TwoAxisPointingGuidance`, `PdAttitudeControl`, `UnifiedPoweredFlightGuidance`, and the inertial- and spacecraft-navigation EKF suites — are WarpOS apps rather than WarpTwin models. They run in the loop the same way real flight code does; Chapter 16 covers them.

## StochasticNavigation

GN&amp;C • from warptwin.StochasticNavigation import

StochasticNavigation

### Signals

| Signal                                         | Type           | Default | Meaning                                  |
|------------------------------------------------|----------------|---------|------------------------------------------|
| rate_hz                                        | double (param) | 0       | Estimate update rate.                    |
| seed_value                                     | int (param)    | 0       | Noise seed — set it per Monte Carlo run. |
| pos_bias_random_walk_std, ...                  | Vector (param) | 0       | Random-walk $1-\sigma$ on each state.    |
| pos_perf, vel_perf, quat_perf, angvel_perf     | (in)           | -       | Truth state, from the vehicle.           |
| ..._gaussian_noise_std                         | (in)           | 0       | White-noise $1-\sigma$ on each state.    |
| pos_stoch, vel_stoch, quat_stoch, angvel_stoch | (out)          | -       | The estimate.                            |

```
from warptwin.StochasticNavigation import StochasticNavigation
from warptwin.WarpTwinPy import connectSignals, END_STEP
```

```
nav = StochasticNavigation(exc, END_STEP, "nav")
nav.params.seed_value(exc.runNumber())
connectSignals(sc.outputs.pos_sc_pci, nav.inputs.pos_perf)
connectSignals(sc.outputs.vel_sc_pci, nav.inputs.vel_perf)
connectSignals(sc.outputs.quat_sc_pci, nav.inputs.quat_perf)
```

## 10.9 Power

Register a source's or load's power output with the spacecraft (`sc.addPowerSource(...)` / `sc.addPowerLoad(...)`) and it folds into `sc.outputs.total_power_in` / `total_power_out`.

|                         |                                                                                                                                                                   |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SolarPanelModel         | Panel power from Sun geometry, area, efficiency, pointing, and eclipse.                                                                                           |
| SolarPanelPowerModel    | The bare power calculation — efficiency $\times$ irradiance $\times$ effective area $\times$ solar intensity — when you already have the geometry from elsewhere. |
| SimpleBatterySystem     | State of charge from net power, with a voltage/capacity curve and a cutoff.                                                                                       |
| PowerLoad               | A load with active / idle / off draw, selected by a mode input.                                                                                                   |
| EffectiveSolarAreaModel | The sunlit projected area of a body, for SRP or thermal work.                                                                                                     |

## SolarPanelModel

Power • from warptwin.SolarPanelModel import SolarPanelModel

### Params / Outputs

| Signal                                                | Type         | Default | Meaning                                         |
|-------------------------------------------------------|--------------|---------|-------------------------------------------------|
| panel_area                                            | double       | 1.0     | Cell area, m <sup>2</sup> .                     |
| panel_eff                                             | double       | 0.28    | Conversion efficiency.                          |
| solar_irr                                             | double       | 1361.0  | Irradiance at 1 AU, W/m <sup>2</sup> .          |
| panel_normal__body                                    | Vector3      | [1,0,0] | Panel normal in the body frame.                 |
| sun_frame_ptr,<br>planet_frame_ptr,<br>body_frame_ptr | frame        | –       | Geometry references.                            |
| power                                                 | double (out) | –       | Generated power, W – pass to sc.addPowerSource. |
| vis_frac                                              | double (out) | 1.0     | Sun-visible fraction (0 in eclipse).            |

## SimpleBatterySystem

Power • from warptwin.SimpleBatterySystem import

SimpleBatterySystem

### Params / signals

| Signal                                                 | Type        | Default | Meaning                          |
|--------------------------------------------------------|-------------|---------|----------------------------------|
| single_battery_capacity                                | double      | 0       | Per-cell capacity, W h.          |
| num_batteries_series,<br>num_batteries_parallel        | int         | 1       | Pack topology.                   |
| initial_charge_state                                   | double      | 1.0     | Starting fraction.               |
| shutoff_capacity                                       | double      | 0       | Cutoff.                          |
| power_generation_in,<br>power_draw_out                 | double (in) | 0       | Wire from the spacecraft totals. |
| depth_of_discharge,<br>system_voltage, power_available | (out)       | –       | Battery state.                   |

## PowerLoad

Power • from warptwin.PowerLoad import PowerLoad

Three draw levels (active\_power\_draw\_W, idle\_power\_draw\_W, off\_power\_draw\_W) selected by the mode input; publishes power\_draw\_W. Register it with sc.addPowerLoad(...).

## 10.10 Communications and Access

|                        |                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------|
| RadioModel             | Full link budget between two radios: EIRP, free-space path loss, $E_b/N_0$ , link margin, and power draw. |
| SimpleComAnalysisModel | A lighter link-analysis model.                                                                            |
| SpacecraftLinkModel    | Range, range-rate, and look angles between two spacecraft, with a visibility flag.                        |
| EarthObservationModel  | Access of a ground target from a spacecraft: range, look angle, off-nadir mask, visit count.              |

## EarthObservationModel

Sensors • from warptwin.EarthObservationModel import

EarthObservationModel

### Params / Outputs

| Signal                         | Type         | Default | Meaning                               |
|--------------------------------|--------------|---------|---------------------------------------|
| spacecraft_frame               | frame        | -       | The observing spacecraft.             |
| planet_rotating_frame          | frame        | -       | From earth.outputs.rotating_frame().  |
| target_lat_rad, target_lon_rad | double       | 0       | The site.                             |
| off_nadir_mask_rad             | double       | 1.0     | Max off-nadir angle for a valid look. |
| range, range_rate, look_angle  | double (out) | -       | Look geometry.                        |
| visible, total_visits,         | (out)        | -       | Access flag and tallies.              |
| obs_time_s                     |              |         |                                       |

## 10.11 State Initialization

OrbitalElementsStateInit

Classical elements → inertial position and velocity.

LlaDeticStateInit

Geodetic latitude/longitude/altitude → position and velocity on a rotating planet.

## OrbitalElementsStateInit

Initial States • from warptwin.OrbitalElementsStateInit import

OrbitalElementsStateInit

### Params / Outputs

| Signal                     | Type          | Default   | Meaning                                      |
|----------------------------|---------------|-----------|----------------------------------------------|
| mu                         | double        | WGS-84    | Gravitational parameter of the central body. |
| a                          | double        | 6878140.0 | Semimajor axis, m.                           |
| e, i, RAAN, w, f           | double        | -         | Eccentricity and the four angles, rad.       |
| pos_inertial, vel_inertial | Vector3 (out) | -         | Resulting state.                             |

This is the model form of `sc.initializeFromOrbitalElements(...)`; use it when you want the state vector as a signal to wire elsewhere.

# 11 Logging and Data Analysis

A simulation is only useful if you can get the numbers out of it and work with them. WarpTwin has two halves for that: **loggers** that write chosen signals to disk during a run, and a set of Python helpers for loading, plotting, and reporting on the result.

## 11.1 Loggers

Two loggers ship with WarpTwin. They have the same interface; they differ only in the file they write:

|            |                                                                                                                                    |
|------------|------------------------------------------------------------------------------------------------------------------------------------|
| Hdf5Logger | Binary HDF5. Exact values, compact, fast to read back. <b>The recommended default.</b>                                             |
| CsvLogger  | Plain-text CSV. Human-readable and openable anywhere, but larger and subject to text rounding (20 significant figures by default). |

Both are used in three steps: construct the logger, add the signals you want, hand it to the log manager with a rate.

```
from warptwin.WarpTwinPy import Hdf5Logger

states = Hdf5Logger(exc, "sc_states.h5")           # 1. construct
states.addParameter(exc.time().base_time, "t")     # 2. add signals
states.addParameter(sc.outputs.pos_sc_pci, "pos_eci_m")
states.addParameter(sc.outputs.vel_sc_pci, "vel_eci_mps")
exc.logManager().addLog(states, 10)                # 3. run it at 10 Hz
```

The constructor also takes a buffer size (`Hdf5Logger(exc, "x.h5", buffer_size=1000)`) — the number of samples held in memory before a flush to disk. The default is small; raise it for a fast logger on a long run.

## 11.2 Choosing What to Log

`addParameter` takes the signal to record and the name to record it under. The signal can be given as **an object** — the usual form — or **as an address string**, which is handy when it is buried in a sub-model:

```
log.addParameter(sc.outputs.pos_sc_pci, "pos")      # by object
log.addParameter(".exc.sc.pr.outputs.altitude", "alt") # by address
```

Find an address with `exc.search(...)` or in `graph_tree.json` (Chapter 7).

A third argument, `recursive=True`, logs the signal *and every loggable signal beneath it*, naming each child `<name>_<child_name>`. Passing a whole model's outputs group recursively is a quick way to capture everything it produces.

**Note.** A **vector or matrix signal expands to one column per element**, named with a numeric suffix: `pos_eci_m_0`, `pos_eci_m_1`, `pos_eci_m_2` for a 3-vector. In HDF5 it is stored as a single 2-D dataset and expanded on load; in CSV it is written as separate columns.

Always log `exc.time().base_time` as your time column — it is the running simulation clock. The other clocks (`tdb_time`, `utc_time`, ...) are available the same way (Chapter 7).

## 11.3 Rates and the Log Manager

`exc.logManager().addLog(logger, rate)` registers a logger and runs it, at `END_STEP`, at the given rate:

---

|                                    |                                           |
|------------------------------------|-------------------------------------------|
| <code>addLog(log, 10)</code>       | An integer is a rate in hertz.            |
| <code>addLog(log, Time(60))</code> | A Time is a period — here, once a minute. |

---

You may register several loggers at different rates and filenames — a fast one for attitude, a slow one for the orbit — each with its own `addParameter` set. Parameters must all be added *before* the run starts; the logger locks its column set at `startup()`.

## 11.4 Where the Output Goes

Everything a run writes lands in the output directory — `results/` by default, changed with `--out-dir` or `exc.logManager().outDir("...")`. A single logger can be redirected with `logger.outDir(...)`. The filename is the one you passed to the constructor (a missing extension is filled in).

Passing `--save-all-outputs=true` additionally writes `all_outputs.csv` containing every output signal in the simulation. It is exhaustive and large; use it to discover what is available, not for routine runs.

## 11.5 CSV and HDF5 in Practice

### 11.5.1 CSV

`CsvLogger` writes a header row of column names, then one row per sample. Two knobs:

---

|                                   |                                                     |
|-----------------------------------|-----------------------------------------------------|
| <code>log.setPrecision(10)</code> | Significant figures written per value (default 20). |
| <code>log.delimiter(";")</code>   | The field separator (default ,).                    |

---

### 11.5.2 HDF5

`Hdf5Logger` writes one dataset per scalar column and one 2-D dataset per vector signal. It stores values exactly. To browse an `.h5` file without code, the [H5Web](#) extension for VS Code shows its datasets and plots them inline.

## 11.6 The JSON Companions

Unless you pass `--write-data-json=false`, every run also writes, into the output directory:

---

|                              |                                                                                                   |
|------------------------------|---------------------------------------------------------------------------------------------------|
| <code>sim_data.json</code>   | Every model's params as they were run. A record of exactly what you simulated.                    |
| <code>graph_tree.json</code> | The complete address map of the simulation, each end signal tagged with its type and final value. |

---

## 11.7 Loading Data in Python

### 11.7.1 One run

```
import pandas as pd
from warptwinutils.analysisutils import readH5Dataframe

df_h5 = readH5Dataframe("results/sc_states.h5")    # HDF5 -> DataFrame
df_csv = pd.read_csv("results/sc_states.csv")      # CSV -> DataFrame

df_h5["speed"] = (df_h5["vel_eci_mps_0"]**2 +
                  df_h5["vel_eci_mps_1"]**2 +
                  df_h5["vel_eci_mps_2"]**2) ** 0.5
```

`readH5Dataframe` expands vector datasets into `_0/_1/...` columns, matching what `CsvLogger` writes, so analysis code is the same for either format. Pass `usecols=[...]` (base names) to load only part of a large file.

### 11.7.2 A Monte Carlo set

```
from warptwinutils.analysisutils import loadFilesMultiRun

runs = loadFilesMultiRun(path="results", filename="sc_states.h5",
                          runpath="run_")
```

This gathers every `results/run_<n>/sc_states.h5` into one structure for ensemble analysis (Chapter 13).

## 11.8 Loading Data in MATLAB

HDF5 reads natively:

```
% one dataset -> a column vector (or an N-by-M array for a vector signal)
t = h5read('results/sc_states.h5', '/t');
pos = h5read('results/sc_states.h5', '/pos_eci_m');
```

## 11.9 Analysis Helpers

warptwinutils carries the plotting and analysis functions the shipped examples use. The main ones, all in `warptwinutils.analysisutils`:

---

|                                             |                                                                                             |
|---------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>plotGoogleEarth(lat, lon, alt)</code> | Write a trajectory to a Google Earth KML. Latitude and longitude in <b>degrees</b> .        |
| <code>plotGoogleEarthPoints(...)</code>     | The same for a set of discrete points.                                                      |
| <code>plotImpactScatter(...)</code>         | An impact-dispersion scatter (for Monte Carlo landing footprints) as a KML.                 |
| <code>plotMcOverlay(...)</code>             | Overlay a scalar against time across every run of a Monte Carlo set.                        |
| <code>plotStateAndCovariance(...)</code>    | Estimate-vs-truth with a $\pm n\sigma$ covariance envelope, for navigation-filter analysis. |
| <code>createPdfCdf(data)</code>             | A probability density / cumulative distribution plot for a sample.                          |
| <code>generateTimingReport(jsonPath)</code> | A runtime-performance report from a run's timing JSON.                                      |

---

The package also has `orbitutils` (orbital-element helpers, including a sun-synchronous element solver), `lambert` (a Lambert solver), and `reports` (canned communications- and power-budget report builders).

## 11.10 Automatic Reports: AutoDocPy

AutoDocPy builds a formatted report from plots and tables you add to it: it writes an AsciiDoc (.adoc) file and renders it to HTML with `asciidoctor`. (Producing a PDF is a further `asciidoctor-pdf` step you run yourself.)

```
import matplotlib.pyplot as plt
from warptwinutils.AutoDocPy import AutoDocPy
from warptwinutils.analysisutils import readH5Dataframe

df = readH5Dataframe("results/sc_states.h5")

doc = AutoDocPy()
doc.title("Orbit Propagation Results")
doc.author("A. Engineer", "a.engineer@example.com")
doc.file("orbit_report.adoc")

plt.plot(df["t"] / 60.0, df["pos_eci_m_2"] / 1000.0)
plt.xlabel("time [min]"); plt.ylabel("z [km]")
doc.addMatPlotLib(plt, "Out-of-plane position")

doc.addDataFrame(df.describe())
doc.closeGenerateDoc()
```

`addPlotly(fig, name)` adds an interactive Plotly figure; `addDataFrame(df)` adds a table. Static image export from Plotly needs the optional `kaleido` package (Chapter 3); interactive HTML export needs nothing extra.

**Note.** `addMatplotlib` / `addPlotly` reference each figure by an *absolute* path on the machine that generated the report, so the `.adoc` and the HTML built from it are not portable — move them to another machine and the images will not resolve. Regenerate the report where it will be read, or fix up the `image:` paths before sharing it.

## 11.11 The Interactive Results Browser

WarpTwin ships a small Streamlit app for browsing and comparing run output without writing code. `wt-gui` launches it automatically after a run; to run it by hand, from its directory under `/usr/share/warptwin/python/plot-app/`:

```
python3 -m streamlit run datamanapp.py -- --data-dir ~/my_run/results
```

It scans the directory for `.h5` and `.csv` files and lets you pick columns to plot and overlay. It is an optional extra — its dependencies (Streamlit, Plotly) are installed separately (Chapter 3).

## 11.12 Stepping Through a Run: FrameViewer

To debug the geometry of a scenario rather than analyze its output, the `FrameViewer` helper opens an interactive window that draws every frame in the simulation and lets you step, play, and run to completion:

```
from warptwinutils.FrameViewer import FrameViewer

fv = FrameViewer(exc)  # place immediately before exc.run()
```

A frame-toggle list on the left hides and shows individual frames; the buttons advance the simulation while the plot updates. It is a diagnostic tool, not a data product — use it to confirm that a spacecraft, a sensor boresight, or a launch site is where you expect.

# 12 Visualization

---

WarpTwin sees a simulation through **MultiBodyViz**, its 3D viewer: planets, spacecraft, trajectories, sensor frustums, and ground tracks, rendered in a web browser, either after a run or streamed live during one. Relative motion — the kind a 2D plot would show directly — is read off the 3D view by selecting a custom viewing frame (Section 12.1.7), which re-expresses every trace in that frame.

## 12.1 MultiBodyViz: The 3D Viewer

### 12.1.1 Turning it on

Pass `--enable-visuals=true` on the command line — no change to the script:

```
python3 script.py --enable-visuals=true
```

This is the recommended pattern: one script runs both headless and with visuals, with nothing to edit between a batch run and an interactive look. To hard-wire the viewer into a script that should always open it, call `exc.enableVisuals()` before `startup()` instead. **Nothing else needs wiring** — every `SpicePlanet`, `CustomPlanet`, `Spacecraft`, and `LaunchVehicle` registers itself with the viewer when visuals are enabled. When the run completes, the viewer opens in your browser at `http://localhost:8002`. Under the container image the port is already published (Chapter 3); run with `./warptwin-docker.sh run script.py --enable-visuals=true` and open the same URL.

### 12.1.2 Browser requirements

MultiBodyViz renders with WebGPU and has no fallback. You need a browser that ships a working WebGPU implementation:

---

|                                  |                                               |
|----------------------------------|-----------------------------------------------|
| Chrome / Edge $\geq$ 113         | Works out of the box (Linux, macOS, Windows). |
| Safari $\geq$ 17.4 (macOS 14.5+) | Works out of the box.                         |
| Firefox                          | Nightly only, behind a flag.                  |
| Mobile browsers                  | Not supported.                                |

---

If the canvas is blank and the console says *no WebGPU adapter available*, the browser or GPU driver is not WebGPU-capable. On Linux, Chrome may need to be started with `--enable-features=Vulkan,UseSkiaRenderer`. Node.js is *not* required to view.

### 12.1.3 What the simulation writes

With visuals enabled, a run writes into its output directory (Chapter 11):

---

|                              |                                                                                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>visuals.json</code>    | The scene configuration — which bodies, spacecraft, frames, and colors. Gets <code>"complete": true</code> written to it when the run ends and the state files are flushed. |
| <code>visuals_sc.h5</code>   | Spacecraft and frame state over the run.                                                                                                                                    |
| <code>visuals_link.h5</code> | Inter-spacecraft link state, if any links were added.                                                                                                                       |

---

These land in the run's output directory alongside the logs, so giving each run its own `--out-dir` (or `exc.logManager().outDir(...)`) is all that is needed to keep concurrent runs on a shared server from overwriting each other's viewer output.

### 12.1.4 Reopening the viewer

The viewer opens itself once, at the end of a run. To open it again — on a finished run, on output copied from another machine, or on a different port — use the launcher:

---

|                                                    |                                    |
|----------------------------------------------------|------------------------------------|
| <code>warptwin-visuals</code>                      | View <code>./visuals.json</code> . |
| <code>warptwin-visuals path/to/visuals.json</code> | View a config elsewhere.           |
| <code>warptwin-visuals --port 8010 FILE</code>     | Serve on a different port.         |
| <code>warptwin-visuals --no-browser FILE</code>    | Start the server only.             |

---

The viewer polls, so pointing it at a run that is still in progress is fine — it stands by and populates when the state files close.

### 12.1.5 Watching a run live

To follow a run as it computes rather than after, put the visuals model in real-time mode:

```
exc.enableVisuals()
exc.visualsModel().params.real_time(True)    # or --real-time=true
```

The simulation then paces itself near real time and streams state to the viewer over UDP as it goes. Open `http://localhost:8002` while the run is underway.

### 12.1.6 Appearance

---

|                                                       |                                                                                                                                                                                                                                             |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sc.params.visuals_model("orion.glb")</code>     | Draw a 3D mesh for this spacecraft. Resolved against the shipped model library; <code>orion.glb</code> is included. Empty draws a plain marker.                                                                                             |
| <code>sc.params.visuals_color("#ff8800")</code>       | Marker, trail, and 2D ground track color, as an HTML hex string. Append two more hex digits (" <code>#ff8800cc</code> ") to set opacity. Empty leaves the viewer's default (cyan). The trail is drawn at about 70% of the marker's opacity. |
| <code>earth.params.visuals_texture("mars.jpg")</code> | Wrap a planet in a specific texture from the shipped texture library.                                                                                                                                                                       |

---

A viewer can also override any craft's color live from the swatch beside its trail toggle; that choice is not written back to the simulation.

### 12.1.7 Custom viewing frames

By default every trace is drawn in the inertial frame. Register any frame as a viewing frame and it appears in the viewer's frame bar:

```
exc.visualsModel().addFrame(sc.lvlh(), "SC LVLH")
exc.visualsModel().addFrame(chaser.body(), "Chaser body")
```

Selecting a custom frame re-expresses every spacecraft trace in it, removing *both* the frame's origin motion and its rotation – so a trajectory viewed in an LVLH frame, a CR3BP synodic frame, or a docking-port frame reads as relative motion instead of the inertial path. Markers stay in their world positions; only the traces are re-expressed.

### 12.1.8 Links, ground stations, and cameras

---

|                                                                     |                                                             |
|---------------------------------------------------------------------|-------------------------------------------------------------|
| <code>exc.visualsModel().addLink(link_model)</code>                 | Draw the line for a <code>SpacecraftLinkModel</code> .      |
| <code>exc.visualsModel().addGroundStation(gs_model)</code>          | Draw a <code>GroundStationModel</code> site and its access. |
| <code>exc.visualsModel().addBodyFixedCameraModel(frame, ...)</code> | A body-fixed camera view.                                   |

---

### 12.1.9 Sampling and trails

The visuals model has a few timing parameters, set through `exc.visualsModel().params`:

---

|                         |                                                                     |
|-------------------------|---------------------------------------------------------------------|
| <code>sc_rate</code>    | How often spacecraft state is sampled for the viewer. Default 10 s. |
| <code>trail_time</code> | How much trailing trajectory the viewer shows. Default 5400 s.      |
| <code>lead_time</code>  | How much of the future trajectory to show (when known).             |

---

### 12.1.10 Navigating the viewer

**Camera.** Left-drag orbits the focus target; middle-drag, right-drag, or Shift+left-drag pans; the wheel zooms. The focus panel snaps the camera to Home, Earth, the Moon, or the spacecraft (which it then follows). Panning detaches an active follow. On the first run the camera auto-frames the body the first spacecraft orbits; any camera input ends auto-framing for the session.

**Timeline.** The bottom panel has a play/pause button, a scrubber you can drag to seek anywhere in the run, and a speed dial – an STK-style shuttle ring covering  $0.1\times$  to  $10\,000\times$ , forward or reverse, with a center field you can type a rate into.

### 12.1.11 The world frame

The viewer's world is ICRF/J2000 equatorial, in meters:  $+Z$  is celestial north (the equatorial plane is  $Z = 0$ ),  $+X$  is the vernal equinox direction,  $+Y$  completes the right-handed set. The world origin is the central body – the most massive body in the scene (Chapter 8) – so adding a Sun to an Earth scene produces a heliocentric view; use the focus panel to snap back to Earth.

# 13 Monte Carlo and Dispersions

Monte Carlo analysis runs the same simulation many times, each time with slightly different inputs drawn from a statistical distribution, to see how uncertainty in those inputs propagates to the results. WarpTwin has a built-in dispersion engine for this, driven by a run number: **run 0 is always the nominal, undispersed case**, and every other run number selects a reproducible draw.

## 13.1 Declaring a Dispersion

A dispersion is a named quantity with a distribution. Declare it on the executive's dispersion engine, after `parseArgs` (which reads `--run` and seeds the engine):

```
d = exc.dispersions()

alt_km = d.createUniformInputDispersion(
    "alt_km", 500.0, 480.0, 520.0,          # name, run-0 value, min, max
    "Orbit altitude"                       # optional description
)

ltan = d.createNormalInputDispersion(
    "ltan", 6.0, 6.0, 1.0,                 # name, run-0 value, mean, std
    ↪ dev
    "Local time of the ascending node"
)
```

There are two distributions: `createUniformInputDispersion` (*name, run-0 value, min, max*) and `createNormalInputDispersion` (*name, run-0 value, mean, standard deviation*). The description is optional and appears in the dispersion file (Section 13.5).

The second argument is the value the simulation uses for **run 0**. Every other run draws from the distribution.

## 13.2 Using a Dispersion's Value

A dispersion object is called twice: once to get the tracked value, once to get the number:

```
value_object = alt_km()          # the tracked value (carries dispersion
    ↪ metadata)
drawn_number = alt_km()()        # the value drawn for this run
```

Apply it either by reading the number and setting a parameter, or — for full dispersion-to-model traceability — by connecting the value object to a model input:

```
from warptwin.WarpTwinPy import connectSignals

sc.params.mass(200.0 + alt_km())          # read and set
connectSignals(ltan(), some_model.inputs.ltan) # or connect for
    ↪ traceability
```

### Example 13.1 – A Monte Carlo script

The shape of every dispersed run. Dispersions are declared before `startup()`; their drawn values are read wherever they are needed.

```
import sys
from warptwin.WarpTwinPy import SimulationExecutive
from warptwin.SpicePlanet import SpicePlanet
from warptwin.Spacecraft import Spacecraft

exc = SimulationExecutive()
exc.args().addDefaultArgument("end", 15000)
exc.parseArgs(sys.argv)                # reads --run, --out-dir; seeds
    ↪ dispersions

earth = SpicePlanet(exc, "earth")
sc = Spacecraft(exc, "sc")
sc.params.planet_ptr(earth)

d = exc.dispersions()
a = d.createUniformInputDispersion("a_m", 6.878e6, 6.858e6, 6.898e6,
    ↪ "Semimajor axis")
ec = d.createNormalInputDispersion("ecc", 1e-3, 1e-3, 2e-4, "Eccentricity")

exc.startup()
sc.initializeFromOrbitalElements(a()( ), ec()( ), 0.0, 0.0, 0.0, 0.0)

exc.writeDispersionFile("./results/dispersions.adoc")
exc.run()
```

## 13.3 Run Numbers and Seeds

|                                  |                                                                                                                       |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>--run=<i>n</i></code>      | Selects the case. 0 is nominal; any other value is a reproducible dispersed draw.                                     |
| <code>--rng-seed=<i>s</i></code> | Shifts the <i>whole ensemble</i> deterministically. The draw for a given run number changes with the seed; default 0. |
| <code>exc.runNumber()</code>     | The current run number, inside the script.                                                                            |

Models with their own internal randomness – `StochasticNavigation`, a noisy sensor, a dispersed launch rail – take a `seed_value`. Set it from the run number so that noise is also reproducible per run, and special-case run 0 to be clean:

```
run = exc.runNumber()
nav.params.seed_value(run)
```

```
pad.params.rail_disp_deg(0.0 if run == 0 else 0.3) # no rail flex on the
↳ nominal run
```

## 13.4 Running a Set

A single run writes into its own directory:

```
python3 script.py --run=7 --out-dir=results/run_7
```

The shipped `multirun.sh` launcher does the whole sweep:

```
./multirun.sh -f script.py -n 500 -o results -j 8
```

- 
- f The simulation script.
  - n Number of runs. Executes run 0 (nominal) through n-1.
  - o Output directory (default results). Each run lands in <dir>/run\_<i>/.
  - j Maximum parallel jobs (default: CPU count).
  - a Extra arguments passed verbatim to every run.
- 

It runs the cases in parallel (using GNU `parallel` if present, otherwise `xargs`), shows a progress bar, and prints a summary naming any run that failed and where its log is. Because each run is an independent `python3 script.py --run=i` invocation, a cluster scheduler such as Slurm can drive the same sweep by dispatching that command per run number.

## 13.5 Recording the Dispersion Set

```
exc.writeDispersionFile("results/dispersions.adoc") # before exc.run()
```

or pass `--dispersions-file=results/dispersions.adoc`. A `.adoc` extension produces a formatted table (name, distribution, parameters, description, drawn value); `.txt` or `.csv` produce a plain list. Writing it into each run's output directory gives every result a record of exactly what it was run with.

## 13.6 Analyzing the Ensemble

`loadFilesMultiRun` gathers a sweep into a list of DataFrames, one per run (Chapter 11):

```
from warptwinutils.analysisutils import loadFilesMultiRun, plotMcOverlay

runs = loadFilesMultiRun(path="results", filename="states.h5",
↳ runpath="run_")

# overlay one quantity against time across every run
```

```
plotMcOverlay(runs, x="time", y="altitude_m")
```

From there, `plotMcOverlay`, `plotImpactScatter` (landing footprints), and `createPdfCdf` (distribution of a scalar outcome) cover the common plots, and `AutoDocPy` assembles them into a report (Chapter 11). The shipped `monte_carlo_sun_synchronous` and `orbital_insertion` examples each carry a full `script.py` + `multirun.sh` + `analysis.py` set and are the best templates to copy.

# 14 Worked Examples

---

Every example in this chapter ships with your delivery and runs out of the box. Each lives in its own directory under `/usr/share/doc/warptwin/` with a `script.py` and, usually, an `analysis.py` that plots the results. Copy one somewhere writable and run it:

```
cp -r /usr/share/doc/warptwin/examples/simple_gravity_spacecraft ~/
cd ~/simple_gravity_spacecraft
python3 script.py
python3 analysis.py
```

The examples are also the best templates to copy from — pick the one closest to what you are building and adapt it. This chapter is a reading order through them.

## 14.1 Start Here

### **simple\_gravity\_spacecraft**

A single spacecraft in an eccentric Earth orbit, propagated and logged, with a ground-track export. This is the example built line by line in Chapter 4.

### **body\_no\_spacecraft**

The same orbit, built *without* the Spacecraft assembly: raw frames, bodies, and nodes, with a `PointMassGravityModel` and a frame-state sensor wired up by hand. Slower to write, but it shows exactly what the assembly does for you (Chapter 8). A matching C++ version is referenced in the script for cross-language readers.

### **moon\_visuals**

A spacecraft around the Moon rather than the Earth, with the 3D viewer enabled — the smallest example that exercises a non-Earth central body and `MultiBodyViz` (Chapter 12).

## 14.2 Orbits and Maneuvers

### **geo\_transfer\_impulsive**

A LEO-to-GEO transfer solved with the built-in Lambert solver (`warptwinutils.lambert`). The script sweeps time-of-flight, picks the cheapest transfer, and schedules the two impulsive burns. Introduces `OrbitalElementsSensorModel`.

### **geo\_transfer\_electric**

The same endpoints with *continuous low thrust* instead: a GTO-to-GEO spiral flown under `DirectionalAdaptiveGuidance`, with a thruster's thrust,  $I_{sp}$ , and mass flow driving the mass depletion.

### **constellation**

Ten spacecraft built in a loop and spaced evenly around one orbit, each running an Earth-observation model against a ground site. The template for anything with repeated structure (Chapter 6).

## 14.3 Payloads and Subsystems

### **earth\_observation**

A sun-synchronous imager that slews to point at a sequence of ground targets, using `TwoAxisPointingGuidance` and `PdAttitudeControl` for the attitude loop, a `StarTracker` for attitude knowledge, and `EarthObservationModel` for access.

### **spacecraft\_power\_budget**

A full power chain over an orbit: `SolarPanelModel` generation, `OccultationModel` for eclipse, `SimpleBatterySystem` state of charge, and `PowerLoad` draws that change with mission mode. The reference for Chapter 10's Power section.

### **spacecraft\_communications\_link\_plot**

A `SpacecraftLinkModel` and a `GroundStationModel`: range, range-rate, look angles, and pass visibility over a run.

### **power\_beaming**

Two spacecraft in GEO with a power-transfer link between them, combining the link and power models. Its `analysis.py` is a thorough example of building a multi-figure report.

## 14.4 Advanced Dynamics

### **nrho\_circular\_restricted\_3\_body\_problem**

An Earth-Moon near-rectilinear halo orbit propagated with `CR3BPDynamicsModel` in the rotating (synodic) frame, using the `muStar` / `lStar` / `tStar` non-dimensionalization helpers, viewed in the 3D viewer's synodic frame (Chapter 12).

### **multi\_planet\_visuals**

Three spacecraft at once — one around the Moon, one around the Earth (with an ADCS holding a two-axis pointing constraint), one around Mars — with full n-body gravity and everything drawn in the 3D viewer (Chapter 8).

## 14.5 Launch

### **sounding\_rocket**

A single-stage sounding rocket flown off a near-vertical rail (`LaunchPadModel`), coasting to an ~80 km apogee and recovered under a drogue-then-main parachute system. The whole flight sequence — ignition, engine cutoff at tank depletion, drogue at apogee, main canopy low down — is flown from the script by writing the `LaunchVehicle`'s thrust and deploy inputs.

### **orbital\_insertion**

A closed-loop guided single-stage ascent to orbit, wrapped in a Monte Carlo dispersion sweep: `StochasticNavigation` feeding a pointing controller that drives the engine gimbal, with dispersed thrust, mass, aerodynamics, navigation error, and launch-rail flex. The most complete example in the set, and the reference for Chapters 6 and 13.

## 14.6 Monte Carlo

### **monte\_carlo\_sun\_synchronous**

The minimal dispersion example: a uniform altitude dispersion and a normal LTAN dispersion feeding

a sun-synchronous element solver, with `multirun.sh` to run the sweep and `analysis.py` to overlay the ensemble (Chapter 13).

## 14.7 Flight Software and Operations

### **rendezvous\_warplink\_connection**

Two spacecraft doing proximity operations in LEO under a translational and an attitude control loop, with WarpOS flight-software managers scheduled alongside, and — in `--real-time=true` mode — a live WarpLink connection so an operator can fly it (Chapters 16 and 17).

### **warpwall\_demo**

A simulated IMU bridged through an Icm20948 device-interface model to a real SPI buffer — a hardware-in-the-loop pattern for testing a driver against simulated sensor output.

## 14.8 Custom Analysis

### **custom\_spacecraft\_analysis**

A user-authored CustomSpacecraft class assembled from JSON component configurations, run through distinct mission modes (safe-mode tumble, nadir-pointing nominal), with self-eclipse, a ground-station elevation mask, and downlink-induced power draw. The template for wrapping your own vehicle model and analysis around WarpTwin (Chapter 15).

## 14.9 The Full List

| Example                                              | One line                                                  |
|------------------------------------------------------|-----------------------------------------------------------|
| <code>simple_gravity_spacecraft</code>               | Eccentric Earth orbit; propagate, log, ground track.      |
| <code>body_no_spacecraft</code>                      | The same, built from raw frames/bodies/nodes.             |
| <code>moon_visuals</code>                            | Lunar orbit with the 3D viewer.                           |
| <code>geo_transfer_impulsive</code>                  | LEO→GEO via the Lambert solver + two burns.               |
| <code>geo_transfer_electric</code>                   | GTO→GEO continuous low-thrust spiral.                     |
| <code>constellation</code>                           | Ten spacecraft in a loop, each observing a site.          |
| <code>earth_observation</code>                       | Sun-sync imager slewing between ground targets.           |
| <code>spacecraft_power_budget</code>                 | Solar / battery / load chain over an orbit, with eclipse. |
| <code>spacecraft_communications_link_plot</code>     | Link and ground-station access geometry.                  |
| <code>power_beaming</code>                           | Power transfer between two GEO spacecraft.                |
| <code>nrho_circular_restricted_3_body_problem</code> | Earth-Moon NRHO in the synodic frame.                     |
| <code>multi_planet_visuals</code>                    | Three spacecraft at Moon / Earth / Mars, full n-body.     |
| <code>sounding_rocket</code>                         | Rail launch, coast to apogee, parachute recovery.         |
| <code>orbital_insertion</code>                       | Guided single-stage ascent to orbit, Monte Carlo.         |
| <code>monte_carlo_sun_synchronous</code>             | Minimal dispersion sweep for a sun-sync orbit.            |
| <code>rendezvous_warplink_connection</code>          | Proximity ops, flight software, live WarpLink.            |
| <code>warpwall_demo</code>                           | Simulated IMU bridged to a real device interface.         |
| <code>custom_spacecraft_analysis</code>              | A user-authored vehicle model and mode analysis.          |
| <code>nesc/orbit_check_case_02</code>                | A propagation check against published NESC results.       |

# 15 Building Custom Models

WarpTwin is built for extension. A custom model is a small C++ class that follows the same contract as every built-in model — `params`, `inputs`, `outputs`, `start()`, `execute()` — and once compiled it is usable from Python scripts and from `wt-gui` exactly like a built-in one. You write custom models to represent your vehicle's subsystems, your sensors and actuators, or your GN&C algorithms.

You do not need the WarpTwin source to do this. WarpWare provides a **custom-model workspace** — a small, ready-made project that links against a precompiled developer build of WarpTwin delivered as a shared-library SDK (headers under `/usr/include/warptwin/`, libraries installed system-wide, data under `/usr/share/warptwin/`). You add your model's source to the workspace, build it, and the workspace produces the C++ library, the Python bindings, and the GUI metadata for you.

**Note.** The custom-model workspace is being streamlined for the WarpTwin 26.09 release — its bootstrap script and CMake setup are changing. This chapter covers the parts that are stable: the model contract, the shape of the workspace, and how a finished model is used. For the exact commands to configure and build, follow the workspace repository's own README.

## 15.1 The Model Contract

Every model — built-in or custom — is a class declared with the `MODEL` macro, exposing three groups of signals and overriding a small set of lifecycle methods.

**START\_PARAMS ... END\_PARAMS** Configuration, set once before the run. Each entry is `SIGNAL(name, type, default)`.

**START\_INPUTS ... END\_INPUTS** Values that change during the run, usually connected to another model's outputs.

**START\_OUTPUTS ... END\_OUTPUTS** Results the model computes.

**int16 start()** Runs once at `startup()`. One-time setup: load a file, precompute a table.

**int16 execute()** Runs every scheduled step. Read `params` and `inputs`, write `outputs`.

**int16 activate() / int16 deactivate()** Turn the model's per-step execution on and off during a run.

Every lifecycle method returns an `int16` error code: `NO_ERROR` (zero) means success, and any non-zero value is treated by the engine as a fault.

## 15.2 A Worked Example: Slope-Intercept

The workspace ships with one example model, `SlopeIntercept`, which implements  $y = mx + b$ . It is deliberately trivial so that the structure is visible.

### 15.2.1 The header — `src/models/SlopeIntercept.h`

The header is the model's complete interface, and it is what serves as the model's documentation.

```
/*
Metadata for the WarpTwin GUI:
imdata = {"displayname" : "Slope Intercept Model",
          "exclude" : False,
```

```

        "category" : "Custom"}
aliases = {"m" : "Multiplier", "b" : "Offset",
          "x" : "Input",      "y" : "Output"}
*/

#ifdef MODELS_SLOPE_INTERCEPT_H
#define MODELS_SLOPE_INTERCEPT_H

#include "simulation/Model.h"

namespace warptwin {

/** @brief Simple implementation of  $y = m x + b$  */
MODEL(SlopeIntercept)
public:
    START_PARAMS
        /** The multiplier in  $y = m x + b$  */
        SIGNAL(m, double, 1.0)
        /** The offset in  $y = m x + b$  */
        SIGNAL(b, double, 0.0)
    END_PARAMS

    START_INPUTS
        /** The model input each step */
        SIGNAL(x, double, 0.0)
    END_INPUTS

    START_OUTPUTS
        /** The output  $y = m x + b$  */
        SIGNAL(y, double, 0.0)
    END_OUTPUTS

    int16 activate() override;
    int16 deactivate() override;

protected:
    int16 start() override;
    int16 execute() override;
};

} // namespace warptwin

#endif

```

### 15.2.2 The implementation – src/models/SlopeIntercept.cpp

```

#include "SlopeIntercept.h"
#include "simulation/SimulationExecutive.h"

using namespace clockwerk;
using namespace warpos;

```

```
namespace warptwin {

// The four constructors are boilerplate. The only thing worth noting is
// START_STEP in the first two: it sets the schedule slot the model runs in
// by default (Chapter on Simulation Architecture).
SlopeIntercept::SlopeIntercept(Model &pnt, const std::string &m_name)
    : Model(pnt, START_STEP, m_name) {}
SlopeIntercept::SlopeIntercept(SimulationExecutive &e, const std::string
    ↪ &m_name)
    : Model(e, START_STEP, m_name) {}
SlopeIntercept::SlopeIntercept(Model &pnt, int schedule_slot, const
    ↪ std::string &m_name)
    : Model(pnt, schedule_slot, m_name) {}
SlopeIntercept::SlopeIntercept(SimulationExecutive &e, int schedule_slot,
    ↪ const std::string &m_name)
    : Model(e, schedule_slot, m_name) {}
SlopeIntercept::~SlopeIntercept() {}

int16 SlopeIntercept::activate() {
    active(true);
    execute();           // so outputs reflect inputs immediately
    return NO_ERROR;
}

int16 SlopeIntercept::deactivate() {
    active(false);
    outputs.y(0.0);
    return NO_ERROR;
}

int16 SlopeIntercept::start() {
    // One-time setup. Nothing to do for this model.
    return NO_ERROR;
}

int16 SlopeIntercept::execute() {
    outputs.y(params.m() * inputs.x() + params.b());
    return NO_ERROR;
}

} // namespace warptwin
```

**Warning.** Put Doxygen @brief/@param comments in the header, not the .cpp. The header is what is delivered as documentation and what the GUI and the Doxygen reference read.

## 15.3 The Workspace

Clone the workspace from [github.com/warpware-distribution/warptwin-custom](https://github.com/warpware-distribution/warptwin-custom). It is laid out as:

---

|                 |                                                                                                                                                                                                                |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/models/     | Your custom models – one .h and one .cpp per model. SlopeIntercept is here as the example.                                                                                                                     |
| test/           | GoogleTest cases. test_SlopeIntercept.cpp and testtools.hpp show the pattern.                                                                                                                                  |
| swig/           | SWIG interface files. WarpTwinPy.i re-exports WarpTwin's Python types so your wrapped model can exchange CartesianVector3, Quaternion, Time and the rest with the rest of a simulation. You rarely touch this. |
| python/scripts/ | Example simulation scripts, including ones that use a custom model.                                                                                                                                            |
| CMakeLists.txt  | The build. Compiles src/ into a shared library, links it against the installed WarpTwin SDK, builds the tests, and auto-generates the Python bindings and the GUI metadata.                                    |
| .devcontainer/  | A VS Code dev-container definition, the smoothest way to get a working environment.                                                                                                                            |

---

Building the workspace (via CMake, typically cmake then make in a build/ directory – see the repository README for the current steps) produces three things:

1. **A C++ shared library** of your models, linked against the WarpTwin SDK, that your GoogleTest cases run against.
2. **A Python package** – your models, SWIG-wrapped, importable alongside the built-in warptwin package.
3. **custom\_models.json** – GUI metadata generated from the imdata/aliases comment blocks in your model headers.

## 15.4 Testing a Custom Model

Tests use GoogleTest and run without a full simulation – construct the model on an executive, set params and inputs, step, and assert on outputs:

```
#include <gtest/gtest.h>
#include "models/SlopeIntercept.h"
#include "simulation/SimulationExecutive.h"

using namespace warptwin;

TEST(SlopeIntercept, ComputesLine) {
    SimulationExecutive exc;
    SlopeIntercept si(exc, "si");

    si.params.m(2.0);
    si.params.b(3.0);
    si.inputs.x(5.0);

    exc.startup();
    exc.step();

    EXPECT_DOUBLE_EQ(si.outputs.y(), 13.0);    // 2*5 + 3
}
```

Run them with the workspace's test target (make test, or the CTest integration in VS Code).

## 15.5 Using a Custom Model in Python

Once built, a custom model is imported and used exactly like a built-in one. With the workspace build directory on your PYTHONPATH:

```
from custom.SlopeIntercept import SlopeIntercept
from warptwin.WarpTwinPy import SimulationExecutive

exc = SimulationExecutive()
si = SlopeIntercept(exc, "si")
si.params.m(2.0)
si.params.b(3.0)
si.inputs.x(5.0)

exc.startup()
exc.step()
print(si.outputs.y())          # 13.0
```

It connects to other models with `connectSignals` and is logged like any other model (Chapters 6 and 11).

## 15.6 Using a Custom Model in the GUI

The `imdata` block in the model header is what the GUI reads:

---

|                          |                                                                          |
|--------------------------|--------------------------------------------------------------------------|
| <code>displayname</code> | The name shown on the block.                                             |
| <code>category</code>    | Which palette group the block appears under. Custom models use "Custom". |
| <code>exclude</code>     | True hides the model from the GUI entirely.                              |

---

The optional `aliases` block renames signals for display — `m` shown as "Multiplier", and so on — without changing the names you use in code. Launch the GUI pointed at the generated metadata:

```
wt-gui --custom-model-file=custom_models.json
```

Your model then appears under the **Custom** category, and a simulation built with it exports to a script just like any other (Chapter 5).

# 16 Flight Software in the Loop

WarpTwin runs flight software under simulated time, against the simulated environment and sensor models, so the code that will fly is exercised long before it reaches hardware. The flight code itself is real and unmodified; only its inputs — sensor measurements, time, commands — and its outputs — actuator demands, telemetry — come from and go to the simulation instead of a vehicle.

There are two ways to bring flight software into a run, and they are a progression from the last chapter's custom models:

**WarpOS, compiled in** WarpWare's flight-software framework has a ready-made integration. The WarpOS libraries are part of your WarpTwin delivery — there is nothing extra to install or build — and every WarpOS app compiled into that delivery appears in a WarpTwin script as an importable class, wired to sensor and actuator models with `connectSignals` like any other unit. This is the supported, canonical path and most of this chapter is about it.

**Your own flight software, out of process** In-house flight code — any framework, any language, on this machine or another — is driven through WarpTwin's external interfaces. The simulation exchanges sensor data, time, commands, and telemetry with your software over a socket or a shared-memory segment, so your flight computer or software-in-the-loop process runs against the simulated vehicle. Section 16.5 covers this.

Everything here builds on Chapter 7's distinction between a *model* and an *app*, and on Chapter 6's "Guidance, Navigation, and Control" section, which frames flight-software-in-the-loop as the verification step after a control law has been prototyped in Python and then settled into a model.

## 16.1 WarpOS Apps as Models

Every WarpOS app compiled into your WarpTwin delivery is exposed to Python as its own class in the `warptwin` package, exactly like a built-in model. You import and construct it the same way:

```
from warptwin.PdAttitudeControl import PdAttitudeControl

att = PdAttitudeControl(exc, "att")    # a WarpOS app, on the sim executive
```

The app carries the same three signal groups as a model — `params`, `inputs`, `outputs` — so it wires into the simulation with `connectSignals` with no special handling. Feed its navigation inputs from a sensor model and route its actuator outputs to a node on the body:

```
from warptwin.WarpTwinPy import connectSignals, Node, Quaternion,
    ↪ CartesianVector3
from warptwin.FrameStateSensorModel import FrameStateSensorModel

# Truth attitude/rate of the body relative to a reference frame
fs = FrameStateSensorModel(exc, "fs")
fs.params.target_frame_ptr(sc.body())
fs.params.reference_frame_ptr(earth.outputs.inertial_frame())

connectSignals(fs.outputs.att_tgt_ref, att.inputs.act_quat)
connectSignals(fs.outputs.omega_tgt_ref__out, att.inputs.act_omega__act)
att.inputs.cmd_quat(Quaternion([1.0, 0.0, 0.0, 0.0]))
```

```
# The app's commanded body torque, applied at a node
att_node = Node("att_node", sc.body())
att_node.moment_frame(sc.body())
connectSignals(att.outputs.cmd_torque__body, att_node.moment)
```

### 16.1.1 An app must be scheduled

This is the one behavioral difference from a model, and it matters. A model places itself in the schedule when it is constructed. An app does not run at all until you register it (Chapter 7, “Models and Apps”). There are two ways to do that.

**Directly in the sim schedule.** `exc.registerApp(app, slot)` runs the app once per simulation step, in the step phase you name — normally `START_STEP` for a sensing or estimation app and `END_STEP` for a control law whose command is picked up on the following step:

```
from warptwin.WarpTwinPy import END_STEP

exc.registerApp(att, END_STEP)
```

**Through a simulated table scheduler**, described next, when you need the app to run on the same deterministic sub-step cadence it will have in flight.

Registering the same app twice is ignored with a warning. An app you construct but never schedule is still built, addressable, and loggable — it simply never executes.

**Tip.** The GN&C apps that ship in the WarpTwin delivery include the `PdAttitudeControl` law, the `SingleAxisPointingControl` and `TwoAxisPointingGuidance` pointing apps, `UnifiedPoweredFlightGuidance` for powered ascent, and the inertial- and spacecraft-navigation EKF suites. Each is imported as `from warptwin.Name import Name`. The header shipped with each app is its full interface reference.

## 16.2 Deterministic Scheduling with the Table Scheduler

Real flight software does not run “once per step.” It runs on a fixed-rate table: a frame of numbered time slots, each slot holding an app that must execute in that slot, in that order, every frame. `SimTableScheduleModel` reproduces that structure inside a WarpTwin run so the apps under test see the timing and ordering they will see on the vehicle.

```
from warptwin.SimTableScheduleModel import SimTableScheduleModel

table = SimTableScheduleModel(exc, "table")
table.params.table_schedule_rate(1000)           # frame rate, Hz
```

`table_schedule_rate` is the rate the internal table runs at; valid values are 1000, 200, 100, 50, and 10 Hz. If that is faster than the simulation rate the model sub-samples and steps the table in sequence within one sim step; if it is slower, the model waits until the frame is due.

Apps are placed in slots with `registerApp(app, slot)`, where `slot` is the integer slot index within the frame:

```
for i in range(0, 1000, 100):
    table.registerApp(att, i + 0)
    table.registerApp(exc.telemetryManager(), i + 1)
    table.registerApp(exc.commandManager(), i + 2)
```

This runs the control app, then telemetry, then command handling, ten times per one-second frame (every 100th slot of a 1000-slot frame), in a fixed order. `exc.telemetryManager()` and `exc.commandManager()` are themselves apps — scheduling them in the table alongside the flight apps is what makes packets get produced and commands get consumed on the same cadence they would have in flight.

**Note.** Use the table scheduler when the point of the run is to verify timing and task ordering. For a control law you are still shaping, `registerApp` straight into `START_STEP` or `END_STEP` is simpler and the difference will not matter.

## 16.3 Simulated Time

Flight code reads the clock through `os.systemTime()` and `os.navigationTime()`. In a WarpTwin run those calls return values published by the simulation's time manager, not the wall clock:

- **System time** drives `os.systemTime()`. By default it starts at zero at `startup()` and advances by exactly the sim step every step.
- **Navigation time** drives `os.navigationTime()`. By default it mirrors system time.

If the flight software cares about the absolute epoch, or models a coarse-resolution real-time clock, shift and quantize system time:

```
from warptwin.WarpTwinPy import Time

offset = Time()
offset.fromFloatingPoint(1_760_000_000.0) # seconds; must be positive
exc.time().configureSystemTime(offset, 1_000_000) # quantize to 1 ms
```

The offset is added to sim time to form system time; the resolution argument, in nanoseconds, quantizes the result. To point the navigation clock at a different scale — for a GPS-disciplined navigation filter, say — name the source:

```
exc.time().configureNavigationTime("GPS") # or "UTC", "TDB", "SYSTEM"
```

The date the other time scales are derived from is set with `exc.setTime(...)` (Chapter 6).

## 16.4 Telemetry and Commands

In flight you configure which packets downlink and how often. You do the same in the sim, *after* `startup()` — telemetry packets are registered during startup:

```
exc.startup()
```

```
rate = Time()
rate.fromFloatingPoint(0.5)           # every 0.5 s -> 2 Hz
exc.scheduleTelemetry(-1, rate)        # -1 = every registered packet
# exc.scheduleTelemetry(81, rate)      # ...or one APID at a time
```

A negative APID schedules every registered packet at the given rate; a non-negative APID schedules just that one.

To read a field out of an incoming telemetry stream — a packet the flight software emitted, decoded back into a signal — use `getTlmByName` with the lower-case packet name and field name:

```
q0 = exc.getTlmByName("tlm_gnc_pd_att_ctrl_data", "err_quat_0")
if q0 is not None:
    print(q0())           # the decoded value, read like any signal
```

It returns `None` if the packet or field is not defined.

### 16.4.1 Routing to a ground system

To put a real ground station in the loop, point the flight-software telemetry and command paths at WarpLink:

```
exc.setFswWarpLinkInterface("192.168.1.20", 5005, 5006)  # ip, tlm, cmd
exc.setWarpLinkTlmStreamSource("192.168.1.20", 5005)    # tlm stream
↪ back in
```

Port 5005 is the conventional telemetry port and 5006 the command port. The operator-side setup — the WarpLink plugin, the targets, sending commands, watching telemetry — is Chapter 17.

### 16.4.2 Running in real time

For an operator to interact with a run as it happens, cap the scheduler at wall-clock rate by launching the script with `--real-time=true`. The simulation then advances one simulated second per real second, and visuals and the WarpLink connection update live.

## 16.5 Driving Your Own Flight Software

When the flight software is a separate process — a different framework, a different language, or a physically separate flight computer — WarpTwin does not load it. Instead you connect individual simulation signals to an external transport and let your software read and write the other end.

```
# The sim publishes a sensor reading OUT and consumes an actuator command IN
exc.connectExternalSocket(imu.outputs.meas_ang_vel_sf, "OUTPUT",
    ↪ "127.0.0.1", 7010)
exc.connectExternalSocket(thruster.inputs.throttle_cmd, "INPUT",
    ↪ "127.0.0.1", 7011)

# ...or over shared memory, for anything on the same host
exc.connectExternalSharedMemory(exc.time().system_time, "OUTPUT",
    ↪ "/fsw_time")
```

The mode argument is "INPUT" or "OUTPUT" (case-insensitive), stated from the simulation's point of view: OUTPUT means the sim publishes the signal (a sensor value, the clock) and your flight software reads it; INPUT means the sim consumes the signal (an actuator demand, a command) and your flight software writes it. Your software takes the mirror-image role on each connection.

| Transport     | Address                              | Payload / when to use                                                                       |
|---------------|--------------------------------------|---------------------------------------------------------------------------------------------|
| Socket (UDP)  | ip, port — one per signal            | Value as ASCII text. Works across hosts; the natural choice for a separate flight computer. |
| Shared memory | /name — one POSIX segment per signal | Raw native-endian value behind a small sequence header. Lowest latency; same host only.     |

Connected signals are synchronized twice per simulation step — once at the top of the step before any model runs, and once at the end after every model has run — so a command that arrives before a step is acted on that step, and a sensor value is published as soon as it is computed.

**Note.** WarpTwin makes no assumption about the framework or language on the far end; only the wire format has to agree. The WarpLink simulation-control channel (Chapter 17) is one specific use of an INPUT socket: it carries {"address": "...", "value": "..."} JSON datagrams that set signals by their dotted address.

### 16.5.1 Hardware in the loop

The same idea extends to a physical device. A simulated sensor model can be bridged onto the OS device interface a driver expects, so a flight computer reading that device sees simulated data. The shipped `warpwall_demo` example does this for an IMU: an IMU physics model feeds `Icm20948Int`, an interface model that lays the readings out in the chip's registers behind a SPI device path, and the real WarpOS `Icm20948` driver on the other side is none the wiser.

## 16.6 Worked References

### Example 16.1 — Flight software driving a rendezvous

The shipped `rendezvous_warplink_connection` example is the end-to-end reference for this chapter. Two spacecraft perform proximity operations in low Earth orbit; the chaser's attitude is held by the `PdAttitudeControl` WarpOS app.

The flight-software portion is:

1. The app is constructed on the executive; its inputs are wired from a frame-state sensor model and its output torque to a node on the chaser body.
2. A `SimTableScheduleModel` is created, and the control app, `exc.telemetryManager()`, and `exc.commandManager()` are registered into successive slots, ten times per frame, so they run in a fixed order on a fixed cadence.
3. `exc.setFswWarpLinkInterface(...)` points telemetry and commands at a ground station.

4. After `exc.startup()`, `exc.scheduleTelemetry(-1, rate)` downlinks every packet at 2 Hz.
  5. Run with `--real-time=true`, the simulation advances at wall-clock rate and an operator flies it from WarpLink; run without that flag it completes as fast as it can for analysis.
- 

For flight-software apps without a ground system in the loop, the `orbital_insertion` example drives a launch vehicle to orbit with two WarpOS apps – `SingleAxisPointingControl` registered into `START_STEP`, and `UnifiedPoweredFlightGuidance` stepped by hand from the run loop at its own guidance cadence – against a `StochasticNavigation` estimate rather than truth.

# 17 WarpLink Interfacing

WarpLink is WarpWare’s ground-operations platform, built on a customized instance of OpenC3 COSMOS. It lets an operator drive a running WarpTwin simulation the way they would drive a spacecraft: setting simulation inputs live while a run is in progress, and – when flight software is in the loop (Chapter 16) – sending it commands and receiving its telemetry.

WarpLink is delivered as a COSMOS plugin, `openc3-cosmos-warplink`. The plugin ships no tools of its own; it is a set of target definitions, interfaces, and a few Python helpers. Installing it gives operators command and telemetry for each flight-software build in the mission, plus the *simulation control channel* that is the subject of most of this chapter.

```
openc3-cosmos-warplink/
plugin.txt             # targets, interfaces, and all tunable settings
targets/
  common/              # template target, copied when adding a build
  WARP_CUBE/           # a flight-software build target
  CONSTELLATION/       # a flight-software build target
  SIM/                 # simulation value injection -- not a build
```

Every target gets its own UDP port pair and its own interface – a hard rule, because COSMOS identifies an incoming packet only among the targets mapped to the interface that received it. The flight-software targets are covered in the WarpOS User’s Guide (“Ground Interfacing with WarpLink”); the SIM target is the one that matters here, and it is deliberately unlike the others.

## 17.1 The Simulation Control Channel

Setting a simulation input while a run is in progress – a spacecraft mass, a drag coefficient, a guidance mode, whatever your script exposes – has nothing to do with flight software. It is not CCSDS, it needs no CRC, and it must not be mixed into the flight command stream. So it gets its own target (SIM), its own interface, and its own UDP port.

What goes on the wire is one UDP datagram carrying exactly one JSON object and nothing else – no CCSDS header, no framing, no CRC, no trailing newline:

```
{"address": ".exc.spacecraft.params.mass", "value": 5.3}
```

The value is sent as its natural JSON type, so the simulation receives a number as a number rather than a quoted string:

| Typed in Sim Control | Sent on the wire | Received as    |
|----------------------|------------------|----------------|
| 5                    | "value":5        | integer        |
| 5.5                  | "value":5.5      | floating point |
| true                 | "value":true     | boolean        |
| SAFE                 | "value":"SAFE"   | string         |

The rule is simple: anything that parses as JSON is sent as that JSON type, and anything else is sent as a string. Plain strings do not need quoting. The value must be a scalar – an object or an array

is rejected on the WarpTwin side.

## 17.2 Addressing Simulation Values

The address field is a dotted path into the simulation's graph tree — the same address scheme used to log a value by address (Chapter 11) and the same one Chapter 7 describes under “Signals and Addresses”:

```
# Logging by address
log.addParameter(".exc.fs.outputs.pos_tgt_ref__out", "rel_pos")

# The same style of address, set live from WarpLink
.exc.spacecraft.params.mass
```

WarpLink does not interpret or validate the address. It passes the string through untouched, so its meaning belongs entirely to your simulation. A misspelled address produces a datagram that leaves perfectly cleanly and is simply never acted on — nothing on the WarpLink side reports it. When in doubt, confirm the address against the running tree with `exc.search("text")` or the `graph_tree.json` written beside the results (Chapter 7).

**Warning.** WarpTwin params are meant to be set before `startup()`; the effect of changing one mid-run is not guaranteed. Anything you intend an operator to change during a run should be reachable as a model *input*, not a param.

## 17.3 The WarpTwin Side

For any of this to land, the simulation has to be listening on the UDP port WarpLink sends to. That is a property of the simulation, not of WarpLink — WarpLink's job ends when the datagram is on the wire.

### 17.3.1 The listener is already there

Every simulation executive owns an external interface that binds the value-injection port automatically at `startup()`. The default port is **5009**, matching the plugin's `sim_write_port` default, so a stock WarpTwin run already accepts Sim Control datagrams with no extra code. On each simulation step the interface drains every datagram waiting on that socket, and for each one:

1. parses the JSON — a malformed datagram is logged and dropped;
2. requires a string `address` and a scalar `value` — anything else is logged and dropped;
3. resolves the address against the simulation tree; an address that does not resolve to exactly one settable signal is logged and dropped;
4. sets the signal, converting the JSON value to the signal's type.

None of these failures stops the step — a bad injection never crashes a run. Successful sets are visible at `LOG_DEBUG`.

To move the listener or turn it off, set the port on the external interface before `startup()` (a value of zero or less disables it):

```
exc.externalInterface().params.sim_cmd_port(5009)    # default; 0 disables
```

### 17.3.2 Binding a specific signal to a specific socket

The automatic listener accepts {address, value} datagrams for any address. When you would rather wire one named signal to one inbound socket — a steady stream of a single value from an external source, say — connect it explicitly:

```
exc.connectExternalSocket(sc.params.mass, "INPUT", "0.0.0.0", 7020)
```

"INPUT" means the simulation reads the signal from the socket; "OUTPUT" publishes it. This is the same mechanism used to drive out-of-process flight software (Chapter 16, Section 16.5); the WarpLink simulation-control channel is the special case of an INPUT socket that carries addressed JSON instead of one bare value.

## 17.4 Flight Software Telemetry and Commands

When flight software runs in the loop (Chapter 16), WarpLink is also the ground system for it: real commands in, real telemetry out, over that build's own flight target. The WarpTwin side is three calls, covered in Chapter 16 and repeated here for context:

```
exc.setFswWarpLinkInterface("192.168.1.20", 5005, 5006)  # ip, tlm, cmd
    ↪ ports
exc.setWarpLinkTlmStreamSource("192.168.1.20", 5005)    # decode stream
    ↪ back in

exc.startup()
rate = Time(); rate.fromFloatingPoint(0.5)
exc.scheduleTelemetry(-1, rate)                          # every packet
    ↪ at 2 Hz
```

After that, `exc.getTlmByName("tlm_packet", "field")` reads a decoded telemetry field back as a signal. Decoding an incoming stream needs the command/telemetry definition file (`cmd_tlm.json`) that ships with your delivery; without it the simulation still sends telemetry but does not parse a return stream.

On the ground side, each flight build is exported to a COSMOS target by an external tool and given its own UDP port pair; the wire format is CCSDS with a CRC and a bit-scanning packet-boundary protocol. That machinery is the WarpOS User's Guide's subject ("Ground Interfacing with WarpLink"); from WarpTwin you only choose the ports and the stream source.

**Note.** One build, one target, one UDP port pair. The SIM target is not a build and never shares a port with one — that separation is the whole reason simulation traffic and flight commands cannot be confused.

## 17.5 Configuring the Connection

Every connection setting in `plugin.txt` is a plugin VARIABLE, so it appears as an editable field in the COSMOS Admin → Plugins install dialog. Nobody edits `plugin.txt` and rebuilds the gem to change a host or a port.

| Variable                    | What it controls                                                                                                                               |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sim_enable</code>     | Whether the SIM target and interface are deployed at all.                                                                                      |
| <code>sim_host</code>       | The machine running the WarpTwin simulation. Defaults to <code>host.docker.internal</code> – the Docker host, seen from the COSMOS containers. |
| <code>sim_write_port</code> | The UDP port the simulation listens on. Defaults to 5009.                                                                                      |

The flight targets carry the equivalent `*_enable`, `*_host`, `*_write_port`, and `*_read_port` variables; each enable flag skips that target entirely, which is how you avoid deploying a build you are not flying.

Connecting a simulation to WarpLink is then three steps:

1. Install the plugin with `sim_enable` true and `sim_host` / `sim_write_port` pointed at the machine running the simulation.
2. Start the simulation (it listens on that port automatically – Section 17.3).
3. Open **Sim Control**, type an address and a value, press Send.

### 17.5.1 Sim Control

Sim Control is a standard WarpLink tool, installed by default alongside Command Sender. It has two fields – simulation address and value – a live preview of the JSON that will be sent, and a Send button; it keeps a session history with a Resend button on each row. Two things are worth knowing:

- It sends through the normal COSMOS command path (the `SIM SET_VALUE` command), not a private socket, so every value an operator sets is logged and shows up in Command Sender history – simulation changes are auditable, which matters when reconstructing a training run.
- If the SIM target is not installed, the tool says so on load and disables its fields, rather than failing on the first Send.

A site that does not want the tool deployed disables it with `OPENC3_NO_SIMCONTROL=1` in `.env`, the same mechanism used for the other default COSMOS tools. The plugin builds and installs like any other:

```
../openc3.sh cli rake build VERSION=1.0.0
../openc3.sh cli load openc3-cosmos-warplink-1.0.0.gem
```

## 17.6 Under the Hood: the SIM Target Definition

You do not need this to use Sim Control, only to extend the channel. The SIM target's `cmd.txt` defines a single command with COSMOS's `JsonAccessor`, which is what makes the packet buffer JSON rather than binary:

```
COMMAND SIM SET_VALUE BIG_ENDIAN "Set a simulation input to a value"
ACCESSOR JsonAccessor
TEMPLATE '{"address":"","value":0}'
APPEND_PARAMETER ADDRESS 2048 STRING "" "Simulation address"
KEY $.address
APPEND_PARAMETER VALUE 64 ANY 0 "Value, sent as its natural JSON type"
KEY $.value
```

TEMPLATE is the starting JSON document and each KEY is a JSONPath naming where that parameter is written into it. VALUE is declared ANY, and that is what preserves the JSON type instead of coercing everything to a string; the declared bit sizes are required by the syntax but carry no meaning for a JSON packet. The interface is write-only:

```
INTERFACE SIM_INT openc3/interfaces/udp_interface.py <sim_host>
  ↪ <sim_write_port> None None None 128 10.0 None
MAP_TARGET SIM
```

None in the read-port position marks the interface unreadable, so COSMOS never opens a listening socket for it and no CRC or packet-boundary protocol is attached. SIM is a command-only target with no telemetry. Unlike the generated flight-target files, SIM/cmd\_tlm/cmd.txt is hand-written and safe to edit.

## 17.7 Troubleshooting the Channel

**UDP is fire and forget.** A successful Send means WarpLink put the datagram on the wire — not that the simulation received it, and not that it accepted it. There is no acknowledgment by design: the interface has no read port.

**Confirm the datagram is leaving.** Packet-capture the sim port:

```
sudo tcpdump -i any -A udp port 5009
```

On Docker, -i any shows each datagram twice — once on the container veth and once on the bridge. That is a capture artifact, not a double send.

**Confirm something is listening** on the simulation host:

```
ss -ltnp | grep 5009
```

This is what separates “sent into the void” from “delivered” — tcpdump shows the datagram whether or not anything is bound.

**Check the address.** Nothing in WarpLink validates it. Confirm it against the running tree with exc.search(), and check the simulation log for a “not found” or “not a settable signal” warning.

**Check the value type.** If the simulation reports a type-conversion failure, compare what you typed against the typing table above — a bare word is sent as a string.

# A Command-Line Arguments and Environment Variables

Any WarpTwin script that calls `exc.parseArgs(sys.argv)` accepts the arguments below. Pass each one as `--name=value` or as `--name value`.

A script can override a built-in default, or add an argument of its own, with `addDefaultArgument` — called *before* `parseArgs` — and read any argument back afterwards with `get`:

```
exc.args().addDefaultArgument("end", 3600)      # change a default
exc.args().addDefaultArgument("my_flag", 0)     # add your own
exc.parseArgs(sys.argv)
my_flag = exc.args().get("my_flag")
```

## A.1 Built-in Script Arguments

| Argument           | Default  | Effect                                                                                                                                                                                                             |
|--------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| --end              | 10000    | Simulation end time, in seconds of simulation time.                                                                                                                                                                |
| --run              | 0        | Monte Carlo run number. 0 is the nominal, undispersed run; any other value draws the dispersions for that run (Chapter 13).                                                                                        |
| --out-dir          | results/ | Directory for log files, <code>sim_data.json</code> , and <code>graph_tree.json</code> . Created if absent.                                                                                                        |
| --rng-seed         | 0        | Seed for the dispersion random-number generator. Changes the draw for every run number.                                                                                                                            |
| --write-data-json  | true     | Write <code>sim_data.json</code> (models as configured) and <code>graph_tree.json</code> (the address map) into the output directory. Set false to suppress both.                                                  |
| --save-all-outputs | false    | Log every output signal in the simulation to <code>all_outputs.csv</code> . Produces very large files.                                                                                                             |
| --dispersions-file | ""       | If set, write the dispersion definitions and their drawn values to this file (extension <code>.adoc</code> , <code>.csv</code> , or <code>.txt</code> ). Equivalent to <code>exc.writeDispersionFile(...)</code> . |
| --enable-visuals   | false    | Enable the 3D viewer. Same as calling <code>exc.enableVisuals()</code> in the script.                                                                                                                              |
| --real-time        | false    | Pace the run so it does not outrun real time, for live viewing.                                                                                                                                                    |

### Example A.1 — Common invocations

```
python3 script.py --end=86400                # run for one day
python3 script.py --enable-visuals=true --real-time=true
python3 script.py --run=7 --out-dir=mc/run_007 # one Monte Carlo case
```

```
python3 script.py --write-data-json=false # logs only, no JSON
```

## A.2 Environment Variables

The simulation engine itself is not configured through environment variables — everything is a script argument (above) or a model param. The variables below affect the tools around it.

|                                           |                                                                                                                                                                                                                              |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DISPLAY, WAYLAND_DISPLAY                  | A graphical <code>wt-gui</code> needs one of these set. Unset means headless: the GUI does nothing (the engine, the Python API, and MultiBodyViz are unaffected — the viewer runs in a browser, not on the simulation host). |
| MPLBACKEND                                | Matplotlib backend. Set it to Agg for plot-to-file analysis on a headless machine.                                                                                                                                           |
| MBV_VISUALS_PORT, MBV_VISUALS_STREAM_PORT | Override the MultiBodyViz HTTP port (default 8002) and live-stream port (default 9002) when the viewer harness is started by hand. <code>warptwin-visuals --port</code> does the same for the HTTP port.                     |

## A.3 Command-Line Tools

|                                                                     |                                                                                                          |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>wt-gui [--custom-model-file=f.json]</code>                    | The graphical simulation builder.                                                                        |
| <code>warptwin-doctor [--full] [--verbose]</code>                   | Installation self-check. <code>--full</code> also runs a simulation; <code>--verbose</code> adds detail. |
| <code>warptwin-license status</code>                                | Show the installed/embedded license.                                                                     |
| <code>warptwin-license install file.enc</code>                      | Install a license (usually with <code>sudo</code> ).                                                     |
| <code>warptwin-visuals [visuals.json] [--no-browser]</code>         | Serve the 3D viewer on port 8002.                                                                        |
| <code>warptwin-docker.sh {shell,doctor,run,gui,visuals,load}</code> | Container launcher (Chapter 3).                                                                          |

# B Troubleshooting

---

Run `warptwin-doctor` first — it names the failing check and what to do about it. `warptwin-doctor -v` adds the underlying detail; that is the output to send to [support@warppware.co](mailto:support@warppware.co) when you need help.

## B.1 Installation

| Symptom                                                  | Cause and fix                                                                                                                                                                                  |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>install.sh</code> says “wrong Ubuntu release”      | The native packages are built per Ubuntu release because the Python extensions and HDF5 linkage are ABI-specific. Use the build that matches your release, or the container image (Chapter 3). |
| <code>apt</code> cannot install the packages             | Almost always no network access to the Ubuntu archive for dependencies. Pre-stage the dependency list from the delivery’s <code>README.md</code> on an air-gapped machine.                     |
| <code>ImportError</code> on <code>import warptwin</code> | Run <code>sudo ldconfig</code> , then <code>warptwin-doctor</code> .                                                                                                                           |
| macOS: “unidentified developer” on double-click          | An unsigned package fetched via a browser. Right-click the <code>.pkg</code> and choose <i>Open</i> , or install with <code>./install.sh</code> .                                              |

## B.2 Running Simulations

| Symptom                                                | Cause and fix                                                                                                                                                                           |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The simulation exits immediately, before any output    | Missing or expired license. Check <code>warptwin - license status</code> (Section 3.7).                                                                                                 |
| A spacecraft is inside the planet / the orbit is wrong | Units. Semimajor axis is meters; inclination, RAAN, argument of perigee, true anomaly are radians (use <code>DEGREES_TO_RADIANS</code> ).                                               |
| AttributeError: ...configFromPlanet                    | There is no such method. Set the parameter: <code>sc.params.planet_ptr(earth)</code> (Chapter 6).                                                                                       |
| A parameter change “does nothing”                      | <code>params</code> must be set <i>before</i> <code>exc.startup()</code> ; initial states must be set <i>after</i> .                                                                    |
| An input signal never changes                          | It was set once with a value rather than connected. Use <code>connectSignals(upstream.outputs.x, downstream.inputs.x)</code> (Chapter 6).                                               |
| Nothing is logged / wrong columns                      | <code>addParameter</code> was called after <code>addLog</code> , or the address string is misspelled. Check with <code>exc.search(...)</code> or <code>results/graph_tree.json</code> . |

## B.3 Visuals

| Symptom                                         | Cause and fix                                                                                                                                                           |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The viewer shows nothing                        | It waits until the run finishes and its HDF5 files close. For a live view, use <code>--real-time=true</code> .                                                          |
| <code>wt-gui</code> does nothing when launched  | No graphical session. It needs <code>DISPLAY</code> or <code>WAYLAND_DISPLAY</code> ; under the container image, an X server (Chapter 3).                               |
| Browser cannot open <code>localhost:8002</code> | Under the container image, the port must be published – use the <code>warptwin-docker.sh</code> launcher, which does this. Otherwise another process may hold the port. |
| The viewer is blank in the browser              | The browser is not WebGPU-capable, or WebGPU is disabled.                                                                                                               |